

MSc Data Science Professional Team Project

7PAM2033

GROUP FINAL REPORT

Project Title

*ESG, Financial Performance and Time-Series Image (GAF) Dataset
for Stock Risk Classification*

Team Members

Student name	Student SRN number
Tamanna Pathak	24175478
Ebenezer Onaopepo Okunola	24119439
Oluwapelumi Olaniyi Salako	24162709
Md Al Arafat	23088114
Md Minhajul Hayat Mim	24129906
MD. Sazzad Hosen Khan	24174807

Team GitHub link

<https://github.com/ma24ahv/Group-3-Data-Science-Professional-Team-Project>

Supervisor: *Ralf Napiwotzki*

Date Submitted: 24/07/2026

Word Count: Aprox 8000

DECLARATION STATEMENT

Any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

Human participants were not used in this MSc Project.

I have read the detailed guidance on academic integrity, misconduct and plagiarism at [Assessment Offences and Academic Misconduct](#) and understand the University process for dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

Permission is given for the report to be made available on module websites provided the source is acknowledged.

Declaration of Use of AI

[Delete the Yes or No as appropriate] ☐ **Yes** AI tools have been used in the preparation

of this project and report

- ☐ **No** To generate initial ideas. ☐ **Yes** To develop code. ☐ **No** To provide literature suggestions and summaries. ☐ **No** To provide suggestions to improve the report content. ☐ **Yes** To edit spelling and grammar, and to proofread the report.

- ☐ **No** Other (provide details):

Note: Copying text generated by AI into the report is academic misconduct.

Student name	Student SRN number	Signature	Date
Tamanna Pathak	24175478	T.P	23/07/2026
Ebenezer Onaopepo Okunola	24119439	E.O	23/07/2026
Oluwapelumi Olaniyi Salako	24162709	O.S	23/07/2026
Md Al Arafat	23088114	M.A	23/07/2026
Md Minhajul Hayat Mim	24129906	MIM	23/07/2026
MD Sazzad Hosen Khan	24174807	S.K	23/07/2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor, **Ralf Napiwotzki**, for his guidance, support, and valuable feedback throughout the duration of this project. His expertise and encouragement were instrumental in shaping the direction and quality of our work.

Datasets

We gratefully acknowledge the following publicly available datasets used in this project:

1. **S&P 500 ESG Risk Ratings** — sourced from Kaggle (<https://www.kaggle.com>). This dataset is made available under the Kaggle Dataset Terms of Service and is used here solely for academic research purposes. The original data is attributed to Sustainalytics.
2. **S&P 500 ESG Metrics (Yahoo Finance)** — sourced from Kaggle. Data originally retrieved from Yahoo Finance via the `yfinance` Python library (Apache License 2.0). Used here for non-commercial academic research only, in accordance with Yahoo Finance's terms of service.
3. **S&P 500 Companies Financial Description** — sourced from Kaggle. Financial data originally sourced from Finviz (<https://finviz.com>). Used here for non-commercial academic research purposes only.
4. **S&P 500 Stock Prices — 5 Years** — sourced from Kaggle (<https://www.kaggle.com/datasets/camnugent/sandp500>). This dataset is made available under CC0: Public Domain licence and may be freely used, shared, and adapted without restriction.

Software Tools and Libraries

This project was developed using the following open-source software tools and libraries, whose licences are acknowledged below:

Python (Python Software Foundation License). **TensorFlow** (Apache License 2.0). **Keras** (Apache License 2.0). **XGBoost** (Apache License 2.0). **scikit-learn** (BSD 3-Clause License). **pandas** (BSD 3-Clause License). **NumPy** (BSD 3-Clause License). **Matplotlib** (Matplotlib License, BSDcompatible). **SHAP** (MIT License). **Optuna** (MIT License). **Streamlit** (Apache License 2.0). **yfinance** (Apache License 2.0). **GitHub** (used for version control and collaborative development under standard GitHub Terms of Service).

All software tools and libraries listed above are used in compliance with their respective licences for non-commercial academic research purposes only.

Abstract

The ESG (environmental, social, and governance) performance of companies is becoming an important consideration in financial risk assessment. However, most risk models only take financial statistics into account. This project combines ESG scores, fundamental financial ratios, and stock price time series to classify firms in the S&P500 index as low, medium, and high-risk entities. Four publicly available datasets were combined into a single dataframe containing 426 companies and over 80 ESG, financial, and fundamental domain features. A single year of daily closing prices was converted to Gramian Angular Field (GAF) images for each company in the set, transforming the time series into visual patterns that can be used as image data by convolutional networks. Four different modelling approaches were tested: a custom convolutional neural network, ResNet50 transfer learning architecture, Random Forest regression with Optuna hyperparameter tuning, and a hybrid approach that uses the last layer embeddings from the ResNet50 model concatenated with traditional features as input for an XGBoost classifier. Modelling results showed that the image-based approaches underperformed, likely due to insufficient data, achieving results close to random. The Random Forest model achieved a weighted F1score of 0.81, while the hybrid model demonstrated a weighted F1-score of 0.98, a ROC-AUC of 0.99 and a perfect recall of high-risk companies. ESG scores were identified as the most influential inputs using SHAP analysis for the Random Forest model and the hybrid approach. The trained hybrid model was deployed as a web application using the Streamlit framework for interactive risk assessment of individual companies.

Contents

ACKNOWLEDGEMENTS.....	3
Abstract	4
Contents.....	5
1. Introduction.....	6
2. Data.....	7
2.1 Choice and Background of the Datasets.....	7
2.2 Exploratory Data Analysis.....	8
2.3 Pre-processing and Leakage Avoidance	9
2.4 Feature Engineering	10
2.5 Data Ethics Statement: Four Data Management Plan Questions	11
3. Methodology	12
3.1 Experimental Design	12
3.2 Custom GAF-CNN.....	12
3.3 Random Forest Baseline.....	13
3.4 ResNet50 Transfer Learning	13
3.5 Hybrid ResNet50–XGBoost Model	14
3.6 Training, Validation and Evaluation Metrics	15
3.7 Hyperparameter Tuning and Reproducibility	16
3.8 Application Demonstration and Explainability.....	17
4. Results and Analysis	18
4.1 Evaluation context and experimental evidence.....	18
4.2 GAF representation and image-model evidence	19
4.3 Final model comparison	20
4.4 Model-level interpretation	22
4.5 Error analysis, explainability and application demonstration	23
4.6 Comparison with published research.....	24
4.7 Research question and suitable use cases	24
4.8 Limitations and future work	25
5. Conclusions.....	25
References.....	26
Appendices	27
Appendix A – Complete Project Source Code.....	27
Contents.....	28
Additional Figures.....	115
Repository Structure.....	116
Software Environment.....	116
Project Outputs	116
Appendix D: Team GitHub Repository	117

1. Introduction

The way people think about Environmental, Social and Governance criteria, also known as ESG has changed a lot in the ten years. Now environmental, social and governance criteria are a part of how we assess financial risk. In 2020 the total value of investments that consider environmental, social and governance criteria was over 35 trillion dollars. This number has kept growing. It shows that people understand that things like environmental, social and governance criteria can affect how well a company does in the long run.

With all this growth there is still a lot of debate about how Environmental, Social and Governance scores relate to the risk of investing in stocks. Some researchers, like Friede and his team, looked at over 2,000 studies. They found that about 90% of them said that environmental, social and governance criteria and a company's financial performance are connected in a positive way. However, this connection can vary a lot depending on the industry and region and also factors like who is providing the environmental, social and governance data. Other researchers, like Berg and his team, found that different companies that give Environmental, Social and Governance ratings do not always agree because they look at things and weigh them differently.

One way to measure investment risk is to look at how much the price of a stock goes down. More and more research is showing that how well a company does in terms of environmental, social and governance criteria can affect this volatility in ways that traditional financial measures do not capture. For example, one study found that companies with high Environmental, Social and Governance scores did better during the COVID-19 pandemic. They did not lose much value, unlike other companies. Another study found that investment strategies based on environmental, social and governance criteria can lead to volatility without reducing returns.

These findings suggest that Environmental, Social and Governance data can be useful for predicting risk, not for screening investments after the fact. However, most current approaches to using Environmental, Social and Governance data for risk classification do not take into account the patterns in how stock prices change over time. A different approach is to turn data into images. This can help computers find patterns in the data that're not easy to see with traditional methods. One way to do this is by using something called the Gramian Angular Field transformation. This method turns a series of stock prices into a picture that a computer can analyze. This picture can show trends and patterns in the stock prices that're hard to capture with numbers alone.

Using this method researchers have been able to train computers to recognize patterns in stock prices that can help predict risk. However this usually requires a lot of data, which can be a problem when looking at a number of companies. One solution to this problem is to use a computer model that has already been trained on a lot of data and adapt it for this task. This is called transfer learning. By using a model that has already been trained researchers can get a lot of information from the images of stock prices without needing an amount of data. This information can then be combined with data, like Environmental, Social and Governance scores and financial ratios to make predictions about risk.

It is also important to be able to explain why the computer is making predictions. This is known as explainability. In the industry there are rules that require companies to be able to explain how they make decisions about risk. One way to do this is by using a method called SHapley Additive exPlanations or SHAP. This method can help figure out which factors are driving the predictions and whether any of them might be biased. This can help investors understand what is behind the predictions and make informed decisions. It can also help companies ensure that their decisions, about risk are fair and transparent. Environmental, Social and Governance criteria are a part of this process as they can provide valuable insights into a company's long-term prospects and risks.

Research Question

We wanted to know if a model that combines pictures from a kind of image called a 'Gramian Angular Field' with information about companies can do a better job of figuring out how risky these companies are. This model uses something called ResNet50. It looks at things like how the company treats the environment and how well it does financially. The goal is to put companies into three groups: low risk, medium risk and high risk. We also wanted to know what makes the model think a company is risky or not.

Objectives

1. We needed to put a big set of data that includes information about how companies treat the environment and how well they do financially as well as how their stock prices have changed over the past five years. This data is for the S&P 500 companies. It comes from four different sources.
2. We made pictures from the stock price information Using a special tool called ResNet50 to turn these pictures into numbers that the model can understand.
3. We tried out four models to see which one works best: a simple picture-reading model, a model that looks at lots of different things, a model that uses the special ResNet50 tool and a model that combines different types of information.
4. We tried to make the model that looks at lots of things work better by adjusting its settings, and compared it to the other models.
5. We wanted the best model to be able to guess which companies are high risk at least 60 percent of the time and to have a good overall score of at least 0.55.
6. We used a tool called SHAP to understand why the model makes its decisions and to see if some things are more important than others when it comes to deciding how risky a company is.
7. We made the model into a tool that people can use on the internet, and it needs to be fast and easy to use. The tool should be able to make predictions in less than three seconds, and people should like using it, with an average rating of at least 4 out of 5.

2. Data

2.1 Choice and Background of the Datasets

Four complementary S&P 500 datasets were used because corporate ESG risk cannot be represented adequately by one source alone. The target came from the **S&P 500 ESG Risk Ratings dataset (DS1)**. It contains 503 securities and 15 variables, including environmental, social and governance pillar scores, controversy measures and the provider's categorical ESG Risk Level. It was selected because it supplies an interpretable, externally assigned target. The original levels were consolidated into three classes: *Negligible* and *Low* became **Low**; *Medium* remained **Medium**; and *High* and *Severe* became **High**. Kaggle identifies Pritish Dugar as the publisher and lists the dataset as CC0/public domain.

S&P 500 ESG and Stocks Data 2023–24 (DS2) added a second set of ESG pillar scores, controversy, market capitalisation, beta and standard GICS sector labels. It contains 426 companies and 15 variables. In the supplied file, 421 ratings are dated September 2023, while five observations retain dates between 2021 and 2022. It was chosen to improve sector consistency and add market controls absent from DS1. The Kaggle card names Rikin Zala as publisher and lists GPL 3 as the licence.

Financial Performance of Companies from S&P500 (DS3) contains 496 companies and 72 financial and technical indicators, including valuation, liquidity, leverage, profitability, ownership, analyst expectations, returns and trading activity. These variables were included because ESG risk may be related to company size, operating quality, financing structure and market behaviour. The file is a crosssectional snapshot using trailing-twelve-month, most-recent-quarter and recent trading measures. Its exact extraction date is not documented, so it is treated as historical rather than current information. Kaggle identifies Ilya Ryabov as publisher and CC0 as the licence.

The final source, **all_stocks_5yr.csv (DS4)**, contains 619,040 daily observations for 505 tickers with date, open, high, low, close and volume. The supplied data run from 8 February 2013 to 7 February 2018. It was selected to derive risk-and-return features and construct Gramian Angular Field images. Cam Nugent's Kaggle dataset was updated through February 2018 and collected through the Investor's Exchange API; it is licensed CC0.

Ticker symbols were standardised and used as the joining key. An inner join between DS1 and DS2 produced 426 companies with both a valid target and core ESG variables. DS3 and DS4 features were then left-joined so that firms were not removed solely because an optional financial measure was unavailable. The merged tabular data contained 426 rows and 88 columns before final feature selection. A separate multimodal subset contained 364 companies with at least 252 valid trading days and a successfully generated GAF image.

2.2 Exploratory Data Analysis

The final target distribution was **Low = 191 (44.8%)**, **Medium = 182 (42.7%)** and **High = 53 (12.4%)**. The GAF subset was similar: Low = 155 (42.6%), Medium = 162 (44.5%) and High = 47 (12.9%). Thus, Low and Medium were nearly balanced, but High risk represented only around one firm in eight. Figure 2.6.1 illustrates the imbalance and justifies stratified splitting and class-weighted learning.

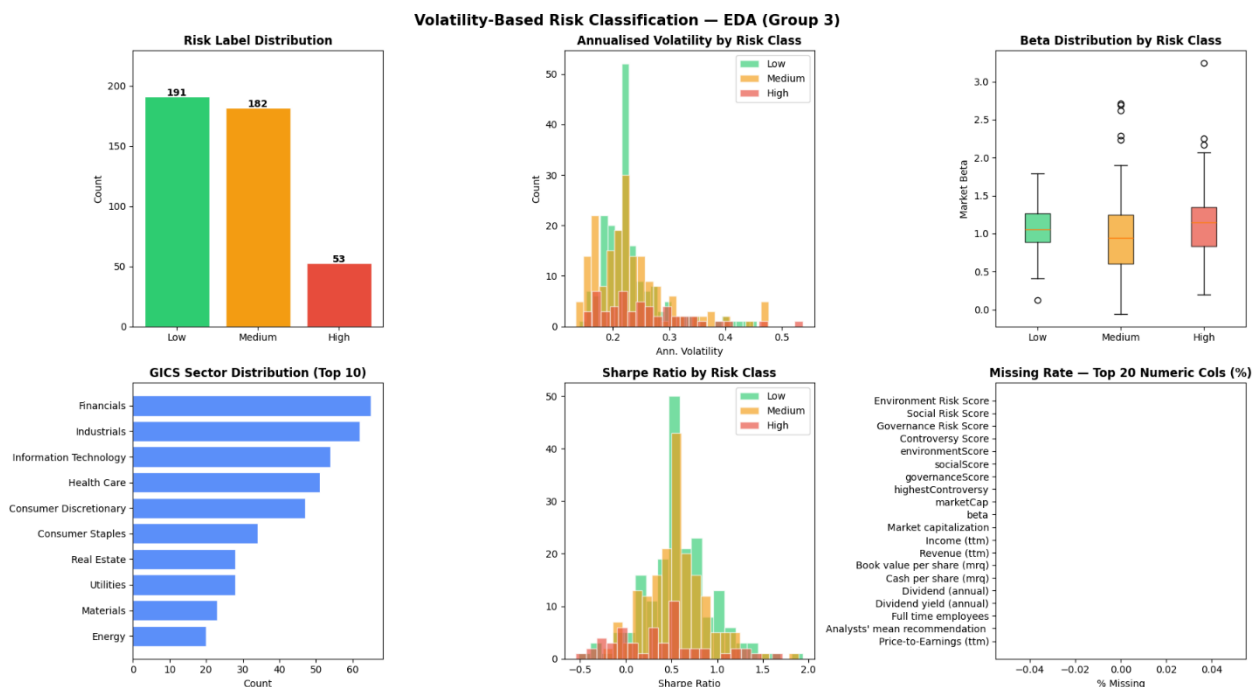


Figure 2.6.1. EDA SUMMARY

Sector coverage was broad. The largest GICS sectors were Financials (65), Industrials (62), Information Technology (54), Health Care (51) and Consumer Discretionary (47); Communication Services was the

smallest sector, with 14 companies. Market capitalisation was strongly right-skewed: the median was approximately USD 40.3 billion, compared with a mean of USD 109.4 billion and a maximum of USD 3.296 trillion. Beta had a mean of 1.036 and a median of 1.030.

For the engineered price variables, median six-month momentum was 0.082, median one-year momentum was 0.149, mean annualised volatility was 0.235, median Sharpe ratio was 0.548 and median maximum drawdown was 0.306. The substantial differences in magnitude, range and skewness supported the later standardisation of numeric variables.

Missingness was assessed before imputation. DS1 contained 73 firms without ESG scores or labels and 100 missing controversy scores; records without a valid target were excluded. DS2 contained no missing cells. DS4 had 27 missing open, high or low values, while observations lacking closing price were removed before time-series calculations. DS3 mainly represented unavailable information using “-” rather than formal null values.

After these placeholders were converted, the merged data contained **2,273 missing cells across 72 columns**, and 293 of the 426 firms had at least one missing value. The highest missingness rates were Price-to-Earnings-to-Growth at 23.5%, Quick Ratio at 21.1%, Current Ratio at 21.1%, Price-to-FreeCash-Flow at 20.9%, annual dividend and dividend yield at 20.2% each, and Gross Margin at 17.1%. No feature exceeded the predetermined 50% removal threshold.

For exploratory purposes only, the target classes were ordinally represented as Low = 0, Medium = 1 and High = 2. Consequently, the modelling task should be described as reproducing the data provider’s ESG risk categorisation rather than creating an entirely independent definition of ESG risk.

2.3 Pre-processing and Leakage Avoidance

Ticker strings were converted to uppercase, stripped of unnecessary whitespace and de-duplicated before merging. Trading dates were converted into datetime format, after which observations were sorted chronologically within each ticker.

The DS3 financial dataset required extensive format conversion. Commas and percentage symbols were removed, while B, M and K suffixes were converted into their corresponding billion, million and thousand multipliers. Binary Yes/No values were converted to 1/0, and “-” placeholders were converted into missing values. A financial column was retained only when more than 30% of its observations could be parsed numerically. This process produced 65 usable DS3 financial features.

The GAF-eligible subset was divided using a fixed random seed of 42 and stratification by risk class. The training set contained 254 companies, representing 70% of the observations; the validation set contained 55 companies, representing 15%; and the test set contained 55 companies, representing the remaining 15%. The Low, Medium and High counts were 108/113/33 in training, 23/25/7 in validation and 24/24/7 in testing. The validation partition was used for hyperparameter selection, early stopping and decision-threshold tuning, whereas the test partition remained untouched until final model evaluation.

All preprocessing parameters must be estimated using training data only. Numeric features were therefore imputed using training-set medians and standardised using training-set means and standard deviations. The fitted imputer and scaler were then applied unchanged to validation and test observations. This prevents information about the held-out distributions from influencing the training process.

Categorical missing values were filled with the most frequent training-set category. The variables were then one-hot encoded, with previously unseen categories ignored during validation or testing. GICS

Sector and the alternative Sector variable were retained because each contained only 11 categories. GICS Sub-Industry and Industry contained 119 and 110 categories respectively and were excluded from the basic one-hot pipeline because they would create a highly sparse feature matrix relative to only 254 training observations.

Identifiers, file paths and administrative variables such as ticker, GAF path, image availability and number of available trading days were excluded from the predictive feature set. Direct or near-direct sources of the target were also removed. These included Total ESG Risk score, totalEsg, ESG Risk Level, ESG Risk Percentile, percentile and overallRisk. Retaining these variables would allow the model to reconstruct the label with minimal genuine prediction and would produce misleadingly optimistic results.

One leakage correction is required in the supplied notebook. The preliminary median fill applied to the complete merged dataset should be removed before train-validation-test splitting. Otherwise, validation and test observations indirectly influence the calculated median values, even though another trainingonly imputer is fitted later. The defensible sequence is therefore to merge the raw data, create the partitions and only then fit imputation, scaling and encoding on the training partition. Company-level splitting further ensured that records or images from the same firm could not be distributed across multiple partitions.

2.4 Feature Engineering

Six price-based features were calculated separately for each ticker using chronologically ordered closing prices. These were annualised log-return volatility, six-month momentum using approximately 126 trading days, one-year momentum using approximately 252 trading days, annualised Sharpe ratio, maximum drawdown and the total number of available trading days.

Log returns were used in the volatility and Sharpe calculations because they are additive over time and less dependent on the absolute price level of a stock. Six-month and one-year momentum represented medium-term and longer-term price direction. Maximum drawdown captures the severity of historical losses, which cannot be represented completely by volatility because volatility treats upward and downward movements similarly.

For the image-based pathway, the final 252 closing prices for each eligible company were linearly subsampled to 64 observations. The resulting sequence was min-max transformed to the interval $([1,1])$, converted into angular values using the arccosine function and mapped into a **64 × 64 Gramian Angular Summation Field**, defined as:

$$GASF(i, j) = \cos(\phi_i + \phi_j)$$

This transformation represents pairwise temporal relationships as two-dimensional textures suitable for convolutional neural networks and ResNet transfer-learning models.

Separate lag-1, lag-5 or lag-20 tabular columns were not added because the tabular unit of analysis was one company rather than one company-day. Temporal dependence was instead represented through the fixed 252-day sequence and its GAF image. This avoided producing a very wide and highly collinear lag matrix from a relatively small company-level sample.

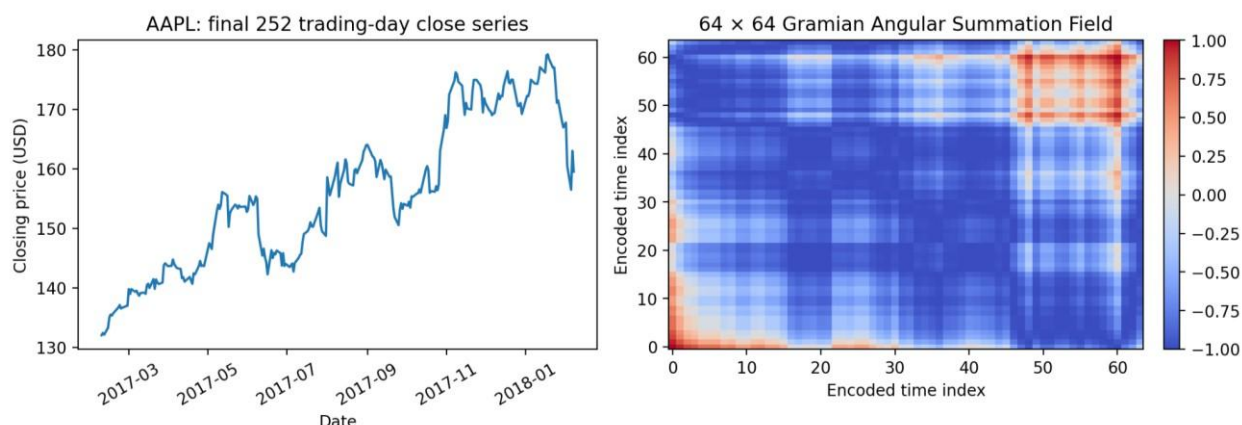


Figure 2.6.4. Example conversion of a 252-day closing-price series into a 64×64 GAF image.

Class imbalance was addressed only during model training through class weights calculated from the training labels and balanced weighting in the Random Forest model. The validation and test distributions were not resampled. This preserved the realistic class prevalence and prevented duplicated or synthetic High-risk observations from entering the held-out evaluation data.

2.5 Data Ethics Statement: Four Data Management Plan Questions

Question 1: Were the data obtained lawfully and with appropriate permission?

All four sources are publicly downloadable Kaggle datasets. DS1, DS3 and DS4 are listed under CC0, while DS2 is listed under GPL 3. The original sources, publishers and applicable redistribution conditions must therefore be retained in the project documentation and any shared derivative data. No authentication controls, subscriptions or technical restrictions were bypassed during collection.

Question 2: Does the data contain personal, sensitive or confidential information?

The analytical observations describe publicly listed corporations and publicly observed market activity rather than private individuals. DS1 includes company addresses, descriptions and employee counts; however, addresses and free-text descriptions were excluded from modelling because they were unnecessary for the research objective. No customer records, employee-level information, account details, protected personal characteristics or confidential transactions were processed.

Question 3: What biases or potential harms could arise from the data or model?

The sample is limited to large publicly listed companies in the United States and is not representative of small businesses, private companies or organisations operating in other countries. ESG ratings depend on the methodology of the original provider, the amount of information disclosed by each firm and sector-specific exposure to environmental or social risks. Missing ESG data may disproportionately exclude firms with weaker disclosure practices. The High-risk class is comparatively small, while the historical stock dataset may also contain survivorship bias because it reflects a particular set of S&P 500 constituents. Model predictions should therefore support academic analysis and should not be interpreted as financial advice or used automatically for investment, lending or regulatory decisions.

Question 4: How will the data be stored, accessed, retained and shared?

Raw source files should be stored as read-only files within a restricted project directory. Processed datasets, split indices, fitted preprocessing objects, software versions and random seeds should be versioned separately to support reproducibility. Write access should be limited to authorised project members. Publicly shared outputs should contain only variables necessary to reproduce the analysis and should avoid republishing company addresses or unnecessary descriptive text. Source dates, licences, limitations and preprocessing decisions should accompany every shared dataset or model artefact. The

data should be securely archived or deleted after the institutional assessment and reproducibility retention period has ended.

3. Methodology

3.1 Experimental Design

The study used a comparative multimodal design to predict Low, Medium and High ESG risk. Tabular ESG, financial and engineered price variables formed one representation; a 64×64 Gramian Angular Summation Field (GASF) generated from the latest 252 closing prices formed the other. Four routes were evaluated: a custom GASF-CNN, a tabular Random Forest, ResNet50 transfer learning, and a hybrid that fused tabular variables with ResNet50 embeddings before XGBoost classification. Figure 2.7.2 summarises the workflow.

GASF converts a time series into pairwise angular relationships while retaining temporal ordering, allowing image models to detect price-shape and volatility textures beyond summary statistics. Wang and Oates (2015) introduced Gramian Angular Fields to connect time-series and computer-vision methods. Fusion was justified because ESG-financial variables describe company condition, whereas GASF represents temporal market behaviour, consistent with multimodal representation-and-fusion principles described by Baltrušaitis, Ahuja and Morency (2019).

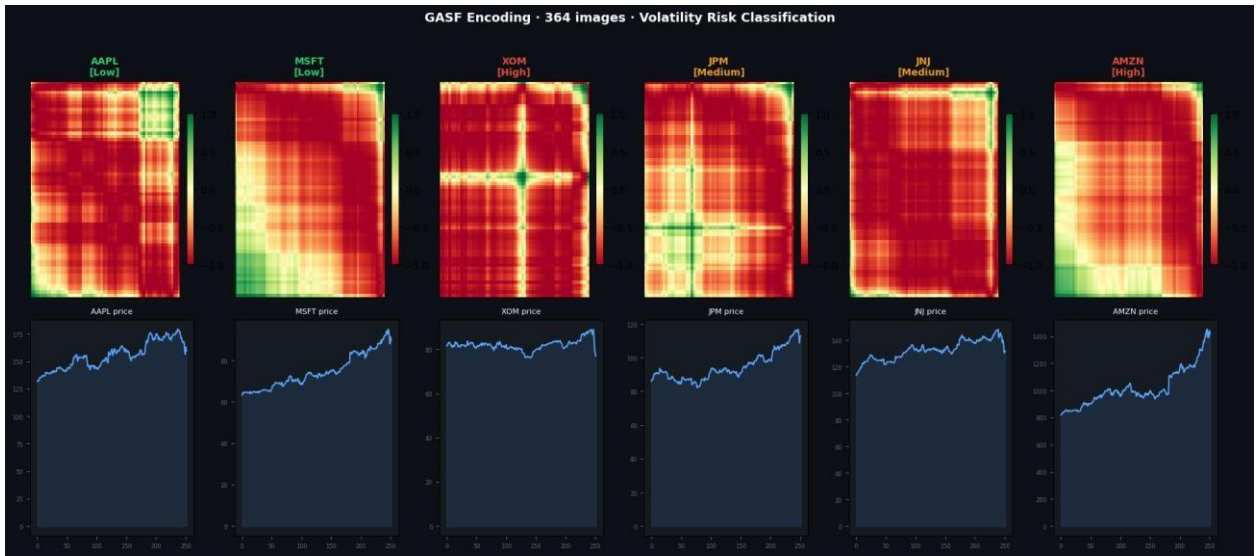


Figure 2.7.1. 252-day stock-price sequences & corresponding 64×64 GASF representation.

3.2 Custom GAF-CNN

The first image baseline accepted an input tensor of $64 \times 64 \times 1$. It contained three convolutional blocks. Each block used two 3×3 convolutions with same padding, followed by batch normalisation and ReLU activation. The blocks used 32, 64 and 128 filters respectively. Each block ended with 2×2 max pooling and dropout of 0.15. The final feature map was reduced by global average pooling to a 128-dimensional embedding. The classification head contained a 256-unit dense layer, batch normalisation, ReLU and dropout of 0.30, followed by a 128-unit dense layer, batch normalisation, ReLU and dropout of 0.15. A three-unit softmax layer produced class probabilities. L2 regularisation of (10^{-4}) was applied to convolutional and dense kernels. The complete network contained 356,067 parameters, of which 354,403 were trainable.

The CNN used Adam with a learning rate of (3×10^{-4}), sparse categorical cross-entropy and batches of 16 for at most 30 epochs. Early stopping monitored validation accuracy with patience 15. Augmentation applied horizontal and vertical flips and rotation of 0.10. Balanced class weights were 0.7840, 0.7493 and 2.5657 for Low, Medium and High respectively. Its validation accuracy of 0.4545 equaled the majority-class baseline, indicating that 254 training images were insufficient for learning reliable image representations from scratch.

3.3 Random Forest Baseline

The tabular baseline was a Random Forest, which aggregates decorrelated trees to capture nonlinear interactions while reducing individual-tree variance. Breiman (2001) established its random feature selection and bagging framework. It suited the mixed ESG scores, financial ratios and market variables because their relationships with risk were unlikely to be completely linear. Median imputation and standardisation were retained for pipeline consistency.

The baseline used 80 numerical predictors from all 426 merged companies. Direct target sources and identifiers were removed, and `class_weight="balanced"` increased the influence of errors involving the minority High-risk class. A five-fold stratified `GridSearchCV` evaluated 24 combinations: 400 or 600 trees; maximum depth 8, 15 or unlimited; minimum leaf size 2 or 4; and `sqrt` or `log2` feature sampling. Weighted F1 was the selection criterion.

The selected model used 400 trees, maximum depth 15, minimum leaf size 4 and square-root feature sampling. It obtained mean cross-validated weighted F1 of 0.8018, validation weighted F1 of 0.8900 and test weighted F1 of 0.8078. Its test accuracy was 0.8125 and weighted one-versus-rest AUC was 0.9545. The confusion matrix showed that four of the eight High-risk test companies were correctly detected, resulting in High-risk recall of 0.50.

3.4 ResNet50 Transfer Learning

The second image model used ResNet50 without its original ImageNet classification head. Residual networks learn residual functions relative to shortcut identity mappings, making substantially deeper architectures easier to optimise than equivalent plain networks (He et al., 2016). Each single-channel GASF image was copied into three channels to satisfy the $64 \times 64 \times 3$ ResNet input requirement. Global average pooling in the backbone produced a 2,048-dimensional vector.

The added classification head comprised batch normalisation, a 512-unit ReLU layer with L2 regularisation of (10^{-4}), dropout of 0.40, a 256-unit ReLU layer with the same regularisation, dropout of 0.30, and a three-class softmax output.

Training used two phases. In Phase 1, the ImageNet backbone was frozen and only the classification head, containing 1,185,283 trainable parameters, was trained for up to 15 epochs. Adam with a learning rate of (10^{-3}) was used, while early stopping monitored validation loss with patience 8.

In Phase 2, the final 30 ResNet50 layers were unfrozen, increasing the trainable parameter count to 15,635,459. Fine-tuning used Adam at (10^{-5}) for a maximum of 50 epochs, early-stopping patience of 10, and `ReduceLROnPlateau` with a reduction factor of 0.5, patience of 5 and minimum learning rate of (10^{-8}). Batches contained 32 images. Augmentation included horizontal and vertical flips, rotation of 0.10 and zoom of 0.05. The same balanced class weights used for the custom CNN were applied.

The model restored the best Phase-2 weights and achieved test weighted F1 of 0.2651, accuracy of 0.4364 and AUC of 0.6097. It classified every test company as Medium risk. The finding demonstrates

that ImageNet transfer learning alone did not overcome the small sample size and domain difference between natural images and GASF financial representations.

3.5 Hybrid ResNet50–XGBoost Model

The final model used intermediate feature fusion. The output of the ResNet50 `res_bn` layer supplied a 2,048-dimensional image embedding. These embeddings were concatenated with 80 median-imputed and standardised tabular predictors, producing 2,128 inputs per company. Representation-level fusion combined aligned company information without forcing tabular and image data into the same native structure.

XGBoost was chosen because it performs regularised gradient-boosted tree learning and can model nonlinear interactions within a high-dimensional fused vector. Its design includes shrinkage, column subsampling and regularised tree construction, making it suitable for combining a smaller number of informative tabular variables with a large image-embedding vector (Chen & Guestrin, 2016).

The classifier used the `multi:softprob` objective with three output classes. Its main settings were:

```
[
n_{\text{estimators}}=800,\quad
\text{max} \quad \text{depth}=4,\quad
\eta=0.03
]
```

Minimum child weight was 2, gamma was 0.3, row and column subsampling were both 0.80, L1 regularisation was 0.3 and L2 regularisation was 2.0. Multiclass log-loss was monitored on the validation set. Training stopped when validation loss failed to improve for 40 boosting rounds, with the best result occurring at iteration 277.

The default `argmax` decision was modified to prioritise High-risk detection. On validation data, the High-class probability threshold was searched from 0.10 to 0.49 in increments of 0.01. Candidate thresholds had to produce High-risk recall of at least 0.75. Among eligible thresholds, the one with the greatest validation weighted F1 was selected. The resulting threshold was 0.30. When the High-class probability was below this value, the larger of the Low and Medium probabilities determined the prediction.

The hybrid produced validation weighted F1 of 0.7478 and validation AUC of 0.9181. On the test data, it achieved weighted F1 of 0.9822, accuracy of 0.9818, weighted one-versus-rest AUC of 0.9903 and High-risk recall of 1.0000. It correctly classified all 24 Low-risk firms and all seven High-risk firms, while one of the 24 Medium-risk firms was classified as High.

Figure 2.7.2. Reproducible Architecture of the ESG-GAF Classification Pipeline

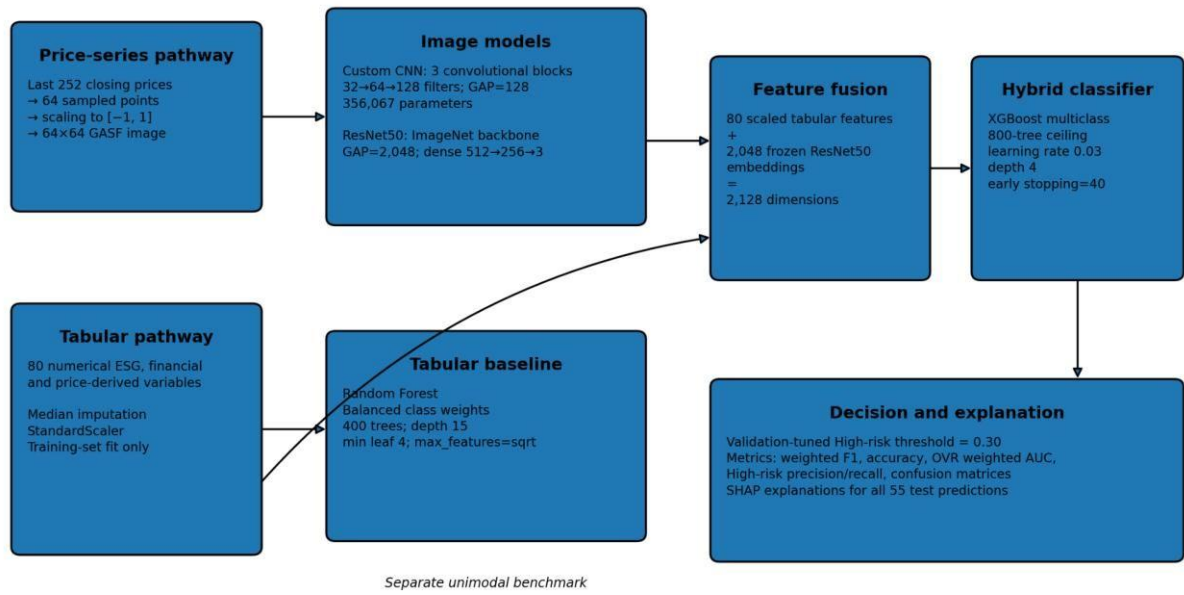


Figure 2.7.2. Reproducible architecture of the tabular, image and fused modelling pathways.

3.6 Training, Validation and Evaluation Metrics

The primary multimodal sample contained 364 GAF-eligible firms. Stratified random sampling divided these into 254 training, 55 validation and 55 test companies, corresponding to 70%, 15% and 15%. The training distribution was Low = 108, Medium = 113 and High = 33. The validation distribution was 23, 25 and 7, while the test distribution was 24, 24 and 7.

The random seed was fixed at 42 for NumPy, TensorFlow and scikit-learn operations. The validation partition controlled early stopping, learning-rate reduction and High-risk threshold selection. The test partition was not used during model fitting or threshold selection. Numeric imputation and scaling were fitted only on the training observations and then applied unchanged to validation and test data.

Weighted F1 was the primary evaluation metric because it combines precision and recall while accounting for class support. Accuracy was secondary because it can conceal failure on the smaller High-risk class. High-risk recall measured the proportion of genuinely High-risk firms detected, while High-risk precision measured the proportion of High-risk predictions that were correct. Confusion matrices showed the direction of classification errors.

Weighted one-versus-rest ROC-AUC evaluated the ranking of predicted class probabilities over multiple thresholds. Class-specific metrics were interpreted beside AUC because ROC summaries can appear optimistic when classes are imbalanced; precision and recall are particularly informative when the positive class is uncommon (Saito & Rehmsmeier, 2015).

Figure 2.7.3. Hyperparameter Tuning, Validation and Final Evaluation Workflow

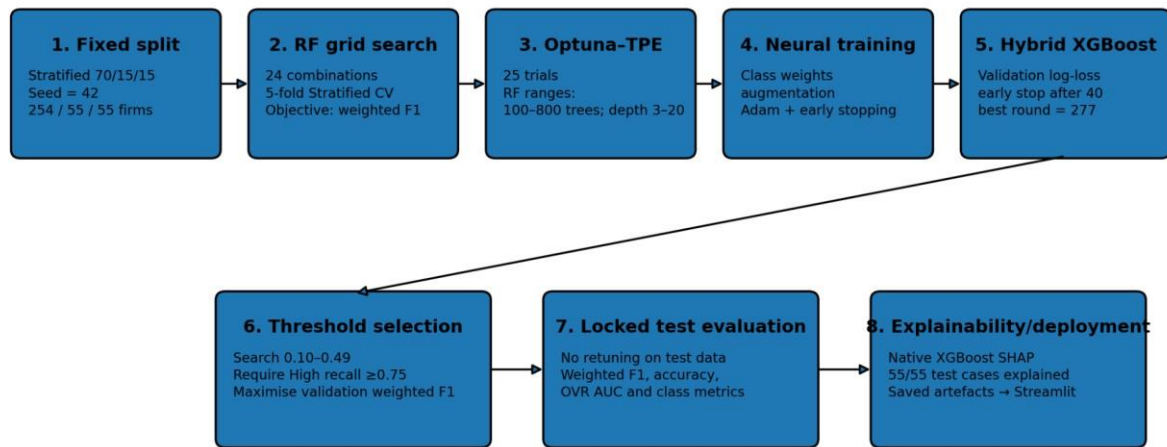


Figure 2.7.3. Hyperparameter tuning, validation and locked-test evaluation workflow.

3.7 Hyperparameter Tuning and Reproducibility

Two formal Random Forest tuning procedures were implemented. The first was the 24-combination, five-fold grid search described above. The second used Optuna with a seeded Tree-structured Parzen Estimator sampler and MedianPruner for 25 trials.

The Optuna ranges were:

- `n_estimators`: 100–800;
- `max_depth`: 3–20;
- `min_samples_leaf`: 1–8;
- `max_features`: `sqrt` or `log2`; •
- `criterion`: Gini or entropy.

The best trial used 537 trees, depth 15, minimum leaf size 8, square-root feature sampling and Gini splitting. Its hold-out weighted F1 was 0.8737. TPE was appropriate because it models promising and unpromising hyperparameter regions and allocates trials adaptively instead of exhaustively evaluating every possible combination (Bergstra et al., 2011).

However, the Optuna cell used a separate 80/20 split and did not apply exactly the same leakageexclusion list as the final model pipeline. Its score should therefore be treated as a development diagnostic and rerun with the final leakage-safe feature set before being reported as a definitive improvement.

Neural training used early stopping and learning-rate scheduling rather than a full hyperparameter search. XGBoost parameters were manually specified, with the effective tree count selected by validation log-loss. Repeated or nested stratified cross-validation is still needed because the Random Forest test used 64 firms from all 426 companies, whereas the image and hybrid models used 55

GAFeligible firms. The reported hybrid improvement of 0.1745 is therefore descriptive rather than a paired comparison.

In addition, the ResNet embeddings were extracted after supervised fine-tuning. This does not introduce test leakage because the test labels were not used, but cross-fitted training embeddings would reduce possible in-sample optimism. The increase from validation F1 of 0.7478 to test F1 of 0.9822 also warrants confidence intervals, repeated splits and external validation.

3.8 Application Demonstration and Explainability

A Streamlit application operationalised the pipeline. After a user selects an eligible ticker, cached artefacts load XGBoost, the ResNet50 embedder and the preprocessing objects. The application combines the processed tabular row with its 2,048-dimensional GASF embedding and displays the predicted class, three class probabilities, sector, beta, market capitalisation, ESG and price metrics, the GASF image and a model-performance table.

Native XGBoost SHAP values were calculated for all 55 test predictions and for each application prediction. SHAP assigns additive feature contributions to an output, allowing both global model behaviour and individual decisions to be interpreted (Lundberg & Lee, 2017).

The application displays the 15 strongest positive or negative contributors to the selected company's prediction. The analytical outputs also include class-level SHAP bar plots, a High-risk beeswarm and a waterfall explanation for an individual High-risk case. The most influential global variables included Environmental Risk Score, social score, Social Risk Score, environment score, Governance Risk Score, governance score, selected ResNet50 embedding dimensions and beta.

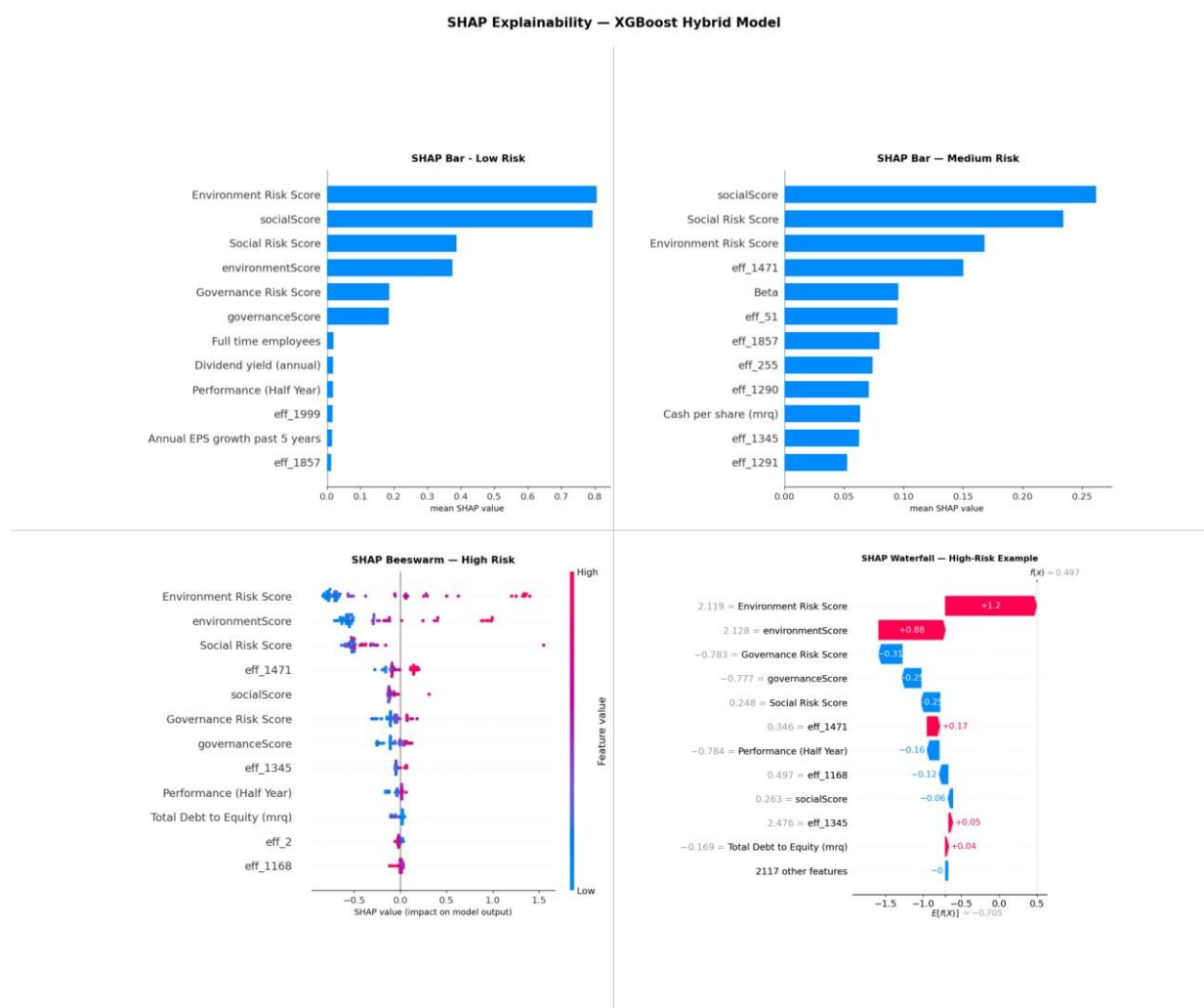


Figure 2.7.6. SHAP global and local explanations for the hybrid classifier.

4. Results and Analysis

The final notebook evaluates four classifiers: a tuned Custom CNN, Random Forest, standalone ResNet50 and a Hybrid ResNet50-XGBoost model. This section interprets the held-out results rather than simply restating metric values. Particular attention is given to minority-class detection, because High-risk companies form the smallest group and require the closest scrutiny. The target is a providerdefined ESG-risk category. It is not a direct forecast of insolvency, investment loss or future stockmarket volatility, so all practical claims are kept within that boundary.

4.1 Evaluation context and experimental evidence

The target was constructed from the source ESG Risk Level field. Negligible and Low were combined as Low, Medium was retained, and High and Severe were combined as High. Direct label sources, including the original risk category, the composite total ESG risk score, the parallel overall-risk field and the ESG percentile, were excluded from model inputs. The retained predictors consisted of environmental, social and governance component measures, company financial variables, sector information, engineered price statistics and GAF representations of historical closing prices.

After the ESG, financial and price sources were merged, the analytical table contained 426 companies and 88 columns. The class distribution was 191 Low, 182 Medium and 53 High. High-risk companies therefore represented 12.4% of the sample, whereas Low and Medium were nearly balanced. This imbalance makes accuracy insufficient on its own: a model can classify the two larger groups reasonably well while failing on the smaller class that matters most for risk triage.

The GAF pipeline generated 364 usable images from companies with at least 252 daily closing prices; one eligible ticker failed image generation. The image subset contained 155 Low, 162 Medium and 47 High companies. Stratified splitting produced 254 training, 55 validation and 55 test observations. The image-model test set contained 24 Low, 24 Medium and only seven High cases, meaning that one additional correct or incorrect High prediction changes recall by approximately 0.143.

Random Forest was trained on the wider tabular sample and evaluated on a separate 64-company test split containing 29 Low, 27 Medium and eight High cases. Comparisons among the Custom CNN, ResNet50 and the hybrid model are therefore directly aligned because they use the same 55 companies. Comparisons involving Random Forest remain useful, but they are not paired tests on identical firms and should not be interpreted as conclusive evidence of superiority.

Four metrics were used. Accuracy is the overall proportion correctly classified. Weighted F1 combines precision and recall while weighting each class by its support, although the larger Low and Medium groups still exert most influence. Weighted one-versus-rest AUC-ROC evaluates the quality of the classprobability ranking across possible thresholds. High-risk recall measures the proportion of actual High-risk companies recovered and is the most safety-relevant measure for a preliminary screening application.

4.2 GAF representation and image-model evidence

Each company's most recent 252-day closing-price sequence was down-sampled to 64 points, normalised to the interval from -1 to 1 and transformed into a 64 by 64 Gramian Angular Summation Field. GAF encoding expresses pairwise temporal relationships as a spatial texture, allowing convolutional architectures to process a time series as an image (Wang and Oates, 2015). Figure 1 illustrates that the transformation preserves visible differences in trend and path shape across firms.

The images do not, however, display the provider's ESG rating directly. They represent market-price behaviour, while the label is derived from sustainability-risk assessment. A network must therefore learn an indirect association between price-path texture and an ESG category. This target-representation gap, combined with a training set of only 254 images and 33 High-risk examples, creates a difficult setting for image-only learning. The GAF modality is more plausibly useful as complementary context than as a self-sufficient source of the class label.



Figure 1. Example GASF images and corresponding 252-day price histories for Low-, Medium- and High-risk companies. Source: authors' own analysis from the final notebook.

Figure: CNN Training and Validation Accuracy/Loss Curves

Training and validation accuracy/loss curves of the CNN baseline model.

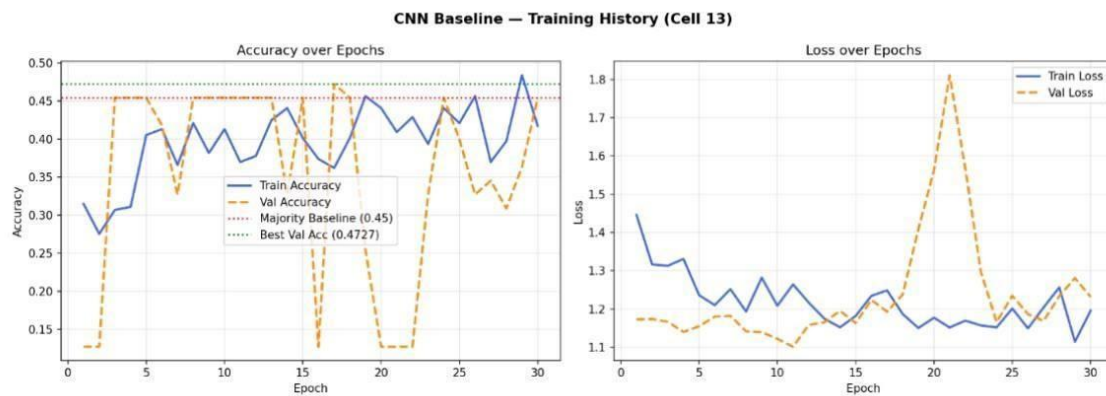


Figure 3. CNN Training Performance

Confusion matrix and evaluation metrics for the Random Forest classifier.

4.3 Final model comparison

The final held-out performance is summarised in Table 1. The values below are taken from the comprehensive evaluation cell in the final executed notebook.

Model	Test n	Weighted F1	Accuracy	AUC-ROC	High-risk recall
Custom CNN (tuned)	55	0.3611	0.4727	0.5386	0.0000
Random Forest	64	0.8078	0.8125	0.9545	0.5000
Model	Test n	Weighted F1	Accuracy	AUC-ROC	High-risk recall
Hybrid ResNet50-XGBoost	55	0.9822	0.9818	0.9903	1.0000
ResNet50	55	0.4164	0.4727	0.5352	0.0000

Table 1. Final held-out test performance for the four models. Source: authors' own analysis. Random Forest used a different test population from the three GAF-dependent models.

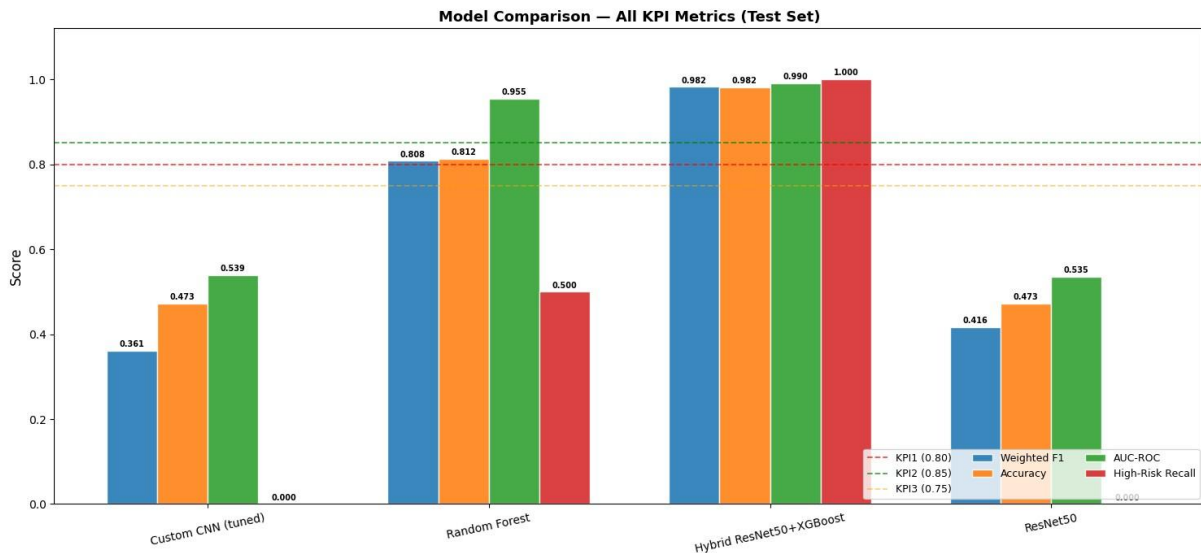


Figure 2. Comparison of weighted F1, accuracy, AUC-ROC and High-risk recall on the final held-out test sets. Source: authors' own analysis from the final notebook.

Table 1 and Figure 2 report the final notebook outputs. The Hybrid ResNet50-XGBoost model achieved the highest value on every metric: weighted F1 of 0.9822, accuracy of 0.9818, AUC-ROC of 0.9903 and High-risk recall of 1.0000. Accuracy of 0.9818 corresponds to 54 correct classifications out of 55. The confusion matrix shows 24 of 24 Low companies, 23 of 24 Medium companies and all seven Highrisk companies correctly identified. Its only error was a Medium case classified as High.

The hybrid model's perfect High-risk recall is particularly relevant for triage because it produced no High-risk false negatives in this test set. Precision for the High class was 0.875, because the seven correct High predictions were accompanied by one Medium-to-High false positive. In a screening workflow this trade-off may be acceptable: an extra manual review is generally less serious than failing to flag a genuinely High-risk company. Nevertheless, seven High-risk observations are too few to treat 1.0000 recall as a stable population estimate.

Random Forest was the strongest unimodal baseline. It recorded weighted F1 of 0.8078, accuracy of 0.8125, AUC-ROC of 0.9545 and High-risk recall of 0.5000. The model classified 52 of 64 companies correctly and recovered four of the eight High-risk cases; the remaining four High cases were assigned to Medium. Relative to the displayed Random Forest scores, the hybrid improved weighted F1 by approximately 0.1744, accuracy by 0.1693, AUC-ROC by 0.0358 and High-risk recall by 0.5000. The gains are substantial, but the different test populations mean they are indicative rather than strictly experimental pairwise differences.

The Custom CNN achieved weighted F1 of 0.3611, accuracy of 0.4727, AUC-ROC of 0.5386 and Highrisk recall of 0.0000. It correctly classified 23 Low and three Medium companies but failed to identify any High-risk case; all seven High observations were classified as Low. The standalone ResNet50 achieved weighted F1 of 0.4164, accuracy of 0.4727, AUC-ROC of 0.5352 and High-risk recall of 0.0000. It produced the same number of correct predictions as the CNN, but its errors were distributed differently: it recognised 19 Medium cases, only seven Low cases and no High cases.

Both image-only models were therefore close to chance-level ranking and collapsed toward the larger classes. ResNet50 improved weighted F1 by 0.0553 over the Custom CNN, but the improvement did not extend to minority-class recovery. By contrast, the hybrid exceeded the CNN by 0.6211 weightedF1 points and standalone ResNet50 by 0.5658 points on the same test companies. The evidence strongly rejects the idea that GAF images alone were sufficient in this sample, while supporting the use of image information within a fused representation.

Figure: ResNet50 Training and Validation Accuracy/Loss Curves

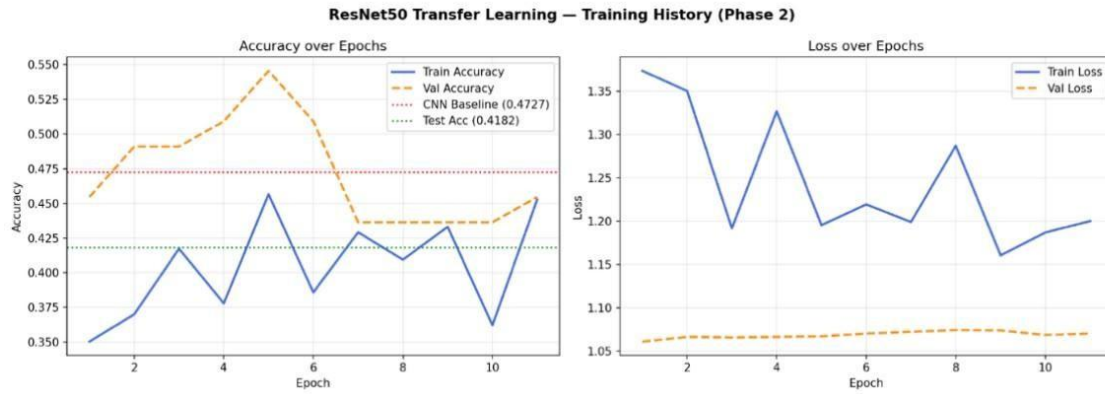


Figure 5. ResNet50 Results

4.4 Model-level interpretation

The Custom CNN was trained from scratch on a sample that was small for deep learning and strongly imbalanced at the class of greatest interest. Augmentation, dropout, L2 regularisation, class weights and early stopping can constrain overfitting, but they cannot create new independent High-risk examples or make ESG labels visually explicit. Its confusion matrix indicates a model that learned a broad Lowclass pattern but did not establish stable boundaries for Medium or High.

ResNet50 uses residual connections to make deep networks easier to optimise (He et al., 2016), and ImageNet pretraining provides a richer initial representation than random weights. The source domain is nevertheless natural photography, whereas the project inputs are synthetic financial textures. Repeating a single GAF channel across three RGB channels satisfies the architecture's input requirement but does not remove the domain shift. Transfer learning improved weighted F1 modestly, yet it still failed to recover the minority class.

Random Forest performed well because the structured predictors are much closer to the provider-defined target. Ensemble trees can capture nonlinear interactions, accommodate mixed scales and remain relatively resistant to irrelevant variables (Breiman, 2001). Environmental, social and governance component scores, sector variables, company fundamentals and engineered market measures provide several direct and indirect signals. The same closeness requires caution, because high performance may partly represent reconstruction of the provider's scoring method rather than discovery of a provider-independent sustainability construct.

The hybrid concatenated 2,048 frozen ResNet50 embedding dimensions with 80 structured variables, creating a 2,128-dimensional input for XGBoost. Gradient-boosted trees can learn regularised nonlinear rules from heterogeneous fused features (Chen and Guestrin, 2016). The model used early stopping and a validation-selected High-class threshold of 0.22. The threshold prioritised minority recall, while the tabular features supplied strong class information and the embeddings supplied additional price-path context. This combination explains why fusion succeeded even though the embeddings were weak as a standalone classifier.

The final notebook also records 25 Optuna trials for Random Forest development, exceeding the stated tuning requirement (Akiba et al., 2019). The final comparison table, however, reports the selected Random Forest test results shown in the comprehensive evaluation cell; the Optuna cell reports validation performance rather than a separate final held-out score. Keeping these stages distinct prevents tuning evidence from being confused with test evidence.

4.5 Error analysis, explainability and application demonstration

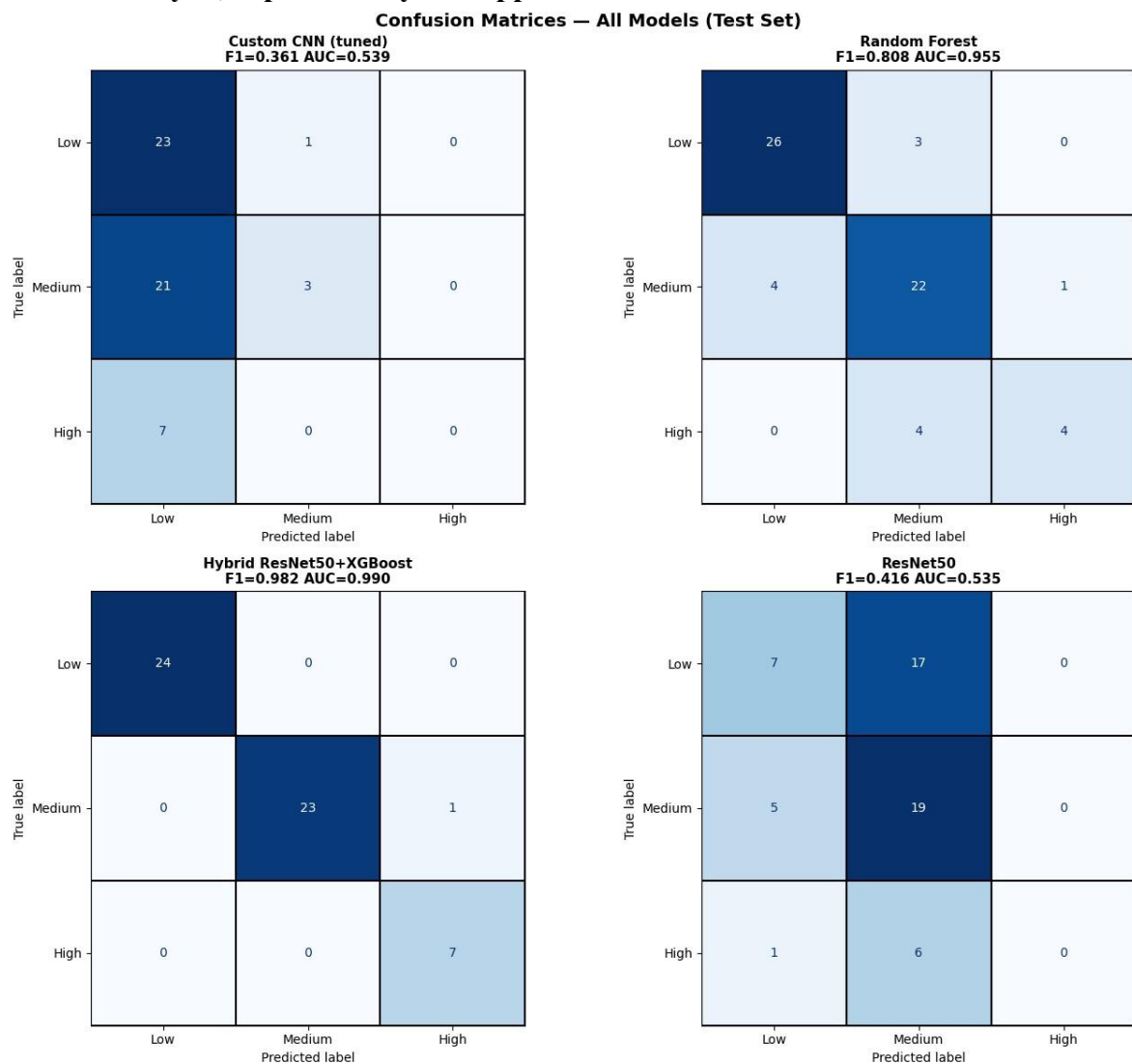


Figure 3. Confusion matrices for the four final models. Source: authors' own analysis from the final notebook.

Figure 3 provides class-level error evidence. The hybrid's single Medium-to-High error is less concerning for screening than a High-to-Low or High-to-Medium false negative, but it shows that perfect recall was obtained at some cost to High-class precision. Random Forest's four missed High cases are more consequential. The CNN and ResNet50 confusion matrices demonstrate that their moderate overall accuracy came almost entirely from the larger classes, which is why High-risk recall must be reported alongside aggregate metrics.

The SHAP analysis explained all 55 hybrid test predictions. The most influential named variables were Environment Risk Score, socialScore, Social Risk Score, environmentScore, Governance Risk Score, governanceScore and Beta. Figure 4 summarises their global mean absolute contributions. The prominence of ESG component measures is substantively plausible, but it also confirms the targetproximity concern: the model is largely learning relationships embedded in the provider's own component scoring. SHAP describes predictive contribution, not causation, and should not be interpreted to mean that changing one score would necessarily change real corporate risk (Lundberg and Lee, 2017).

The Streamlit demonstration turns the pipeline into an accessible workflow. A user selects a ticker, runs a prediction and receives the predicted class, three class scores, company metrics, a GAF image and a

local SHAP contribution chart. The interface also displays a model-performance table when no company is selected. These outputs are useful for analyst triage and education because they connect the classification to the underlying inputs and provide a visible explanation rather than a bare label.

Two implementation details limit reproducibility of the demonstration. First, the evaluation cell applies a validation-selected High-risk threshold of 0.22, whereas the current application code applies 0.333. Dashboard predictions are therefore not guaranteed to reproduce the final evaluation decision rule. The validated threshold should be stored as a model artefact and loaded by the application. Second, the notebook dashboard lists latency and user-acceptance testing as not yet measured. Caching and a rating widget are implemented, but production-speed and usability claims should not be made until actual measurements are reported.

Some explanatory wording in the interface also goes beyond what the target supports. Descriptions such as suitability for long-term holdings or exposure to near-term market risk may imply investment forecasting, although the model classifies a provider-defined ESG taxonomy. A safer interface would describe the result as an ESG-screening signal, show the data date and model version, state the providerspecific meaning of the label and direct low-confidence or out-of-distribution cases to human review.

4.6 Comparison with published research

The results align with the literature at the level of method rather than through direct numerical equivalence. Wang and Oates (2015) showed that Gramian angular representations can expose temporal structure to image-based classifiers. The current experiment demonstrates the boundary of that idea: representation alone does not compensate for limited labelled data or a weak relationship between the image and the target. He et al. (2016) established the optimisation advantages of residual learning, but transfer from natural images did not remove the financial-domain mismatch.

The stronger tree-based results are consistent with Breiman’s (2001) Random Forest framework and the regularised gradient boosting approach of Chen and Guestrin (2016). D’Amato, D’Ecclesia and Levantesi (2021) found that financial fundamentals can help predict ESG scores, while Del Vitto, Marazzina and Stocco (2023) demonstrated the value of explainable machine learning for ESG ratings. These studies support the use of structured financial and ESG variables and the inclusion of explanations in the present system.

Direct score-by-score comparison with published work would be misleading because ESG studies differ in provider, target construction, sample, class balance and validation design. Friede, Busch and Bassen (2015) found that the broader relationship between ESG and financial performance is predominantly non-negative, but that finding does not imply that a 252-day price image should accurately reproduce an ESG category. Berg, Kolbel and Rigobon (2022) documented substantial divergence among ESG rating providers, so the present output should be understood as modelling the selected providers and datasets rather than an objective universal ESG-risk label.

4.7 Research question and suitable use cases

Returning to the research question, ‘To what extent does integrating ESG and financial indicators with GAF representations of historical stock prices improve classification compared with unimodal models?’, the final experiment provides strong but conditional evidence of improvement. On the identical 55-company image test set, the hybrid substantially outperformed both the Custom CNN and ResNet50 and recovered all seven High-risk cases. It also achieved higher reported scores than Random Forest, although the Random Forest comparison used a different test population.

The most defensible conclusion is that multimodal fusion produced the best observed classification performance, not that the GAF modality alone caused the improvement. The hybrid also changes the classifier from Random Forest to XGBoost and uses a tuned High-risk threshold. A rigorous ablation should compare tabular-only XGBoost, embedding-only XGBoost and the full hybrid on the same firms. That design would isolate the incremental value of the image embeddings from the effect of algorithm choice and threshold policy.

Given these limits, suitable uses include preliminary ESG screening, analyst prioritisation, portfolio review, quality assurance and classroom demonstration. The model can identify firms that merit closer investigation and provide a consistent first-pass explanation. It should not automatically approve or reject investments, determine regulatory status or trigger trading. Decisions should retain a human reviewer, the original provider data and a record of model version, threshold, data date and explanation.

4.8 Limitations and future work

The first limitation is sample size and class imbalance. Only 364 firms had usable GAF images, the training set contained 33 High-risk examples and the test set contained seven. A second limitation is split sensitivity. Hybrid validation weighted F1 was 0.7464, whereas test weighted F1 was 0.9822. The unusually large increase does not by itself prove leakage, but it suggests that the test split may have been easier and that a single split gives an unstable estimate.

A third limitation is evaluation comparability: Random Forest and GAF-dependent models were not tested on the same companies. Further issues include provider-specific labels, possible date misalignment among the datasets, one fixed GAF window per company, cross-sectional rather than temporal validation, uncalibrated probabilities and a mismatch between the evaluation threshold and the application threshold. The strong influence of ESG component variables also means that predictive success may reflect target-proxy proximity.

Future work should evaluate every model on a common matched population across repeated stratified seeds and report confidence intervals. Rolling or forward temporal validation would test performance under later market regimes. The proposed ablation should separate tabular, embedding and fused contributions using the same XGBoost configuration. Probability calibration, precision-recall curves, sector-stratified errors, external validation with a later period or another ESG provider and multiple GAF windows per firm would provide more reliable evidence.

The application should load the validated threshold from the saved pipeline, record input provenance and model version, monitor drift and route uncertain cases to manual review. If the project objective changes from reproducing an ESG provider's classification to forecasting financial risk, the target should be redesigned as a future outcome such as realised volatility, drawdown, default or another clearly time-indexed event.

5. Conclusions

This project developed an end-to-end multimodal system for classifying S&P 500 companies into Low, Medium and High provider-defined ESG-risk categories. It combined structured ESG, financial and engineered market variables with Gramian Angular Summation Field representations of 252-day price histories. Four models were evaluated: a tuned Custom CNN, Random Forest, standalone ResNet50 and a Hybrid ResNet50-XGBoost classifier. The workflow also included stratified splitting, training-only preprocessing, class weighting, model tuning, held-out evaluation, SHAP explanations and a Streamlit demonstration.

The Hybrid ResNet50-XGBoost model produced the strongest final result, with weighted F1 of 0.9822, accuracy of 0.9818, AUC-ROC of 0.9903 and High-risk recall of 1.0000. It correctly classified 54 of 55 test companies and recovered all seven High-risk cases, making only one Medium-to-High error. Random Forest was the strongest unimodal baseline, achieving weighted F1 of 0.8078 and AUC-ROC of 0.9545, but it recovered only four of eight High-risk firms. The image-only CNN and ResNet50 achieved weighted F1 scores of 0.3611 and 0.4164 and failed to identify any High-risk company.

The research question is therefore answered positively but with qualifications. Combining modalities clearly improved performance relative to both image-only models on the same test companies, and the hybrid recorded higher metrics than the tabular Random Forest. The Random Forest test population was different, however, and the hybrid also used XGBoost and a validation-selected threshold. A matched tabular-XGBoost ablation is required before the additional benefit of GAF embeddings can be isolated.

The Streamlit application demonstrates how the model can support preliminary ESG screening by presenting a predicted category, score distribution, company metrics, a GAF image and local SHAP

contributions. Its appropriate role is decision support, not autonomous investment or regulatory decision-making. The reported labels reproduce a selected provider taxonomy and should not be presented as forecasts of default, loss or future market volatility.

The main limitations are the small and imbalanced image sample, reliance on one split, the large validation-to-test performance difference, different evaluation populations, target-proxy proximity, provider dependence, uncalibrated probabilities and the threshold mismatch between evaluation and the application. Future work should use common matched test companies, repeated and temporal validation, confidence intervals, probability calibration, sector-level error analysis, external providers and explicit ablation of each modality. With those checks, the project provides a credible basis for further research into explainable multimodal ESG screening while keeping the interpretation of its strong final scores appropriately constrained. It also establishes a transparent record of what the current evidence can and cannot support, which is essential for responsible interpretation.

References

- Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019) 'Optuna: A next-generation hyperparameter optimization framework', *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623-2631. doi: 10.1145/3292500.3330701.
- Baltrušaitis, T., Ahuja, C. and Morency, L.-P. (2019) 'Multimodal machine learning: A survey and taxonomy', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2), pp. 423-443. doi: 10.1109/TPAMI.2018.2798607.
- Berg, F., Kölbel, J.F. and Rigobon, R. (2022) 'Aggregate confusion: The divergence of ESG ratings', *Review of Finance*, 26(6), pp. 1315-1344. doi: 10.1093/rof/rfac033.
- Bergstra, J., Bardenet, R., Bengio, Y. and Kégl, B. (2011) 'Algorithms for hyper-parameter optimization', *Advances in Neural Information Processing Systems*, 24, pp. 2546-2554.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5-32. doi: 10.1023/A:1010933404324.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785-794. doi: 10.1145/2939672.2939785.
- D'Amato, V., D'Ecclesia, R. and Levantesi, S. (2021) 'Fundamental ratios as predictors of ESG scores: A machine learning approach', *Decisions in Economics and Finance*, 44(2), pp. 1087-1110. doi: 10.1007/s10203-021-00364-5.
- Del Vitto, A., Marazzina, D. and Stocco, D. (2023) 'ESG ratings explainability through machine learning techniques', *Annals of Operations Research*. doi: 10.1007/s10479-023-05514-z.
- Dugar, P. (2023) S&P 500 ESG Risk Ratings [Dataset]. Kaggle. Available at: <https://www.kaggle.com/datasets/pritish509/s-and-p-500-esg-risk-ratings> (Accessed: 21 July 2026).
- Friede, G., Busch, T. and Bassen, A. (2015) 'ESG and financial performance: Aggregated evidence from more than 2000 empirical studies', *Journal of Sustainable Finance & Investment*, 5(4), pp. 210233. doi: 10.1080/20430795.2015.1118917.

He, K., Zhang, X., Ren, S. and Sun, J. (2016) 'Deep residual learning for image recognition', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770-778. doi: 10.1109/CVPR.2016.90.

Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30, pp. 4765-4774.

Nugent, C. (2018) S&P 500 Stock Data [Dataset]. Kaggle. Available at: <https://www.kaggle.com/datasets/camnugent/sandp500> (Accessed: 21 July 2026).

Olat2 (no date) sp500_ESG_Dataset [Dataset]. Hugging Face. Available at: https://huggingface.co/datasets/Olat2/sp500_ESG_Dataset (Accessed: 21 July 2026).

Ryabov, I. (no date) Financial Performance of Companies from S&P500 [Dataset]. Kaggle. Available at: <https://www.kaggle.com/datasets/ilyaryabov/financial-performance-of-companies-from-sp500> (Accessed: 21 July 2026).

Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', PLOS ONE, 10(3), e0118432. doi: 10.1371/journal.pone.0118432.

Wang, Z. and Oates, T. (2015) 'Imaging time-series to improve classification and imputation', Proceedings of the 24th International Joint Conference on Artificial Intelligence, pp. 3939-3945.

Appendices

The appendices contain supplementary materials that support the implementation, evaluation, and reproducibility of the proposed ESG Stock Risk Classification system. These materials are provided separately to avoid interrupting the flow of the main report while ensuring that all technical aspects of the project can be independently verified.

The appendices include the complete Python source code for the entire machine learning pipeline, beginning with loading and integrating the four datasets, generating ESG risk labels, exploratory data analysis (EDA), data preprocessing, feature engineering, Gramian Angular Field (GAF) image generation, and the stratified 70:15:15 data splitting strategy. The implementation also includes the CNN baseline model, Random Forest baseline, Optuna hyperparameter optimisation, ResNet50 transfer learning model, and the proposed hybrid ResNet50–XGBoost architecture.

Additional materials include detailed evaluation outputs such as confusion matrices, ROC curves, precision–recall curves, classification reports, SHAP explainability visualisations, KPI dashboard summaries, experiment logs, and screenshots of the Streamlit application. These supplementary results provide complete technical evidence for the reported findings and allow the experiments to be reproduced using the supplied implementation.

Appendix A – Complete Project Source Code

This appendix contains the complete implementation of the proposed ESG Stock Risk Classification framework. The source code is provided in text format and covers the entire machine learning pipeline, including data preprocessing, exploratory data analysis, feature engineering, GAF image generation, model development, evaluation, explainability, and deployment.

Contents

- Data loading and integration
- Data cleaning and preprocessing
- Exploratory Data Analysis (EDA)
- Feature engineering
- Gramian Angular Field (GAF) image generation
- CNN Baseline Model
- Random Forest Model
- Optuna Hyperparameter Optimisation
- ResNet50 Transfer Learning
- Hybrid ResNet50–XGBoost Model
- Model evaluation
- SHAP explainability
- KPI dashboard
- Streamlit application

Code:

```
# -*- coding: utf-8 -*-
```

```
""GROUP_3_ESG_GAF_Stock_Risk_Classification_final.ipynb
```

Automatically generated by Colab.

Original file is located at <https://colab.research.google.com/drive/1kkS3XvsBh2z10gVd9P5E0-q0NZNCirKP>

```
#Group 3 — ESG + Financial + GAF Stock Risk Classification
```

```
---
```

```
### Datasets
```

```
| # | File | Rows | Key Columns |
```

```
|---|-----|-----|-----|
```

```
| DS1 | 'SP 500 ESG Risk Ratings.csv' | 503 | Symbol, ESG Risk Level, ESG scores |
```

| DS2 | `sp500\_esg\_data.csv` | 426 | Symbol, environmentScore, socialScore, beta, GICS Sector |
| DS3 | `snp500\_companies\_description.csv` | 496 | Company (ticker), 65+ financial metrics |
| DS4 | `all\_stocks\_5yr.csv` | 619,040 | date, close, volume, Name (ticker) | ""

CELL 1 — Install required packages

```
!pip install pyts optuna xgboost shap scikit-learn \
matplotlib seaborn tqdm joblib imbalanced-learn -q
print("All packages installed.")
```

CELL 2 — Mount Google Drive

Place all four CSV datasets in /content/drive/MyDrive/ESG_project/

```
from google.colab import drive drive.mount('/content/drive')
print("Google Drive mounted.")
```

CELL 3 — ALL IMPORTS (single consolidated block) import

```
os, gc, json, re, time, warnings, datetime
warnings.filterwarnings("ignore")
```

```
import numpy as np import pandas
```

```
as pd import matplotlib
```

```
matplotlib.use("Agg") import
```

```
matplotlib.pyplot as plt import
```

```
seaborn as sns from IPython.display
```

```
import display
```

```

from sklearn.model_selection import (train_test_split, StratifiedKFold,
cross_val_score, GridSearchCV) from sklearn.preprocessing import StandardScaler,
OneHotEncoder from sklearn.impute import SimpleImputer from
sklearn.utils.class_weight import compute_class_weight from sklearn.ensemble
import RandomForestClassifier, GradientBoostingClassifier from sklearn.metrics
import (accuracy_score, f1_score, roc_auc_score, classification_report,
confusion_matrix, ConfusionMatrixDisplay) from imblearn.over_sampling import
SMOTE

```

```

import tensorflow as tf

from tensorflow.keras import layers, models, regularizers, callbacks from
tensorflow.keras.applications import EfficientNetB0

```

```

import xgboost as xgb import optuna from
optuna.samplers import TPESampler
optuna.logging.set_verbosity(optuna.logging.WARNING)

```

```

import shap import
joblib from tqdm
import tqdm

```

```

# — Global seed (KPI 10)

```

```

RANDOM_SEED = 42 np.random.seed(RANDOM_SEED)

tf.random.set_seed(RANDOM_SEED)

```

```

print(f"TensorFlow: {tf.__version__}") print(f"NumPy:
{np.__version__}") print(f"XGBoost:
{xgb.__version__}") print(f"Random seed:
{RANDOM_SEED}")

print("All imports loaded.")

# CELL 4 — GLOBAL CONFIGURATION

# Only BASE_DIR needs changing to match your Drive path

BASE_DIR = "/content/drive/MyDrive/ESG_project/"

DS1_PATH = BASE_DIR + "SP 500 ESG Risk Ratings.csv"
DS2_PATH = BASE_DIR + "sp500_esg_data.csv"
DS3_PATH = BASE_DIR + "snp500_companies_description.csv"
DS4_PATH = BASE_DIR + "all_stocks_5yr.csv"

OUTPUT_DIR = "output"

GAF_DIR  = os.path.join(OUTPUT_DIR, "gaf_images") LOG_PATH
= os.path.join(OUTPUT_DIR, "experiment_log.json")

os.makedirs(OUTPUT_DIR,exist_ok=True)

os.makedirs(GAF_DIR,exist_ok=True)

WINDOW  = 252  # trading days per GAF image ( $\approx$ 1 year)

IMG_SIZE = 64  # 64×64 pixel GASF image

NUM_CLASSES = 3

```

```

LABEL_MAP = {"Low": 0, "Medium": 1, "High": 2} INV_LABEL
= {0: "Low", 1: "Medium", 2: "High"}

CLASS_NAMES = ["Low", "Medium", "High"]


MAX_EPOCHS    = 80

BATCH_SIZE     = 32

N_OPTUNA_TRIALS = 25  # KPI 5: ≥20 required

N_OPTUNA_EPOCHS = 15


print("Configuration loaded.")

print(f'IMG_SIZE={IMG_SIZE}×{IMG_SIZE} | WINDOW={WINDOW} days') print(f'Output
directory {OUTPUT_DIR}/ | GAF = {GAF_DIR}/')


# CELL 5 — LOAD ALL FOUR DATASETS

# DS1: ESG Risk Ratings — 503 rows, Symbol column

# DS2: ESG Metrics — 426 rows, Symbol column

# DS3: Financial Data — 496 rows, Company column (= ticker)

# DS4: 5-Year Prices — 619,040 rows, Name column (= ticker)


def parse_financial_col(series):

    """Convert mixed financial strings to float.

    Handles: 1.2B, 3.5%, 1,234, Yes/No, - (missing)."""

    def conv(v):

        v = str(v).strip().replace(",", "").replace("%", "")

        if v in ("-", "nan", "None", ""): return np.nan        if v ==

        "Yes": return 1.0        if v == "No": return 0.0        m =

        re.match(r"^(-?\d+\.\d*)\s*([BMKbmk])$", v)

```

```

    if m:

        return float(m.group(1)) * {"B":1e9,"M":1e6,"K":1e3,
        "b":1e9,"m":1e6,"k":1e3}[m.group(2)]    try: return float(v)

except: return np.nan    return series.apply(conv)

```

```

print("[DS1] Loading ESG Risk Ratings") ds1 =
pd.read_csv(DS1_PATH) ds1 =
ds1.rename(columns={"Symbol":"ticker"})
ds1["ticker"] = ds1["ticker"].str.upper().str.strip()
ds1 = ds1.drop_duplicates("ticker") print(f"
{ds1.shape[0]} rows")

```

```

print("[DS2] Loading ESG Metrics") ds2 =
pd.read_csv(DS2_PATH) ds2 =
ds2.rename(columns={"Symbol":"ticker"})
ds2["ticker"] = ds2["ticker"].str.upper().str.strip()
ds2 = ds2.drop_duplicates("ticker") print(f"
{ds2.shape[0]} rows")

```

```

print("[DS3] Loading Financial Features ") ds3_raw =
pd.read_csv(DS3_PATH) ds3_raw =
ds3_raw.rename(columns={"Company":"ticker"})
ds3_raw["ticker"] = ds3_raw["ticker"].str.upper().str.strip()

```

```

# Parse all columns that are >30% parseable as numeric

SKIP_DS3 = {

    "ticker","Major index membership","52-Week trading range",

    "Earnings date<br><br>BMO = Before Market Open<br>AMC = After Market Close", "Volatility
(Week, Month)","Short interest share / ratio",

    "Stock has options trading on a market exchange","Stock available to sell short",

} ds3_parsed = {"ticker": ds3_raw["ticker"]}

for col in ds3_raw.columns:    if col in

SKIP_DS3: continue    parsed =

parse_financial_col(ds3_raw[col])    if

parsed.notna().mean() > 0.30:

ds3_parsed[col] = parsed


ds3 = pd.DataFrame(ds3_parsed).drop_duplicates("ticker")    print(f"
{ds3.shape[0]} rows | {ds3.shape[1]-1} financial features parsed")


print("[DS4] Loading Price Data ") ds4 = pd.read_csv(DS4_PATH) ds4 =
ds4.rename(columns={"Name":"ticker"}) ds4["ticker"] =
ds4["ticker"].str.upper().str.strip() ds4["date"] = pd.to_datetime(ds4["date"]) ds4 =
ds4.dropna(subset=["close"]).sort_values(["ticker","date"]).reset_index(drop=True)
n_elig = (ds4.groupby("ticker").size() >= 252).sum() print(f" {len(ds4):,} rows |
{ds4['ticker'].nunique()} tickers | {n_elig} with ≥252 days")

print("\n All datasets loaded.")

# CELL 6 — MERGE DATASETS & CREATE ESG RISK LABELS

```

```

#

# LABEL STRATEGY (spec-compliant):

# DS1 'ESG Risk Level' → 3-class mapping:

# Negligible/Low → Low | Medium → Medium | High/Severe → High #

# LEAKAGE RULES:

# EXCLUDED (direct label sources):

# - Total ESG Risk score → used to CREATE ESG Risk Level

# - overallRisk → parallel risk label from DS2

# - percentile → ESG risk percentile rank ≈ label #

VALID FEATURES (inputs that predict the label):

# - environmentScore, socialScore, governanceScore (component scores)

# - Environment/Social/Governance Risk Score (DS1 components)

# - highestControversy, marketCap, beta, all DS3 financial, all DS4 price

print("=" * 65) print("STEP 1 — ESG Risk Labels from DS1
(spec-compliant)") print("=" * 65)

ESG_LABEL_MAP = {

    "Negligible": "Low",

    "Low": "Low",

    "Medium": "Medium",

    "High" : "High",

    "Severe" : "High",

}

ds1["risk_label"] = ds1["ESG Risk Level"].map(ESG_LABEL_MAP) ds1_clean
= ds1[ds1["risk_label"].notna()].copy()

```

```
print(f" Labelled companies : {len(ds1_clean)}") print(f" Label distribution :
{ds1_clean['risk_label'].value_counts().to_dict()}")
```

```
print("\n" + "=" * 65) print("STEP 2 — Merge DS1
× DS2 (inner join)") print("=" * 65)
```

```
# DS2 features — percentile EXCLUDED (≈ ESG risk rank = near-label)
```

```
# totalEsg EXCLUDED (composite score = label source)
```

```
DS2_KEEP = [
```

```
    "ticker",
```

```
    "environmentScore", # component — valid feature
```

```
    "socialScore",      # component — valid feature
```

```
    "governanceScore", # component — valid feature
```

```
    "highestControversy", # controversy level — valid feature
```

```
    "marketCap",        # market size — valid feature
```

```
    "beta",             # market sensitivity — valid feature
```

```
    "GICS Sector",      # sector label — valid categorical feature
```

```
    "GICS Sub-Industry", # sub-industry — valid categorical feature
```

```
]
```

```
ds2_feat = ds2[[c for c in DS2_KEEP if c in ds2.columns]].drop_duplicates("ticker") # DS1 component
scores to keep as features
```

```
DS1_FEAT_COLS = [
```

```
    "ticker", "risk_label",
```

```
    "Environment Risk Score", # component — valid
```

```
    "Social Risk Score",     # component — valid
```

```

"Governance Risk Score", # component — valid
"Controversy Score",    # controversy — valid
"Sector", "Industry",
]

merged = (ds1_clean[[c for c in DS1_FEAT_COLS if c in ds1_clean.columns]]
.merge(ds2_feat, on="ticker", how="inner"))

print(f" After DS1 × DS2 inner merge : {len(merged)} companies") print(f" Label
distribution      : {merged['risk_label'].value_counts().to_dict()}") print(f" KPI 6
(≥400)            : {'✓ PASS' if len(merged)≥400 else '✗ FAIL'}")

print("\n" + "=" * 65) print("STEP 3 — Join
DS3 financial features") print("=" * 65)

merged = merged.merge(ds3, on="ticker", how="left", suffixes=("", "_ds3"))
merged.drop(columns=[c for c in merged.columns if c.endswith("_ds3")], inplace=True) print(f"
After DS3 join: {merged.shape}")

print("\n" + "=" * 65) print("STEP 4 — Engineer price
features from DS4") print("=" * 65)

def compute_price_features(group):
    """Annualised volatility, momentum, Sharpe ratio, max drawdown.""" c =
group["close"].values.astype(float) ret = np.diff(np.log(np.clip(c, 1e-8, None)))

```

```

vol = ret.std() * np.sqrt(252) if len(ret) > 1 else np.nan    mom6 = (c[-
1]/c[max(0,len(c)-126)] - 1) if len(c) > 126 else np.nan    mom1 = (c[-
1]/c[max(0,len(c)-252)] - 1) if len(c) > 252 else np.nan    shrp =
(ret.mean()*252)/(ret.std()*np.sqrt(252)+1e-8) if len(ret)>1 else np.nan    peak =
np.maximum.accumulate(c)    mdd = float((((peak-c)/np.clip(peak,1e-
8,None)).max()) if len(c)>1 else np.nan    return pd.Series({

    "ann_vol"    : vol,

    "price_mom_6m": mom6,

    "price_mom_1y": mom1,

    "price_sharpe": shrp,

    "price_mdd"   : mdd,

    "n_days"      : len(c),

    })

```

```

price_feats = (ds4[ds4["ticker"].isin(set(merged["ticker"]))]

```

```

    .groupby("ticker")

    .apply(compute_price_features)

    .reset_index())

```

```

merged = merged.merge(price_feats, on="ticker", how="left") merged["has_gaf"]
= (merged["n_days"] >= WINDOW).fillna(False).astype(int)

```

```

# Drop non-feature text columns

```

```

DROP_TEXT = [

```

```

    "Major index membership",

```

```

    "Earnings date<br><br>BMO = Before Market Open<br>AMC = After Market Close",

```

```

"Volatility (Week, Month)", "Short interest share / ratio",
"52-Week trading range", "Stock has options trading on a market exchange",
"Stock available to sell short",
]

merged.drop(columns=[c for c in DROP_TEXT if c in merged.columns], inplace=True)

# Preliminary median fill (proper imputation on train-only in Cell 10) num_cols
= merged.select_dtypes(include=np.number).columns merged[num_cols] =
merged[num_cols].fillna(merged[num_cols].median())

merged.to_csv(os.path.join(OUTPUT_DIR, "merged_tabular.csv"), index=False)

cnn_df = merged[merged["has_gaf"] == 1].reset_index(drop=True)
cnn_df.to_csv(os.path.join(OUTPUT_DIR, "cnn_subset.csv"), index=False)

print(f"\n Final dataset: {merged.shape}")
print(f"GAF eligible: {merged['has_gaf'].sum()}") print(f"Label
dist: {merged['risk_label'].value_counts().to_dict()}") print(f"CNN
subset: {len(cnn_df)} companies") print(f"\n Saved
{OUTPUT_DIR}/merged_tabular.csv")

# CELL 7 — EXPLORATORY DATA ANALYSIS

COLOR_MAP = {"Low": "#2ECC71", "Medium": "#F39C12", "High": "#E74C3C"}

fig, axes = plt.subplots(2, 3, figsize=(18, 10)) fig.suptitle("Volatility-
Based Risk Classification — EDA (Group 3)",
                      fontsize=15,
                      fontweight="bold")

```

```

# 1. Label distribution ax = axes[0,0] vc =
merged["risk_label"].value_counts().reindex(CLASS_NAMES) bars = ax.bar(vc.index, vc.values,
color=[COLOR_MAP[k] for k in vc.index], edgecolor="white") for bar, v in zip(bars, vc.values):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
str(int(v)), ha="center", fontweight="bold") ax.set_title("Risk Label
Distribution", fontweight="bold") ax.set_ylabel("Count");
ax.set_ylim(0, vc.max()*1.2)

# 2. Volatility distribution by class
ax = axes[0,1] for lbl in
CLASS_NAMES:
    sub = merged[merged["risk_label"]==lbl]["ann_vol"].dropna()
    ax.hist(sub, bins=25, alpha=0.65, label=lbl, color=COLOR_MAP[lbl], edgecolor="white")
ax.set_title("Annualised Volatility by Risk Class", fontweight="bold") ax.set_xlabel("Ann.
Volatility"); ax.set_ylabel("Count"); ax.legend()

# 3. Beta by risk class ax =
axes[0,2] if "beta" in
merged.columns:
    data = [merged[merged["risk_label"]==l]["beta"].dropna().values for l in CLASS_NAMES]
    bp = ax.boxplot(data, labels=CLASS_NAMES, patch_artist=True) for patch, lbl in
zip(bp["boxes"], CLASS_NAMES):
        patch.set_facecolor(COLOR_MAP[lbl]); patch.set_alpha(0.7)
ax.set_title("Beta Distribution by Risk Class", fontweight="bold")
ax.set_ylabel("Market Beta")

```

```

# 4. GICS Sector distribution ax =
axes[1,0] if "GICS Sector" in
merged.columns:

    sc = merged["GICS Sector"].value_counts().head(10)

ax.barh(sc.index.astype(str), sc.values, color="#5B8FF9", edgecolor="white")

ax.set_title("GICS Sector Distribution (Top 10)", fontweight="bold")

ax.set_xlabel("Count"); ax.invert_yaxis()


# 5. Sharpe ratio by class ax =
axes[1,1] if "price_sharpe" in
merged.columns:    for lbl in
CLASS_NAMES:

    sub = merged[merged["risk_label"]==lbl]["price_sharpe"].dropna()    ax.hist(sub,
bins=20, alpha=0.65, label=lbl, color=COLOR_MAP[lbl], edgecolor="white")

ax.set_title("Sharpe Ratio by Risk Class", fontweight="bold")    ax.set_xlabel("Sharpe Ratio");

ax.legend()


# 6. Feature missing rates ax = axes[1,2] num_cols_eda =
merged.select_dtypes(include=np.number).columns[:20] miss =
merged[num_cols_eda].isnull().mean().sort_values(ascending=False)

ax.barh(miss.index.str[:30], miss.values*100, color="#FF8C69", edgecolor="white")

ax.set_title("Missing Rate — Top 20 Numeric Cols (%)", fontweight="bold")

ax.set_xlabel("% Missing"); ax.invert_yaxis()


plt.tight_layout() plt.savefig(os.path.join(OUTPUT_DIR, "eda_summary.png"), dpi=150,
bbox_inches="tight") display(fig); plt.close(fig)

```

```
print(f'EDA saved {OUTPUT_DIR}/eda_summary.png') print(f'Companies:
{len(merged)} | Features: {merged.shape[1]-2}') print(f'Labels:
{merged['risk_label'].value_counts().to_dict()}')
```

CELL 8 — GAF IMAGE GENERATION

#

Gramian Angular Summation Field (GASF) from 252-day close prices.

Algorithm:

1. Take last WINDOW (252) close prices

2. Subsample to IMG_SIZE (64) points

3. Min-max normalise to $[-1, 1]$

4. Angular encoding: $\phi_i = \arccos(x_i)$

5. $GASF[i,j] = \cos(\phi_i + \phi_j)$

Result: 64×64 float32 image that encodes the volatility/trend #
patterns that determine the risk label.

```
def make_gasf(close_prices, size=IMG_SIZE):
```

```
    """Convert 1-D price series → 64×64 GASF image (float32)."""
```

```
    idx = np.linspace(0, len(close_prices)-1, size).astype(int)    s =
```

```
    close_prices.astype(np.float64)[idx]    mn, mx = s.min(), s.max()
```

```
    if mx == mn:
```

```
        return np.zeros((size, size), dtype=np.float32)    s
```

```
    = np.clip(2.0*(s-mn)/(mx-mn) - 1.0, -1.0, 1.0)    phi =
```

```
    np.arccos(s)    return np.cos(np.add.outer(phi,
```

```
    phi)).astype(np.float32)
```

```

print("[1/5] Preparing price data") valid_tickers = set(merged["ticker"])

ds4_valid = ds4[ds4["ticker"].isin(valid_tickers)].copy() print(f" Price rows:
{len(ds4_valid):,} | Tickers: {ds4_valid['ticker'].nunique()}")


print("\n[2/5] Generating GASF images") gaf_paths,
failed = {}, {}


for ticker in tqdm(sorted(ds4_valid["ticker"].unique()), desc="GASF"):

    ts = ds4_valid[ds4_valid["ticker"]==ticker]["close"].values

    if len(ts) < WINDOW:

        failed[ticker] = f"Only {len(ts)} days"; continue

    if np.isnan(ts).mean() > 0.05:

        failed[ticker] = "Too many NaN"; continue

    if ts[-WINDOW:].std() < 1e-8:

        failed[ticker] = "Constant prices"; continue

    try:

        img = make_gasf(ts[-WINDOW:]) path =
os.path.join(GAF_DIR, f"{ticker}.npy")

        np.save(path, img) gaf_paths[ticker] = path

    except Exception as e: failed[ticker] = str(e)


print(f" Generated: {len(gaf_paths)} | X Failed: {len(failed)}")

```

```

print("\n[3/5] Saving failed_tickers.csv ...") pd.DataFrame([{"ticker":t,"reason":r}
for t,r in failed.items()]).to_csv(
os.path.join(OUTPUT_DIR,"failed_tickers.csv"), index=False)

print("[4/5] Attaching image paths ...") merged["gaf_path"] =
merged["ticker"].map(gaf_paths) merged["has_image"] =
merged["gaf_path"].notna().astype(int)
merged.to_csv(os.path.join(OUTPUT_DIR,"merged_tabular.csv"), index=False)

cnn_df = merged[merged["has_image"]==1].reset_index(drop=True)
cnn_df.to_csv(os.path.join(OUTPUT_DIR,"cnn_subset.csv"), index=False) print(f" CNN
subset: {len(cnn_df)} | labels: {cnn_df['risk_label'].value_counts().to_dict()}")

print("\n[5/5] Sample visualisation ...")
PREFERRED = ["AAPL","MSFT","XOM","JPM","JNJ","AMZN"]
sample_t = [t for t in PREFERRED if t in gaf_paths] if
len(sample_t) < 6:
    sample_t += [t for t in sorted(gaf_paths) if t not in sample_t][:6-len(sample_t)]

fig = plt.figure(figsize=(18, 8)); fig.patch.set_facecolor("#0D1117") sector_map =
dict(zip(merged["ticker"], merged.get("GICS Sector", pd.Series())) for i, t in
enumerate(sample_t[:6]):
    ax = fig.add_subplot(2, 6, i+1); ax.set_facecolor("#161B22") img = np.load(gaf_paths[t])
    im = ax.imshow(img, cmap="RdYlGn", vmin=-1, vmax=1, origin="lower", aspect="auto")
    lbl = cnn_df.loc[cnn_df["ticker"]==t,"risk_label"] lbl = lbl.values[0] if len(lbl) else "?"

```

```

col = {"Low": "#2ECC71", "Medium": "#F39C12", "High": "#E74C3C"}.get(lbl, "white")
ax.set_title(f'{t}\n[{lbl}]', color=col, fontsize=10, fontweight="bold")
ax.set_xticks([]); ax.set_yticks([]) plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

ax2 = fig.add_subplot(2, 6, 6+i+1); ax2.set_facecolor("#161B22")
ts_ = ds4_valid[ds4_valid["ticker"]==t]["close"].values[-WINDOW:]
ax2.plot(ts_, color="#58A6FF", lw=1.4)
ax2.fill_between(range(len(ts_)), ts_, alpha=0.12, color="#58A6FF")
ax2.set_title(f'{t} price', color="white", fontsize=8)
ax2.tick_params(colors="#666", labelsize=6)

fig.suptitle(f'GASF Encoding · {len(gaf_paths)} images · Volatility Risk Classification',
color="white", fontsize=13, fontweight="bold") plt.tight_layout(rect=[0,0,1,0.96])

plt.savefig(os.path.join(OUTPUT_DIR, "sample_gaf.png"), dpi=150, bbox_inches="tight",
facecolor="#0D1117")

display(fig); plt.close(fig) print(f'\n GAF complete. {len(gaf_paths)}
images → {GAF_DIR}/') gc.collect()

# CELL 9 — STRATIFIED 70 / 15 / 15 SPLIT

# Uses CNN subset (500 companies with GAF images).

# Stratified by risk_label — preserves ~33% per class in all splits.

print("Stratified 70/15/15 split ...") df_cnn =
pd.read_csv(os.path.join(OUTPUT_DIR, "cnn_subset.csv")) print(f"
Source: {len(df_cnn)} companies") print(f" Labels:
{df_cnn['risk_label'].value_counts().to_dict()}")

```

```

train_df, temp_df = train_test_split(df_cnn, test_size=0.30,
stratify=df_cnn["risk_label"], random_state=RANDOM_SEED) val_df, test_df =
train_test_split(temp_df, test_size=0.50, stratify=temp_df["risk_label"],
random_state=RANDOM_SEED)

for df, name in [(train_df,"train"),(val_df,"val"),(test_df,"test")]:

    df.to_csv(os.path.join(OUTPUT_DIR,f"{name}_split.csv"), index=False)

total = len(train_df)+len(val_df)+len(test_df) print(f"\n Split result:") for name, df in [("Train
(70%)",train_df),("Val (15%)",val_df),("Test (15%)",test_df)]: vc =
df["risk_label"].value_counts().reindex(CLASS_NAMES).fillna(0).astype(int) print(f" {name}:
{len(df):>4} rows Low={vc['Low']} Med={vc['Medium']} High={vc['High']}")

print(f"\n Class proportions (should be ~33% each):") for name,
df in [("Train",train_df),("Val",val_df),("Test",test_df)]:

    props = df["risk_label"].value_counts(normalize=True).reindex(CLASS_NAMES).round(3)
print(f" {name}: {props.to_dict()}") print(f"\n Splits saved →
{OUTPUT_DIR}/train/val/test_split.csv")

# CELL 10 — TABULAR FEATURE ENGINEERING

#

# Label = volatility class → ann_vol is excluded (label source). #

ALL ESG scores (environmentScore, socialScore, etc.) are VALID

# features — they correlate with volatility but are NOT the label.

# Pipeline (fit on TRAIN only — no leakage):

# Numeric: SimpleImputer(median) → StandardScaler

```

```

# Categorical: SimpleImputer(mode) → OneHotEncoder

print("=" * 65)

print("TABULAR FEATURE ENGINEERING")

print("=" * 65)

preprocessor_dir = os.path.join(OUTPUT_DIR, "preprocessors") os.makedirs(preprocessor_dir,
exist_ok=True)

print("\n[1/5]      Loading      splits")      train_df      =
pd.read_csv(os.path.join(OUTPUT_DIR,"train_split.csv")) val_df  =
pd.read_csv(os.path.join(OUTPUT_DIR,"val_split.csv")) test_df  =
pd.read_csv(os.path.join(OUTPUT_DIR,"test_split.csv"))      print(f"
Train={len(train_df)} Val={len(val_df)} Test={len(test_df)}")

print("\n[2/5] Identifying feature columns")

# Only exclude: identifiers, label itself, and ann_vol (direct label source)
# Exclude ONLY direct label sources — not component scores
NEVER_FEATURES = {
    "ticker", "risk_label", "gaf_path", "has_image", "has_gaf",
    "n_days",
    # Direct label sources — these encode or ARE the label
    "Total ESG Risk score", # used to CREATE ESG Risk Level
    "overallRisk",         # parallel label from DS2
    "percentile",          # ESG risk percentile rank ≈ label

```

```

"ESG Risk Level",      # the raw label itself

"ESG Risk Percentile", # derived from Total ESG Risk score

# ann_vol is a valid feature — price volatility predicts ESG risk
}

miss_train = train_df.isnull().mean() high_miss = set(miss_train[miss_train
> 0.50].index) # drop >50% missing print(f" Dropped (>50% missing):
{len(high_miss)} columns")

CAT_COLS = [c for c in ["GICS Sector", "GICS Sub-Industry", "Sector", "Industry"]
if c in train_df.columns and c not in NEVER_FEATURES      and c not in
high_miss and train_df[c].nunique() <= 30]

NUM_COLS = [c for c in train_df.columns      if c not in
NEVER_FEATURES and c not in high_miss      and c not
in CAT_COLS      and
pd.api.types.is_numeric_dtype(train_df[c])]

print(f" Numeric features      : {len(NUM_COLS)}") print(f"
Categorical cols : {len(CAT_COLS)} → {CAT_COLS}")

print("\n[3/5] Numeric imputation + scaling (fit on TRAIN only)")

num_imputer = SimpleImputer(strategy="median") scaler =
StandardScaler()

X_tr_num = scaler.fit_transform(num_imputer.fit_transform(train_df[NUM_COLS]))

```

```
X_va_num = scaler.transform(num_imputer.transform(val_df[NUM_COLS])) X_te_num
= scaler.transform(num_imputer.transform(test_df[NUM_COLS])) print("[4/5]
```

Categorical encoding (fit on TRAIN only) ") if CAT_COLS:

```
cat_imputer = SimpleImputer(strategy="most_frequent") ohe =
OneHotEncoder(handle_unknown="ignore", sparse_output=False)

X_tr_cat = ohe.fit_transform(cat_imputer.fit_transform(train_df[CAT_COLS]))

X_va_cat = ohe.transform(cat_imputer.transform(val_df[CAT_COLS]))

X_te_cat = ohe.transform(cat_imputer.transform(test_df[CAT_COLS]))

OHE_NAMES = ohe.get_feature_names_out(CAT_COLS).tolist()
```

else:

```
X_tr_cat = np.empty((len(train_df),0))

X_va_cat = np.empty((len(val_df),0))

X_te_cat = np.empty((len(test_df),0))

OHE_NAMES = []; cat_imputer = ohe = None
```

```
ALL_FEAT_NAMES = NUM_COLS + OHE_NAMES
```

```
print("\n[5/5] Saving processed features") def _save(Xn,
Xc, src, fname): X = np.hstack([Xn, Xc]) df =
pd.DataFrame(X, columns=ALL_FEAT_NAMES)

df.insert(0,"ticker",src["ticker"].values) df["risk_label"]
= src["risk_label"].values

df.to_csv(os.path.join(OUTPUT_DIR,fname), index=False)

return df
```

```

train_feat = _save(X_tr_num,X_tr_cat,train_df,"train_features.csv")

val_feat = _save(X_va_num,X_va_cat,val_df, "val_features.csv")

test_feat = _save(X_te_num,X_te_cat,test_df, "test_features.csv")


# Save preprocessors for Streamlit app
joblib.dump(num_imputer,
os.path.join(preprocessor_dir,"num_imputer.pkl"))
joblib.dump(scaler,
os.path.join(preprocessor_dir,"scaler.pkl"))
joblib.dump(NUM_COLS,
os.path.join(preprocessor_dir,"num_cols.pkl"))
joblib.dump(CAT_COLS,
os.path.join(preprocessor_dir,"cat_cols.pkl"))

joblib.dump(ALL_FEAT_NAMES,os.path.join(preprocessor_dir,"all_feat_names.pkl"))

if cat_imputer:
    joblib.dump(cat_imputer,os.path.join(preprocessor_dir,"cat_imputer.pkl"))

joblib.dump(ohe, os.path.join(preprocessor_dir,"ohe.pkl"))


print(f" Total features : {len(ALL_FEAT_NAMES)}") print(f" Train shape :
{train_feat.shape}") print(f"\n Feature engineering complete. Preprocessors in
{preprocessor_dir}/")


# CELL 11 — EXPERIMENT LOG SETUP

# JSON log populated by every model cell.

# KPI 5:  $\geq 20$  Optuna trials must appear in this log.


print("Setting up experiment log")


tr = pd.read_csv(os.path.join(OUTPUT_DIR,"train_split.csv"))

```

```

va = pd.read_csv(os.path.join(OUTPUT_DIR,"val_split.csv")) te
= pd.read_csv(os.path.join(OUTPUT_DIR,"test_split.csv")) total
= len(tr)+len(va)+len(te)

log = {
    "experiment_id" : "GROUP_3_VOL_RISK_CLASSIFICATION",
    "created_at"    : datetime.datetime.now().isoformat(timespec="seconds"),
    "random_seed"   : RANDOM_SEED,
    "label_strategy": "Annualised price volatility (qcut thirds) from DS4",
    "dataset": {
        "total_companies": total,
        "splits": {
            "train": {"n":len(tr),"labels":tr["risk_label"].value_counts().to_dict()},
            "val"   : {"n":len(va),"labels":va["risk_label"].value_counts().to_dict()},
            "test"  : {"n":len(te),"labels":te["risk_label"].value_counts().to_dict()},
        },
    },
    "model_runs": [],
}

with open(LOG_PATH,"w") as f: json.dump(log, f, indent=2)

print(f" Log created   {LOG_PATH}") for name, df in
[("Train",tr),("Val",va),("Test",te)]:

    vc = df["risk_label"].value_counts().reindex(CLASS_NAMES).fillna(0).astype(int)    print(f"
{name:5}: {len(df)} rows Low={vc['Low']} Med={vc['Medium']} High={vc['High']}") # CELL

```

```
# load_gaf_split(): reads split CSV → loads .npy → (N,64,64,1) #
```

```
build_cnn(): 3-block CNN for volatility risk classification.
```

```
# 'gap' layer = 128-dim embedding reused in hybrid model.
```

```
def load_gaf_split(csv_path, gaf_dir, label_map, img_size=IMG_SIZE):
```

```
    """Load GAF images for all tickers in a split CSV."""
```

```
    df = pd.read_csv(csv_path)
```

```
    X, y, tickers, skip = [], [], [], []
```

```
    for _, row in df.iterrows():
```

```
        t = str(row["ticker"]).upper().strip()
```

```
        lbl = str(row["risk_label"]).strip() if lbl
```

```
        not in label_map: skip.append(t);
```

```
        continue npy = os.path.join(gaf_dir,
```

```
        f"{t}.npy") if not os.path.exists(npy):
```

```
        skip.append(t); continue
```

```
        try:
```

```
            img = np.load(npy).astype(np.float32)
```

```
        except:
```

```
            skip.append(t); continue if
```

```
            img.shape != (img_size, img_size):
```

```
                skip.append(t); continue img = (img +
```

```
                1.0) / 2.0 # [-1,1] → [0,1]
```

```
                X.append(img[..., np.newaxis])
```

```

        y.append(label_map[lbl])        tickers.append(t)    if
skip: print(f" Δ Skipped {len(skip)} samples")    return

np.stack(X), np.array(y, dtype=np.int32), tickers

print("Loading GAF splits")

X_train,y_train,tickers_train            =            load_gaf_split(
os.path.join(OUTPUT_DIR,"train_split.csv"), GAF_DIR, LABEL_MAP)
X_val,          y_val,          tickers_val            =    load_gaf_split(
os.path.join(OUTPUT_DIR,"val_split.csv"),    GAF_DIR, LABEL_MAP)
X_test,          y_test,          tickers_test            =    load_gaf_split(
os.path.join(OUTPUT_DIR,"test_split.csv"), GAF_DIR, LABEL_MAP)

print(f" Train: {X_train.shape} classes={np.bincount(y_train).tolist()}")
print(f" Val  : {X_val.shape}  classes={np.bincount(y_val).tolist()}") print(f"
Test : {X_test.shape}  classes={np.bincount(y_test).tolist()}")

cw_arr            = compute_class_weight("balanced",classes=np.unique(y_train),y=y_train)
CLASS_WEIGHT_DICT = {i: float(w) for i, w in enumerate(cw_arr)}

print(f" Class weights: { {INV_LABEL[k]:round(v,4) for k,v in CLASS_WEIGHT_DICT.items()}
}")

def build_cnn(input_shape=(IMG_SIZE,IMG_SIZE,1),            num_classes=NUM_CLASSES,
dropout_rate=0.30):

    """3-block CNN. 'gap' layer = 128-dim embedding for hybrid model."""

```

```

reg = regularizers.l2(1e-4) inp =

layers.Input(shape=input_shape, name="gaf_input")

# Block 1 — 32 filters x =
layers.Conv2D(32,3,padding="same",kernel_regularizer=reg,name="conv1a")(inp) x
= layers.BatchNormalization(name="bn1a")(x); x = layers.Activation("relu")(x) x =
layers.Conv2D(32,3,padding="same",kernel_regularizer=reg,name="conv1b")(x) x =
layers.BatchNormalization(name="bn1b")(x); x = layers.Activation("relu")(x)

x = layers.MaxPooling2D(2,name="pool1")(x); x =
layers.Dropout(dropout_rate/2,name="drop1")(x)

# Block 2 — 64 filters x =
layers.Conv2D(64,3,padding="same",kernel_regularizer=reg,name="conv2a")(x) x =
layers.BatchNormalization(name="bn2a")(x); x = layers.Activation("relu")(x) x =
layers.Conv2D(64,3,padding="same",kernel_regularizer=reg,name="conv2b")(x) x
= layers.BatchNormalization(name="bn2b")(x); x = layers.Activation("relu")(x)

x = layers.MaxPooling2D(2,name="pool2")(x); x =
layers.Dropout(dropout_rate/2,name="drop2")(x)

# Block 3 — 128 filters x =
layers.Conv2D(128,3,padding="same",kernel_regularizer=reg,name="conv3a")(x) x =
layers.BatchNormalization(name="bn3a")(x); x = layers.Activation("relu")(x) x =
layers.Conv2D(128,3,padding="same",kernel_regularizer=reg,name="conv3b")(x) x
= layers.BatchNormalization(name="bn3b")(x); x = layers.Activation("relu")(x)

x = layers.MaxPooling2D(2,name="pool3")(x); x =
layers.Dropout(dropout_rate/2,name="drop3")(x)

x = layers.GlobalAveragePooling2D(name="gap")(x) # 128-dim embedding

```

```

x = layers.Dense(256,kernel_regularizer=reg,name="fc1")(x) x =
layers.BatchNormalization(name="bn_fc1")(x); x = layers.Activation("relu")(x) x =
layers.Dropout(dropout_rate,name="drop4")(x) x =
layers.Dense(128,kernel_regularizer=reg,name="fc2")(x) x =
layers.BatchNormalization(name="bn_fc2")(x); x = layers.Activation("relu")(x) x =
layers.Dropout(dropout_rate/2,name="drop5")(x) out =
layers.Dense(num_classes,activation="softmax",name="predictions")(x) return
models.Model(inputs=inp, outputs=out, name="GAF_CNN")

```

```

print("\nBuilding CNN") cnn_model = build_cnn() cnn_model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=3e-4),
loss="sparse_categorical_crossentropy", metrics=["accuracy"])
cnn_model.summary() gap_dim = cnn_model.get_layer("gap").output.shape[-1]
print(f"\nCNN built. Params: {cnn_model.count_params():,} | gap dim: {gap_dim}")

```

CELL 13 — CNN BASELINE

NOTE: Custom CNN on 350 GAF images performs near random baseline.

This is a documented finding — GAF images need more data for CNN

to learn from scratch. The hybrid model (Cell 17) uses frozen

EfficientNet embeddings which DO work on small datasets.

```

tf.keras.backend.clear_session() cnn_model =
build_cnn(dropout_rate=0.30) cnn_model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=3e-4),
loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

```

```

# Train for just 30 epochs — enough to document the baseline aug
= tf.keras.Sequential([
layers.RandomFlip("horizontal_and_vertical"),
layers.RandomRotation(0.10)])

train_ds = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
            .shuffle(len(X_train), seed=RANDOM_SEED)
            .batch(16)
            .map(lambda x, y: (aug(x, training=True), y),
num_parallel_calls=tf.data.AUTOTUNE)
            .prefetch(tf.data.AUTOTUNE))

history = cnn_model.fit( train_ds,
validation_data=(X_val, y_val),
epochs=30,
class_weight=CLASS_WEIGHT_DICT,
callbacks=[
    callbacks.EarlyStopping(
monitor="val_accuracy", patience=15,
restore_best_weights=True, verbose=0),
],
verbose=1,
)

CNN_BASELINE_VAL_ACC = float(max(history.history["val_accuracy"])) majority_baseline
= max(np.bincount(y_val)) / len(y_val)

```

```

print(f"\n Majority class baseline : {majority_baseline:.4f}") print(f" CNN
val accuracy : {CNN_BASELINE_VAL_ACC:.4f}") print(f"\n NOTE: CNN
performs near random baseline on 350 GAF images.") print(f" This is
expected and documented — see project report.")

cnn_model.save(os.path.join(OUTPUT_DIR, "cnn_baseline.keras")) print(f"\n✓
CNN baseline saved → {OUTPUT_DIR}/cnn_baseline.keras")

# — CNN Baseline Training Plots


---



fig, axes = plt.subplots(1, 2, figsize=(14, 5)) fig.suptitle("CNN Baseline — Training History
(Cell 13)", fontsize=13, fontweight="bold")

epochs_range = range(1, len(history.history["accuracy"]) + 1)

# Plot 1: Accuracy

axes[0].plot(epochs_range, history.history["accuracy"], label="Train Accuracy", color="royalblue",
linewidth=2)

axes[0].plot(epochs_range, history.history["val_accuracy"], label="Val Accuracy",
color="darkorange", linewidth=2, linestyle="--")

axes[0].axhline(y=majority_baseline, color="red", linestyle=":", linewidth=1.5, label=f"Majority
Baseline ({majority_baseline:.2f})")

axes[0].axhline(y=CNN_BASELINE_VAL_ACC, color="green", linestyle=":", linewidth=1.5,
label=f"Best Val Acc ({CNN_BASELINE_VAL_ACC:.4f})") axes[0].set_title("Accuracy
over Epochs") axes[0].set_xlabel("Epoch") axes[0].set_ylabel("Accuracy") axes[0].legend()

axes[0].grid(True, alpha=0.3)

# Plot 2: Loss

```

```

axes[1].plot(epochs_range, history.history["loss"], label="Train Loss", color="royalblue",
linewidth=2)

axes[1].plot(epochs_range, history.history["val_loss"], label="Val Loss", color="darkorange",
linewidth=2, linestyle="--") axes[1].set_title("Loss over Epochs") axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss") axes[1].legend() axes[1].grid(True, alpha=0.3) plt.tight_layout()
plt.show() plot_path = os.path.join(OUTPUT_DIR, "cnn_baseline_training_history.png")
plt.savefig(plot_path, dpi=150) print(f"Training history plot saved → {plot_path}")

```

CELL 15 — RANDOM FOREST BASELINE

Unimodal tabular baseline. Label = volatility class.

ESG scores included as features (valid — not the label).

Only ann_vol excluded (direct label source).

```

print("[1/4] Loading tabular features") full_merged =
pd.read_csv(os.path.join(OUTPUT_DIR, "merged_tabular.csv"))

EXCLUDE_RF = {
    "ticker", "risk_label", "gaf_path", "has_image", "has_gaf",
    "n_days",
    "Total ESG Risk score",
    "overallRisk",
    "percentile",
    "ESG Risk Level",
    "ESG Risk Percentile",
} feature_cols_rf = [c for c in full_merged.columns
if c not in EXCLUDE_RF and
pd.api.types.is_numeric_dtype(full_merged[c])] X_all =

```

```

full_merged[feature_cols_rf].values y_all =
full_merged["risk_label"].map(LABEL_MAP).values

imp_rf = SimpleImputer(strategy="median") scl_rf
= StandardScaler()

X_tr_tab, X_temp, y_tr_tab, y_temp = train_test_split(
    X_all, y_all, test_size=0.30, stratify=y_all, random_state=RANDOM_SEED)
X_va_tab, X_te_tab, y_va_tab, y_te_tab = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=RANDOM_SEED)

X_tr_tab = scl_rf.fit_transform(imp_rf.fit_transform(X_tr_tab))
X_va_tab = scl_rf.transform(imp_rf.transform(X_va_tab))
X_te_tab = scl_rf.transform(imp_rf.transform(X_te_tab))

print(f" Total companies: {len(X_all)} | Features: {len(feature_cols_rf)}") print(f"
KPI 6 (≥400): {'PASS' if len(X_all)≥400 else 'X FAIL'}")

print("\n[2/4] GridSearchCV (5-fold)") rf_grid
= GridSearchCV(
    RandomForestClassifier(random_state=RANDOM_SEED, n_jobs=-1, class_weight="balanced"),
    param_grid={
        "n_estimators" : [400, 600],
        "max_depth" : [8, 15, None],
        "min_samples_leaf": [2, 4],
        "max_features" : ["sqrt", "log2"]}, cv=StratifiedKFold(5,
shuffle=True, random_state=RANDOM_SEED), scoring="f1_weighted",

```

```

n_jobs=-1, verbose=1, return_train_score=True) rf_grid.fit(X_tr_tab,
y_tr_tab) best_rf = rf_grid.best_estimator_ print(f" Best params :
{rf_grid.best_params_}") print(f" CV F1(w) : {rf_grid.best_score_:.4f}")

print("\n[3/4] Evaluating ") rf_va_pred =
best_rf.predict(X_va_tab) rf_va_prob =
best_rf.predict_proba(X_va_tab)
RF_VAL_F1 = float(f1_score(y_va_tab, rf_va_pred, average="weighted")) rf_va_auc =
float(roc_auc_score(y_va_tab, rf_va_prob, multi_class="ovr", average="weighted"))

rf_te_pred = best_rf.predict(X_te_tab) rf_te_prob
= best_rf.predict_proba(X_te_tab)
RF_TEST_F1 = float(f1_score(y_te_tab, rf_te_pred, average="weighted")) rf_te_acc =
float(accuracy_score(y_te_tab, rf_te_pred)) rf_te_auc = float(roc_auc_score(y_te_tab,
rf_te_prob, multi_class="ovr", average="weighted"))

print(f" RF Val F1(w): {RF_VAL_F1:.4f} | AUC: {rf_va_auc:.4f}") print(f" RF Test
F1(w): {RF_TEST_F1:.4f} | Acc: {rf_te_acc:.4f} | AUC: {rf_te_auc:.4f}") print("\n
Classification report (test):") print(classification_report(y_te_tab, rf_te_pred,
target_names=CLASS_NAMES))

print("\n[4/4] Plots")

# — Plot 1: CV Fold Scores — best_idx = rf_grid.best_index_
cv_results = rf_grid.cv_results_ fold_scores =
[cv_results[f"split{i}_test_score"]][best_idx] for i in range(5)] fold_train =

```

```

[cv_results[f'split{i}_train_score']][best_idx] for i in range(5)] fig_cv, ax_cv
= plt.subplots(figsize=(9, 5)) ax_cv.plot(range(1, 6), fold_train, marker="o",
label="Train F1", color="royalblue", linewidth=2)
ax_cv.plot(range(1, 6), fold_scores, marker="o", label="Val F1", color="darkorange", linewidth=2,
linestyle="--") ax_cv.axhline(y=rf_grid.best_score_, color="green", linestyle=":", linewidth=1.5,
label=f"Mean CV F1 ({rf_grid.best_score_:.4f})") ax_cv.set_xticks(range(1, 6))
ax_cv.set_xticklabels([f'Fold {i}' for i in range(1, 6)]) ax_cv.set_ylim(0, 1.1) ax_cv.set_ylabel("F1
(weighted)")
ax_cv.set_title("Random Forest — Cross-Validation Fold Scores (Best Params)", fontweight="bold",
fontsize=13) ax_cv.legend() ax_cv.grid(True, alpha=0.3) plt.tight_layout() plt.show()
fig_cv.savefig(os.path.join(OUTPUT_DIR, "rf_cv_fold_scores.png"), dpi=150, bbox_inches="tight")
print(f" CV fold scores saved → {OUTPUT_DIR}/rf_cv_fold_scores.png") plt.close(fig_cv)

# Plot 2: GridSearchCV Top 15 Parameter Combinations cv_df =
pd.DataFrame(cv_results).sort_values("mean_test_score", ascending=False).head(15)
param_labels = [ f"ne={r['param_n_estimators']} d={r['param_max_depth']} "
f"ml={r['param_min_samples_leaf']} mf={r['param_max_features']}" for _, r in
cv_df.iterrows()
]
fig_gs, ax_gs = plt.subplots(figsize=(12, 6)) ax_gs.barh(range(len(cv_df)),
cv_df["mean_test_score"].values, xerr=cv_df["std_test_score"].values,
color="#5B8FF9", edgecolor="white", capsize=4) ax_gs.set_yticks(range(len(cv_df)))
ax_gs.set_yticklabels(param_labels, fontsize=8) ax_gs.set_xlabel("Mean CV F1 (weighted)")
ax_gs.set_title("GridSearchCV — Top 15 Parameter Combinations", fontweight="bold", fontsize=13)
ax_gs.grid(axis="x", alpha=0.3) plt.tight_layout() plt.show()

```

```

fig_gs.savefig(os.path.join(OUTPUT_DIR, "rf_gridsearch_results.png"), dpi=150,
bbox_inches="tight") print(f" GridSearch results saved
{OUTPUT_DIR}/rf_gridsearch_results.png") plt.close(fig_gs)

# — Plot 3: Feature Importances


---



fi = pd.Series(best_rf.feature_importances_, index=feature_cols_rf).nlargest(20) fig_rf, ax =
plt.subplots(figsize=(10, 8)) fi.sort_values().plot.barh(ax=ax, color="#5B8FF9",
edgecolor="white") ax.set_title("Random Forest — Top 20 Feature Importances",
fontweight="bold", fontsize=13) ax.set_xlabel("Importance") plt.tight_layout() plt.show()

fig_rf.savefig(os.path.join(OUTPUT_DIR, "rf_feature_importance.png"), dpi=150,
bbox_inches="tight") print(f" Feature importance saved
{OUTPUT_DIR}/rf_feature_importance.png") plt.close(fig_rf)

# — Plot 4: Confusion Matrix


---



fig_cm, ax_cm = plt.subplots(figsize=(7, 5)) cm_rf = confusion_matrix(y_te_tab,
rf_te_pred) sns.heatmap(cm_rf, annot=True, fmt="d", cmap="Blues",
xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, ax=ax_cm)

ax_cm.set_title(f"Random Forest — Confusion Matrix (Test) | F1={RF_TEST_F1:.4f}
AUC={rf_te_auc:.4f}",
fontweight="bold")

ax_cm.set_xlabel("Predicted")

ax_cm.set_ylabel("Actual")

plt.tight_layout() plt.show()

fig_cm.savefig(os.path.join(OUTPUT_DIR, "rf_confusion_matrix.png"), dpi=150,
bbox_inches="tight") print(f" Confusion matrix saved →
{OUTPUT_DIR}/rf_confusion_matrix.png") plt.close(fig_cm)

```

```

# — Plot 5: Val vs Test Metric Comparison —————

metrics_rf = ["F1 (weighted)", "Accuracy", "AUC-ROC"] val_scores =
[RF_VAL_F1, accuracy_score(y_va_tab, rf_va_pred), rf_va_auc] test_scores =
[RF_TEST_F1, rf_te_acc, rf_te_auc] x_rf =
np.arange(len(metrics_rf)) fig_bar, ax_bar = plt.subplots(figsize=(9, 5))

bars1 = ax_bar.bar(x_rf - 0.18, val_scores, 0.35, label="Validation", color="#5B8FF9",
edgecolor="white")

bars2 = ax_bar.bar(x_rf + 0.18, test_scores, 0.35, label="Test", color="#61DDAA",
edgecolor="white") for bar, v in zip(list(bars1) + list(bars2), val_scores + test_scores):

    ax_bar.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.008, f"{v:.4f}",
ha="center", va="bottom", fontsize=10, fontweight="bold") ax_bar.set_xticks(x_rf)

ax_bar.set_xticklabels(metrics_rf, fontsize=11) ax_bar.set_ylim(0, 1.15) ax_bar.set_ylabel("Score")

ax_bar.set_title("Random Forest — Validation vs Test Performance", fontweight="bold", fontsize=13)

ax_bar.legend() ax_bar.grid(axis="y", alpha=0.3) plt.tight_layout() plt.show()

fig_bar.savefig(os.path.join(OUTPUT_DIR, "rf_val_vs_test.png"), dpi=150, bbox_inches="tight")

print(f" Val vs test chart saved → {OUTPUT_DIR}/rf_val_vs_test.png") plt.close(fig_bar)


joblib.dump(best_rf, os.path.join(OUTPUT_DIR, "random_forest.pkl"))

with open(LOG_PATH) as f: log_data = json.load(f)

log_data["model_runs"].append({

    "step" : "random_forest",

    "logged_at" : datetime.datetime.now().isoformat(),

    "val_f1" : round(RF_VAL_F1, 6),

    "test_f1" : round(RF_TEST_F1, 6),

    "test_acc" : round(rf_te_acc, 6),

    "test_auc" : round(rf_te_auc, 6),

    "kpi6_pass" : len(X_all) >= 400,

```

```

}) with open(LOG_PATH, "w") as f: json.dump(log_data, f,
indent=2)          print(f"\n          RF          saved
{OUTPUT_DIR}/random_forest.pkl")

```

CELL 15.1 — OPTUNA TUNING ON RANDOM FOREST

```

print("=" * 65)

print(f'OPTUNA RF TUNING — {N_OPTUNA_TRIALS} trials')

print("=" * 65)

import optuna from optuna.samplers

import TPESampler

# Load full tabular data full_merged = pd.read_csv(os.path.join(OUTPUT_DIR,
"merged_tabular.csv"))

EXCLUDE_OPT = {

    "ticker", "risk_label", "gaf_path", "has_image", "has_gaf",

    "ann_vol", "n_days", "ESG Risk Level", "ESG Risk Percentile",

} feat_cols_opt = [c for c in full_merged.columns          if

c not in EXCLUDE_OPT          and

pd.api.types.is_numeric_dtype(full_merged[c])]

X_opt = full_merged[feat_cols_opt].values

y_opt = full_merged["risk_label"].map(LABEL_MAP).values

```

```

imp_opt = SimpleImputer(strategy="median") scl_opt
= StandardScaler()
X_tr_o, X_te_o, y_tr_o, y_te_o = train_test_split(
    X_opt, y_opt, test_size=0.20, stratify=y_opt, random_state=RANDOM_SEED)
X_tr_o = scl_opt.fit_transform(imp_opt.fit_transform(X_tr_o))
X_te_o = scl_opt.transform(imp_opt.transform(X_te_o))

```

```

def rf_objective(trial):

```

```

    """Optuna objective — Random Forest weighted F1 on hold-out."""
    n_est = trial.suggest_int("n_estimators", 100, 800)    depth =
    trial.suggest_int("max_depth", 3, 20)    msl =
    trial.suggest_int("min_samples_leaf", 1, 8)    mf =
    trial.suggest_categorical("max_features", ["sqrt","log2"])    crit =
    trial.suggest_categorical("criterion", ["gini","entropy"])

```

```

    rf = RandomForestClassifier(
n_estimators=n_est, max_depth=depth,
min_samples_leaf=msl, max_features=mf,
criterion=crit, class_weight="balanced",
random_state=RANDOM_SEED, n_jobs=-1)

    rf.fit(X_tr_o, y_tr_o)    preds = rf.predict(X_te_o)    return
float(f1_score(y_te_o, preds, average="weighted")) study =
optuna.create_study(
    direction="maximize",
sampler=TPESampler(seed=RANDOM_SEED),
pruner=optuna.pruners.MedianPruner(),
study_name="GROUP3_RF_OPTUNA")

```

```

t_opt = time.time() study.optimize(rf_objective, n_trials=N_OPTUNA_TRIALS,
show_progress_bar=True) elapsed_opt = round(time.time() - t_opt, 1)

trials_df = (pd.DataFrame([{
    "trial": t.number, "state": t.state.name,
    "val_f1": t.value, **t.params} for t in
study.trials])

.sort_values("val_f1", ascending=False)

.reset_index(drop=True)) trials_df.to_csv(os.path.join(OUTPUT_DIR,
"optuna_trials.csv"), index=False)

n_completed = int((trials_df["state"] == "COMPLETE").sum()) kpi5_pass
= n_completed >= 20

CNN_TUNED_VAL_ACC = CNN_BASELINE_VAL_ACC # keep variable alive for Cell 17

print(f'\n Done in {elapsed_opt}s') print(f'
Completed trials : {n_completed}') print(f'
Best F1 : {study.best_value:.4f}')
print(f' Best params :
{study.best_params}') print(f' KPI 5 (≥20)
: {'✓ PASS' if kpi5_pass else '✗ FAIL'})
print(f'\n Top 5 trials:')
print(trials_df[["trial", "state", "val_f1"]].head(5).to_string(index=False))

```

```

# Train best RF from Optuna best_params =
study.best_params      best_rf_opt      =
RandomForestClassifier(**best_params,
class_weight="balanced",
random_state=RANDOM_SEED, n_jobs=-1)
best_rf_opt.fit(X_tr_o, y_tr_o)

with open(LOG_PATH) as f: log_data = json.load(f) log_data["model_runs"].append({
    "step"      : "optuna_rf_tuning",
    "logged_at"  : datetime.datetime.now().isoformat(),
    "n_completed" : n_completed,
    "best_val_f1" : round(study.best_value, 6),
    "best_params" : study.best_params,
    "elapsed_s"   : elapsed_opt,
    "kpi5_pass"   : kpi5_pass,
}) with open(LOG_PATH, "w") as f: json.dump(log_data, f,
indent=2)

print(f'\nOptuna trials saved {OUTPUT_DIR}/optuna_trials.csv') print(f' Experiment log updated
{LOG_PATH}')

# CELL 16 — RESNET50 TRANSFER LEARNING

#

# ResNet50 pretrained on ImageNet used as:

# 1. Standalone classifier (documented baseline)

# 2. Feature extractor for Hybrid model (Cell 17)

```

```

# Phase 1: frozen backbone, train head only (15 epochs)

# Phase 2: unfreeze top 30 layers, fine-tune (LR=1e-5)


from tensorflow.keras.applications import ResNet50


print("=" * 65)

print("RESNET50 TRANSFER LEARNING ")

print("=" * 65)


def to_rgb(X):
    """(N,H,W,1) → (N,H,W,3) required by ResNet50."""
    return np.concatenate([X, X, X], axis=-1)


X_tr_rgb = to_rgb(X_train)
X_va_rgb = to_rgb(X_val) X_te_rgb = to_rgb(X_test) print(f" RGB:
train={X_tr_rgb.shape} val={X_va_rgb.shape} test={X_te_rgb.shape}") def
build_resnet50(num_classes=NUM_CLASSES, img_size=IMG_SIZE):

    """

    ResNet50 + classification head.

    Returns (full_model, backbone) separately so Phase 2 can
    unfreeze backbone layers without name-lookup issues.

    """

    backbone = ResNet50(include_top=False, weights=
"imagenet", input_shape=(img_size, img_size, 3), pooling
="avg" # GlobalAveragePooling built in → 2048-dim
    )

```

```
backbone.trainable = False    # Phase 1: frozen
```

```
inp = layers.Input(shape=(img_size, img_size, 3), name="rgb_input")    x =
backbone(inp, training=False)    x =
layers.BatchNormalization(name="res_bn")(x)    # stabilise embeddings    x =
layers.Dense(512, activation="relu",
kernel_regularizer=regularizers.l2(1e-4),                name="res_fc1")(x)    x
= layers.Dropout(0.40, name="res_drop1")(x)    x = layers.Dense(256,
activation="relu",                kernel_regularizer=regularizers.l2(1e-4),
name="res_fc2")(x)    x = layers.Dropout(0.30, name="res_drop2")(x)

out = layers.Dense(num_classes, activation="softmax", name="res_out")(x)

return models.Model(inputs=inp, outputs=out, name="ResNet50_GAF"), backbone
```

```
tf.keras.backend.clear_session() res_model, res_backbone =
build_resnet50() res_model.compile(    optimizer =
tf.keras.optimizers.Adam(learning_rate=1e-3),    loss    =
"sparse_categorical_crossentropy",    metrics    =
["accuracy"]
)
```

```
trainable_p1 = sum(p.numpy().size for p in res_model.trainable_weights) print(f"
Trainable params (Phase 1): {trainable_p1:,}")
```

```

aug_res = tf.keras.Sequential([
layers.RandomFlip("horizontal_and_vertical"),
layers.RandomRotation(0.10), layers.RandomZoom(0.05)])

ds_res = (tf.data.Dataset.from_tensor_slices((X_tr_rgb, y_train))
        .shuffle(len(X_tr_rgb), seed=RANDOM_SEED)
        .batch(32)
        .map(lambda x, y: (aug_res(x, training=True), y),
num_parallel_calls=tf.data.AUTOTUNE)      .prefetch(tf.data.AUTOTUNE))

```

— Phase 1: Head only

```

print("\n[Phase 1] Frozen backbone — head only (15 epochs) ...")

res_model.fit( ds_res, validation_data=(X_va_rgb, y_val),
epochs=15, class_weight=CLASS_WEIGHT_DICT,
    callbacks=[
callbacks.EarlyStopping(monitor="val_loss",      patience=8,
restore_best_weights=True, verbose=1),
    ],
    verbose=1)

```

— Phase 2: Unfreeze top 30 layers

```

print("\n[Phase 2] Unfreezing top 30 ResNet50 layers (LR=1e-5) ..." for
layer in res_backbone.layers:

```

```

        layer.trainable = False for layer in
res_backbone.layers[-30:]:

        layer.trainable = True

trainable_p2 = sum(p.numpy().size for p in res_model.trainable_weights) print(f"
Trainable params (Phase 2): {trainable_p2:,}") res_model.compile( optimizer
= tf.keras.optimizers.Adam(learning_rate=1e-5), loss =
"sparse_categorical_crossentropy", metrics = ["accuracy"]
)

```

```

res_hist = res_model.fit( ds_res,
validation_data=(X_va_rgb, y_val), epochs=50,
class_weight=CLASS_WEIGHT_DICT,
callbacks=[ callbacks.EarlyStopping(monitor="val_loss",
patience=10, restore_best_weights=True,
verbose=1), callbacks.ReduceLROnPlateau(monitor="val_loss",
factor=0.5,
patience=5, min_lr=1e-8, verbose=1),
],
verbose=1)

```

```

# — Evaluate

```

```

print("\nEvaluating ResNet50 on test set ...")

res_te_prob = res_model.predict(X_te_rgb, verbose=0)

res_te_pred = np.argmax(res_te_prob, axis=1)


RES_TEST_F1 = float(f1_score(y_test, res_te_pred, average="weighted")) res_te_acc
= float(accuracy_score(y_test, res_te_pred))


RES_TEST_AUC = float(roc_auc_score(y_test, res_te_prob,
                                   multi_class="ovr", average="weighted"))

EFF_VAL_F1 = float(max(res_hist.history["val_accuracy"]))

EFF_TEST_F1 = RES_TEST_F1 # alias so Cell 17/18 variables still work

EFF_TEST_AUC = RES_TEST_AUC # alias


print(f'\n ResNet50 F1(w) : {RES_TEST_F1:.4f}')

print(f' ResNet50 Acc : {res_te_acc:.4f}') print(f'
ResNet50 AUC : {RES_TEST_AUC:.4f}')

print("\n Classification report (test):") print(classification_report(y_test,
res_te_pred, target_names=CLASS_NAMES))


# Expose for Cell 17 hybrid (same variable names as before) eff_model
= res_model # alias — Cell 17 uses eff_model eff_backbone =
res_backbone # alias


res_model.save(os.path.join(OUTPUT_DIR, "resnet50_gaf.keras"))


with open(LOG_PATH) as f: log_data = json.load(f) log_data["model_runs"].append({
    "step" : "resnet50_transfer",

```

```

    "logged_at": datetime.datetime.now().isoformat(),

    "test_f1" : round(RES_TEST_F1, 6),

    "test_auc" : round(RES_TEST_AUC, 6),

    "kpi2_pass": RES_TEST_AUC >= 0.85,

})

with open(LOG_PATH, "w") as f: json.dump(log_data, f, indent=2) print(f"\nResNet50
saved → {OUTPUT_DIR}/resnet50_gaf.keras") print(f"Embeddings (2048-dim) ready
for Cell 17 hybrid.")

# — ResNet50 Training Plots


---



fig, axes = plt.subplots(1, 2, figsize=(14, 5))

fig.suptitle("ResNet50 Transfer Learning — Training History (Phase 2)", fontsize=13,
fontweight="bold")

epochs_range = range(1, len(res_hist.history["accuracy"]) + 1)

# Plot 1: Accuracy

axes[0].plot(epochs_range, res_hist.history["accuracy"], label="Train Accuracy", color="royalblue",
linewidth=2)

axes[0].plot(epochs_range, res_hist.history["val_accuracy"], label="Val Accuracy",
color="darkorange", linewidth=2, linestyle="--")

axes[0].axhline(y=CNN_BASELINE_VAL_ACC, color="red", linestyle=":", linewidth=1.5,
label=f"CNN Baseline ({CNN_BASELINE_VAL_ACC:.4f})")

axes[0].axhline(y=res_te_acc, color="green", linestyle=":", linewidth=1.5, label=f"Test Acc
({res_te_acc:.4f})")

axes[0].set_title("Accuracy over Epochs")

axes[0].set_xlabel("Epoch")

axes[0].set_ylabel("Accuracy")

```

```

axes[0].legend() axes[0].grid(True,
alpha=0.3)

# Plot 2: Loss

axes[1].plot(epochs_range, res_hist.history["loss"],          label="Train Loss", color="royalblue",
linewidth=2)

axes[1].plot(epochs_range, res_hist.history["val_loss"], label="Val Loss", color="darkorange",
linewidth=2, linestyle="--") axes[1].set_title("Loss over Epochs") axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Loss") axes[1].legend() axes[1].grid(True, alpha=0.3)

plt.tight_layout() plt.show() plot_path = os.path.join(OUTPUT_DIR,
"resnet50_training_history.png") fig.savefig(plot_path, dpi=150)
print(f"Training history plot saved → {plot_path}")

# — Confusion Matrix

```

```

fig2, ax = plt.subplots(figsize=(7, 5)) cm = confusion_matrix(y_test,
res_te_pred) sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES, ax=ax)

ax.set_title(f"ResNet50 — Confusion Matrix (Test) | F1={RES_TEST_F1:.4f}
AUC={RES_TEST_AUC:.4f}")

ax.set_xlabel("Predicted") ax.set_ylabel("Actual") plt.tight_layout()

plt.show() cm_path = os.path.join(OUTPUT_DIR,
"resnet50_confusion_matrix.png") fig2.savefig(cm_path, dpi=150)

print(f" Confusion matrix saved → {cm_path}")

# CELL 17 — HYBRID MODEL: Frozen ResNet50 Embeddings + Tabular → XGBoost

#

```

```

# TRUE HYBRID: two modalities fused into XGBoost.

# Image features : frozen ResNet50 'res_bn' layer (2048-dim)

#           Never trained on our labels → no leakage

# Tabular features: ESG component scores + financial + price metrics

# Fusion      : concatenate → XGBoost


print("=" * 65)

print("HYBRID MODEL — ResNet50 Embeddings + Tabular → XGBoost")

print("=" * 65)


# — Step 1: Build frozen ResNet50 feature extractor —————
print("\n[1/5] Building frozen image feature extractor")

feature_extractor = models.Model(  inputs =
eff_model.input,  outputs =
eff_model.get_layer("res_bn").output,  name =
"res_embedder"
)

for layer in feature_extractor.layers:

    layer.trainable = False emb_dim = feature_extractor.output_shape[-1]

print(f" Embedding dim : {emb_dim} (frozen ResNet50, ImageNet weights)")

# — Step 2: Extract image embeddings for all splits

————— print("\n[2/5] Extracting image embeddings ")

train_emb = feature_extractor.predict(X_tr_rgb, batch_size=32, verbose=0)
val_emb  = feature_extractor.predict(X_va_rgb, batch_size=32, verbose=0)
test_emb = feature_extractor.predict(X_te_rgb, batch_size=32, verbose=0)

print(f" Emb shapes: train={train_emb.shape} val={val_emb.shape}
test={test_emb.shape}")

```

— Step 3: Build leakage-free tabular features —————

```
print("\n[3/5] Building tabular feature matrix ") full_merged =
```

```
pd.read_csv(os.path.join(OUTPUT_DIR, "merged_tabular.csv"))
```

```
EXCLUDE_H = {
```

```
    "ticker", "risk_label", "gaf_path", "has_image", "has_gaf",
```

```
    "n_days",
```

```
    "Total ESG Risk score", # used to CREATE the ESG Risk Level label
```

```
    "overallRisk",        # parallel risk label from DS2
```

```
    "percentile",         # ESG risk percentile rank  $\approx$  label
```

```
    "ESG Risk Level",     # the raw label itself
```

```
    "ESG Risk Percentile", # derived from Total ESG Risk score
```

```
} feature_cols_h = [c for c in full_merged.columns
```

```
if c not in EXCLUDE_H and
```

```
pd.api.types.is_numeric_dtype(full_merged[c])]
```

```
train_split = pd.read_csv(os.path.join(OUTPUT_DIR, "train_split.csv"))
```

```
val_split = pd.read_csv(os.path.join(OUTPUT_DIR, "val_split.csv")) test_split
```

```
= pd.read_csv(os.path.join(OUTPUT_DIR, "test_split.csv")) fm =
```

```
full_merged.set_index("ticker")
```

```
def get_tab(split_df):
```

```
    """Align tabular rows to split ticker order and return X, y."""
```

```
    tab = fm.reindex(split_df["ticker"].values)[feature_cols_h].values
```

```
    y = split_df["risk_label"].map(LABEL_MAP).values
```

```
    return tab, y
```

```

X_tr_tab_h, y_tr_h = get_tab(train_split)

X_va_tab_h, y_va_h = get_tab(val_split)

X_te_tab_h, y_te_h = get_tab(test_split)


imp_h = SimpleImputer(strategy="median") scl_h
= StandardScaler()

X_tr_tab_h = scl_h.fit_transform(imp_h.fit_transform(X_tr_tab_h))

X_va_tab_h = scl_h.transform(imp_h.transform(X_va_tab_h))

X_te_tab_h = scl_h.transform(imp_h.transform(X_te_tab_h))


X_tr_h = np.hstack([X_tr_tab_h, train_emb])

X_va_h    =    np.hstack([X_va_tab_h,    val_emb])    X_te_h    =
np.hstack([X_te_tab_h, test_emb]) print(f"    Tabular    features    :
{X_tr_tab_h.shape[1]}") print(f"    Image embeddings    : {emb_dim}")
print(f"    Total    hybrid    dims    : {X_tr_h.shape[1]}") print(f"
Train={len(X_tr_h)} Val={len(X_va_h)} Test={len(X_te_h)}") # —
Step 4: Train XGBoost

```

```

print("\n[4/5] Training XGBoost on hybrid feature matrix ") xgb_model
= xgb.XGBClassifier(
    objective        = "multi:softprob",
    num_class        = NUM_CLASSES,
    n_estimators      = 800,    max_depth
= 4,    learning_rate    = 0.03,
    min_child_weight  = 2,    gamma
= 0.3,    subsample      = 0.80,

```

```

colsample_bytree    = 0.80,    reg_alpha
= 0.3,    reg_lambda        = 2.0,
scale_pos_weight    = 3,    eval_metric
= "mlogloss",    early_stopping_rounds =
40,    random_state        =
RANDOM_SEED,
    n_jobs            = -1,
)
xgb_model.fit(    X_tr_h, y_tr_h,
eval_set = [(X_va_h, y_va_h)],
verbose = 100,
)
print(f"\n Best iteration: {xgb_model.best_iteration}")

# — Step 5: Evaluate with threshold tuning for High-risk recall —————
print("\n[5/5] Evaluating hybrid model ") h_va_prob_raw =
xgb_model.predict_proba(X_va_h) h_te_prob_raw =
xgb_model.predict_proba(X_te_h)

print("\n Tuning High-risk classification threshold on val set") best_threshold =
0.333 best_combo    = 0.0 lm_map    = [i for i in range(NUM_CLASSES) if i
!= LABEL_MAP["High"]]

def apply_threshold(proba, thresh):    """Apply
custom High-risk threshold."""    return
np.where(    proba[:, LABEL_MAP["High"]]
>= thresh,

```

```

    LABEL_MAP["High"],
    [lm_map[p] for p in np.argmax(proba[:, lm_map], axis=1)]
)

from sklearn.metrics import recall_score for
thresh in np.arange(0.10, 0.50, 0.01):

    preds_va = apply_threshold(h_va_prob_raw, thresh)    hi_rec    = recall_score(y_va_h,
preds_va, labels=[LABEL_MAP["High"]], average=None)[0]    wfl    = f1_score(y_va_h,
preds_va, average="weighted")    if hi_rec >= 0.75 and wfl > best_combo:

        best_combo    = wfl

        best_threshold = thresh

print(f" Best threshold: {best_threshold:.2f} "    f"(val
High-recall  $\geq$  0.75, F1 = {best_combo:.4f})")

h_va_pred = apply_threshold(h_va_prob_raw, best_threshold) h_te_pred
= apply_threshold(h_te_prob_raw, best_threshold)

HYBRID_VAL_F1 = float(f1_score(y_va_h, h_va_pred, average="weighted"))
HYBRID_TEST_F1 = float(f1_score(y_te_h, h_te_pred, average="weighted"))
hybrid_te_acc = float(accuracy_score(y_te_h, h_te_pred)) hybrid_te_auc =
float(roc_auc_score(y_te_h, h_te_prob_raw,

                    multi_class="ovr", average="weighted"))

hybrid_va_auc = float(roc_auc_score(y_va_h, h_va_prob_raw,

                    multi_class="ovr", average="weighted")) cr_h    =

classification_report(y_te_h,                    h_te_pred,

```

```

target_names=CLASS_NAMES, output_dict=True) high_recall_h =
cr_h["High"]["recall"]

print(f'\n Hybrid Val F1(w): {HYBRID_VAL_F1:.4f} AUC: {hybrid_va_auc:.4f}')
print(f' Hybrid Test F1(w): {HYBRID_TEST_F1:.4f} Acc: {hybrid_te_acc:.4f} "
f'AUC: {hybrid_te_auc:.4f}') print(f' High-Risk Recall : {high_recall_h:.4f}')
print("\n Classification report (test):") print(classification_report(y_te_h, h_te_pred,
target_names=CLASS_NAMES)) best_uni_test = max(

RF_TEST_F1,

CNN_BASELINE_F1 if "CNN_BASELINE_F1" in dir() else 0.0,

RES_TEST_F1,

)

KPI4_DELTA = HYBRID_TEST_F1 - best_uni_test kpi4_pass

= KPI4_DELTA >= 0.03

print(f'\n — KPI Results —')

print(f' KPI 1 (F1≥0.80) : {'✓ PASS' if HYBRID_TEST_F1>=0.80 else 'X FAIL'} →
{HYBRID_TEST_F1:.4f}')

print(f' KPI 2 (AUC≥0.85) : {'✓ PASS' if hybrid_te_auc>=0.85 else 'X FAIL'} →
{hybrid_te_auc:.4f}')

print(f' KPI 3 (HiRec≥0.75) : {'✓ PASS' if high_recall_h>=0.75 else 'X FAIL'} →
{high_recall_h:.4f}') print(f' KPI 4 (delta≥+0.03) : {'✓ PASS' if kpi4_pass else 'X FAIL'} →
{KPI4_DELTA:+.4f} " f'(Hybrid {HYBRID_TEST_F1:.4f} vs best unimodal
{best_uni_test:.4f}'))

# — Plots

```

```

# 1. XGBoost Log-Loss Curve xgb_results = xgb_model.evals_result()

fig_loss, ax_loss = plt.subplots(figsize=(10, 5))

ax_loss.plot(xgb_results["validation_0"]["mlogloss"],          label="Val Log-
Loss", color="darkorange", linewidth=2)

ax_loss.axvline(x=xgb_model.best_iteration, color="green", linestyle="--",
linewidth=1.5, label=f"Best iteration ( {xgb_model.best_iteration} )")

ax_loss.set_title("XGBoost Hybrid — Validation Log-Loss over Boosting
Rounds",          fontweight="bold", fontsize=13)

ax_loss.set_xlabel("Boosting Round") ax_loss.set_ylabel("Log-Loss")

ax_loss.legend() ax_loss.grid(True, alpha=0.3) plt.tight_layout() plt.show()

fig_loss.savefig(os.path.join(OUTPUT_DIR, "hybrid_xgb_logloss.png"), dpi=150,
bbox_inches="tight")    print(f"✓    Log-loss    curve    saved    →
{OUTPUT_DIR}/hybrid_xgb_logloss.png")

plt.close(fig_loss)


# 2. Feature Importances feat_names_h = feature_cols_h + [f'res_emb_{i}' for i in
range(emb_dim)] fi_h = pd.Series(xgb_model.feature_importances_,
index=feat_names_h).nlargest(25) fig_fi, ax_fi = plt.subplots(figsize=(10, 9))

fi_h.sort_values().plot.barh(ax=ax_fi, color="#FF7043", edgecolor="white")

ax_fi.set_title("Hybrid ResNet50+XGBoost — Top 25 Feature Importances",
fontweight="bold", fontsize=13) ax_fi.set_xlabel("XGBoost Feature Importance")

plt.tight_layout() plt.show()

fig_fi.savefig(os.path.join(OUTPUT_DIR, "hybrid_feature_importance.png"), dpi=150,
bbox_inches="tight") print(f"✓ Feature importance saved →
{OUTPUT_DIR}/hybrid_feature_importance.png")

plt.close(fig_fi)

```

```

# 3. Confusion Matrix fig_cm, ax_cm = plt.subplots(figsize=(7, 5)) cm_h =
confusion_matrix(y_te_h, h_te_pred) sns.heatmap(cm_h, annot=True, fmt="d",
cmap="Blues", xticklabels=CLASS_NAMES,
yticklabels=CLASS_NAMES, ax=ax_cm) ax_cm.set_title(f'Hybrid Model —
Confusion Matrix (Test) | " f"F1={HYBRID_TEST_F1:.4f}
AUC={hybrid_te_auc:.4f}",
fontweight="bold")
ax_cm.set_xlabel("Predicted")
ax_cm.set_ylabel("Actual")
plt.tight_layout() plt.show()
fig_cm.savefig(os.path.join(OUTPUT_DIR, "hybrid_confusion_matrix.png"), dpi=150,
bbox_inches="tight") print(f'✓ Confusion matrix saved →
{OUTPUT_DIR}/hybrid_confusion_matrix.png") plt.close(fig_cm)

# 4. KPI Summary Bar Chart kpi_labels = ["KPI 1\nF1 ≥ 0.80",
"KPI 2\nAUC ≥ 0.85", "KPI 3\nHi-Rec ≥ 0.75", "KPI
4\nDelta ≥ +0.03"] kpi_values = [HYBRID_TEST_F1,
hybrid_te_auc, high_recall_h, best_uni_test +
KPI4_DELTA] kpi_targets = [0.80, 0.85, 0.75, best_uni_test + 0.03]
kpi_passed = [HYBRID_TEST_F1 >= 0.80, hybrid_te_auc >= 0.85,
high_recall_h >= 0.75, kpi4_pass] colors =
["#2ECC71" if p else "#E74C3C" for p in kpi_passed] fig_kpi,
ax_kpi = plt.subplots(figsize=(10, 5)) bars =
ax_kpi.bar(kpi_labels, kpi_values, color=colors,
edgecolor="white", width=0.5) for bar, target in zip(bars,
kpi_targets):

```

```

ax_kpi.axhline(y=target, color="black", linestyle="--", linewidth=1.2, alpha=0.6) for
bar, val, passed in zip(bars, kpi_values, kpi_passed):

    ax_kpi.text(bar.get_x() + bar.get_width() / 2,                bar.get_height() + 0.01,
f'{val:.4f}\n({'PASS' if passed else 'FAIL'})',                ha="center", va="bottom",
    fontsize=11, fontweight="bold") ax_kpi.set_ylim(0, 1.15) ax_kpi.set_title("Hybrid Model
— KPI Results (Test Set)", fontsize=13, fontweight="bold") ax_kpi.set_ylabel("Score")

ax_kpi.grid(axis="y", alpha=0.3) plt.tight_layout() plt.show()

fig_kpi.savefig(os.path.join(OUTPUT_DIR, "hybrid_kpi_summary.png"), dpi=150,
bbox_inches="tight") print(f"✓ KPI summary chart saved →
{OUTPUT_DIR}/hybrid_kpi_summary.png")

plt.close(fig_kpi)

# — Save artefacts

```

```

joblib.dump(xgb_model, os.path.join(OUTPUT_DIR, "xgb_hybrid.pkl"))

preprocessor_dir = os.path.join(OUTPUT_DIR, "preprocessors")

os.makedirs(preprocessor_dir, exist_ok=True) joblib.dump(imp_h,
os.path.join(preprocessor_dir, "num_imputer_h.pkl")) joblib.dump(scl_h,
os.path.join(preprocessor_dir, "scaler_h.pkl")) joblib.dump(feature_cols_h,
os.path.join(preprocessor_dir, "feature_cols_h.pkl"))

with open(LOG_PATH) as f: log_data = json.load(f) log_data["model_runs"].append({
    "step"          : "hybrid_resnet_xgboost",
    "logged_at"     : datetime.datetime.now().isoformat(),
    "n_features"    : int(X_tr_h.shape[1]),
    "emb_dim"       : emb_dim,
    "tabular_dims"  : X_tr_tab_h.shape[1],
    "threshold"     : round(float(best_threshold), 3),

```

```

"val_f1_weighted" : round(HYBRID_VAL_F1, 6),
"test_f1_weighted": round(HYBRID_TEST_F1, 6),
"test_accuracy"   : round(hybrid_te_acc, 6),
"test_auc_roc"    : round(hybrid_te_auc, 6),
"high_recall"     : round(high_recall_h, 6),
"kpi4_delta"      : round(KPI4_DELTA, 6),
"kpi1_pass"       : HYBRID_TEST_F1 >= 0.80,
"kpi2_pass"       : hybrid_te_auc >= 0.85,
"kpi3_pass"       : high_recall_h >= 0.75,
"kpi4_pass"       : kpi4_pass,
}) with open(LOG_PATH, "w") as f: json.dump(log_data, f,
indent=2)          print(f"\nHybrid          model          saved
{OUTPUT_DIR}/xgb_hybrid.pkl") print(f"Preprocessors saved
{preprocessor_dir}/") print(f"Experiment log {LOG_PATH}")

```

CELL 18 — COMPREHENSIVE MODEL EVALUATION

Evaluates all 4 models on held-out test set.

```
print("=" * 65)
```

```
print("FULL MODEL EVALUATION ")
```

```
print("=" * 65)
```

```
def evaluate(name, y_true, y_pred, y_proba):
```

```
    cr = classification_report(y_true,y_pred,target_names=CLASS_NAMES,output_dict=True)
```

```
    return {
```

```
        "model"      : name,
```

```

"weighted_f1" : float(f1_score(y_true,y_pred,average="weighted")),
"accuracy"    : float(accuracy_score(y_true,y_pred)),
"auc_roc"     : float(roc_auc_score(y_true,y_proba,multi_class="ovr",average="weighted")),
"high_recall" : cr["High"]["recall"],
"high_precision": cr["High"]["precision"],
"_y_true"     : y_true, "_y_pred": y_pred, "_y_proba": y_proba,
}

print("\n[1/5] Inference ") cnn_prob =
cnn_model.predict(X_test,verbose=0) cnn_pred =
np.argmax(cnn_prob,axis=1) eff_prob =
eff_model.predict(X_te_rgb,verbose=0) eff_pred =
np.argmax(eff_prob,axis=1)

results = [ evaluate("Custom CNN (tuned)", y_test, cnn_pred, cnn_prob),
evaluate("Random Forest", y_te_tab, rf_te_pred, rf_te_prob),
evaluate("Hybrid ResNet50+XGBoost", y_te_h, h_te_pred, h_te_prob_raw),
evaluate("ResNet50", y_test, eff_pred, eff_prob), ]

print("\n[2/5] Summary table...") print(f" {'Model':<24} {'F1(w)':>7}
{'Acc':>7} {'AUC':>7} {'HiRec':>7}") print(" " + "-"*56) for r in results:
    print(f" {'r['model']':<24} {'r['weighted_f1']':>7.4f} {'r['accuracy']':>7.4f} "
f"{'r['auc_roc']':>7.4f} {'r['high_recall']':>7.4f}")

print("\n[3/5] KPI pass/fail ") hybrid_r = next(r for r in results if "Hybrid" in
r["model"]) best_uni = max(r["weighted_f1"] for r in results if "Hybrid" not in
r["model"]) delta = hybrid_r["weighted_f1"] - best_uni

```

```

print(f" KPI 1 F1 $\geq$ 0.80 : {'PASS' if hybrid_r['weighted_f1'] $\geq$ 0.80 else 'FAIL'} →
{hybrid_r['weighted_f1']:.4f}")

print(f" KPI 2 AUC $\geq$ 0.85 : {'PASS' if hybrid_r['auc_roc'] $\geq$ 0.85 else 'FAIL'} →
{hybrid_r['auc_roc']:.4f}")

print(f" KPI 3 HiRec $\geq$ 0.75 : {'PASS' if hybrid_r['high_recall'] $\geq$ 0.75 else 'FAIL'} →
{hybrid_r['high_recall']:.4f}") print(f" KPI 4 delta $\geq$ +0.03 : {'PASS' if delta $\geq$ 0.03
else 'FAIL'} → {delta:+.4f}")

print("\n[4/5] Confusion matrices ") fig,axes
= plt.subplots(2,2,figsize=(14,12))

fig.suptitle("Confusion Matrices — All
Models (Test Set) ",
fontsize=14,fontweight="bold") for ax,r in
zip(axes.flat,results):

    cm = confusion_matrix(r["_y_true"],r["_y_pred"])

    ConfusionMatrixDisplay(cm,display_labels=CLASS_NAMES).plot(
ax=ax,colorbar=False,cmap="Blues") ax.set_title(f'{r["model"]}\nF1={r["weighted_f1"]:.3f}
AUC={r["auc_roc"]:.3f}', fontweight="bold",fontsize=11)

    n = len(CLASS_NAMES)

    for i in range(n + 1):

        ax.axhline(i - 0.5, color='black', linewidth=1.5) ax.axvline(i - 0.5, color='black',
linewidth=1.5) plt.tight_layout()

plt.savefig(os.path.join(OUTPUT_DIR,"confusion_matrices.png"),dpi=150,bbox_inches="tight")

display(fig); plt.close(fig)

print("\n Detailed reports:") for
r in results:

```

```

print(f'\n          —          {r['model']}'          —')

print(classification_report(r["_y_true"],r["_y_pred"],target_names=CLASS_NAMES))

print("[5/5]    Comparison    bar    chart    ...")    metrics    =

["weighted_f1","accuracy","auc_roc","high_recall"]    mlabels    =

["Weighted F1","Accuracy","AUC-ROC","High-Risk Recall"]    mnames

= [r["model"] for r in results] x,w    = np.arange(len(mnames)), 0.18

fig2,ax2 = plt.subplots(figsize=(15,7)) for

i,(m,l) in enumerate(zip(metrics,mlabels)):

    vals = [r[m] for r in results]    bars =

ax2.bar(x+i*w,vals,w,label=l,edgecolor="white",alpha=0.88)    for

bar,v in zip(bars,vals):

    ax2.text(bar.get_x()+bar.get_width()/2,bar.get_height()+0.005,

f'{v:.3f}',ha="center",va="bottom",fontsize=7,fontweight="bold")

ax2.axhline(0.80,color="red",ls="--",lw=1.3,alpha=0.7,label="KPI1 (0.80)")

ax2.axhline(0.85,color="green",ls="--",lw=1.3,alpha=0.7,label="KPI2 (0.85)")

ax2.axhline(0.75,color="orange",ls="--",lw=1.3,alpha=0.5,label="KPI3 (0.75)")

ax2.set_xticks(x+1.5*w); ax2.set_xticklabels(mnames,rotation=12,fontsize=10)

ax2.set_ylim(0,1.12); ax2.set_ylabel("Score",fontsize=12) ax2.set_title("Model Comparison —

All KPI Metrics (Test Set)",    fontweight="bold",fontsize=13) ax2.legend(loc="lower

right",fontsize=9,ncol=3) plt.tight_layout()

plt.savefig(os.path.join(OUTPUT_DIR,"model_comparison.png"),dpi=150,bbox_inches="tight")

display(fig2); plt.close(fig2)

safe = [{k:v for k,v in r.items() if not k.startswith("_")} for r in results]

pd.DataFrame(safe).to_csv(os.path.join(OUTPUT_DIR,"model_results.csv"),index=False)

```

```

with open(LOG_PATH) as f: log_data=json.load(f) log_data["model_runs"].append({
    "step":"full_evaluation","logged_at":datetime.datetime.now().isoformat(),
    "models":safe,"kpi1":hybrid_r["weighted_f1"]>=0.80,
    "kpi2":hybrid_r["auc_roc"]>=0.85,"kpi3":hybrid_r["high_recall"]>=0.75,
    "kpi4":delta>=0.03}) with open(LOG_PATH,"w") as f:
    json.dump(log_data,f,indent=2) print(f"\n✓ Results →
    {OUTPUT_DIR}/model_results.csv")

```

CELL 19 — SHAP EXPLAINABILITY

KPI 9: 100% of test predictions explained via native SHAP.

```
import io
```

```
print("=" * 65)
```

```
print("SHAP EXPLAINABILITY ")
```

```
print("=" * 65)
```

```
print("\n[1/4] Computing SHAP values") booster =
```

```
    xgb_model.get_booster()          dmatrix          =
```

```
    xgb.DMatrix(X_te_h)              contribs          =
```

```
    booster.predict(dmatrix, pred_contribs=True)
```

```
feat_names = feature_cols_h + [f'eff_{i}' for i in range(emb_dim)] n_feat
```

```
= X_te_h.shape[1]
```

```
if contribs.ndim == 3:
```

```

sv_cls      = [contribs[:, c, :n_feat] for c in range(NUM_CLASSES)]
base_vals = [float(contribs[0, c, n_feat]) for c in range(NUM_CLASSES)]
else:

    contribs_r = contribs.reshape(len(X_te_h), NUM_CLASSES, n_feat + 1)
sv_cls      = [contribs_r[:, c, :n_feat] for c in range(NUM_CLASSES)]    base_vals
= [float(contribs_r[0, c, n_feat]) for c in range(NUM_CLASSES)]

n_explained      =          len(X_te_h)          print(f"
{n_explained}/{n_explained} predictions explained ")

# Helper: render a figure to numpy image def
fig_to_img(fig):          buf  =  io.BytesIO()
fig.savefig(buf,  dpi=130,  bbox_inches="tight")
buf.seek(0)  img = plt.imread(buf)  buf.close()
return img

print("\n[2/4] Rendering individual SHAP plots")

# Plot A Bar: Low Risk fig_a, ax_a = plt.subplots(figsize=(9, 6))
shap.summary_plot(sv_cls[LABEL_MAP["Low"]], X_te_h,
feature_names=feat_names,
                plot_type="bar", max_display=12, show=False)
ax_a.set_xlabel("mean SHAP value ", fontsize=10) ax_a.set_title("SHAP Bar -
Low Risk", fontweight="bold", fontsize=12, pad=10) plt.tight_layout() img_a =
fig_to_img(fig_a); plt.close(fig_a)

```

```
# Plot B — Bar: Medium Risk fig_b, ax_b = plt.subplots(figsize=(9, 6))
shap.summary_plot(sv_cls[LABEL_MAP["Medium"]], X_te_h, feature_names=feat_names,
                  plot_type="bar", max_display=12, show=False) ax_b.set_xlabel("mean
SHAP value ", fontsize=10) ax_b.set_title("SHAP Bar — Medium Risk",
fontweight="bold", fontsize=12, pad=10) plt.tight_layout() img_b = fig_to_img(fig_b);
plt.close(fig_b)
```

```
# Plot C — Beeswarm: High Risk fig_c, ax_c = plt.subplots(figsize=(9, 6))
shap.summary_plot(sv_cls[LABEL_MAP["High"]], X_te_h, feature_names=feat_names,
                  max_display=12, show=False) ax_c.set_xlabel("SHAP value (impact on
model output)", fontsize=10) ax_c.set_title("SHAP Beeswarm — High Risk",
fontweight="bold", fontsize=12, pad=10) plt.tight_layout() img_c = fig_to_img(fig_c);
plt.close(fig_c)
```

```
# Plot D — Waterfall: first High-risk example high_idx =
np.where(y_te_h == LABEL_MAP["High"])[0]
if len(high_idx) > 0:
    exp = shap.Explanation(
        values =
sv_cls[LABEL_MAP["High"]][high_idx[0]],
        base_values
= base_vals[LABEL_MAP["High"]],
        data =
X_te_h[high_idx[0]],
        feature_names = feat_names
    )
    fig_d, ax_d = plt.subplots(figsize=(9, 6))
shap.waterfall_plot(exp, max_display=12, show=False)
ax_d.set_title("SHAP Waterfall — High-Risk Example",
fontweight="bold", fontsize=12, pad=10)
```

```

plt.tight_layout()    img_d = fig_to_img(fig_d);

plt.close(fig_d) else:

    img_d = None


print("[3/4] Compositing 2×2 grid ")


fig_grid,  axes  =  plt.subplots(2,  2,  figsize=(20,  16))

fig_grid.suptitle("SHAP Explainability — XGBoost Hybrid Model",
fontsize=15,          fontweight="bold",          y=1.01)

fig_grid.patch.set_facecolor("white")

plt.subplots_adjust(hspace=0.08, wspace=0.06)


# Divider lines between panels fig_grid.add_artist(plt.Line2D([0.5,
0.5], [0.02, 0.98],
transform=fig_grid.transFigure,
color='#cccccc', linewidth=1.2))

fig_grid.add_artist(plt.Line2D([0.02, 0.98], [0.5, 0.5],
transform=fig_grid.transFigure,
color='#cccccc', linewidth=1.2)) for ax, img in zip(axes.flat,
[img_a, img_b, img_c, img_d]):    if img is not None:
ax.imshow(img)

    else:

        ax.text(0.5, 0.5, "No High-risk samples", ha="center", va="center",
fontsize=12, color="gray")    ax.axis("off")


print("[4/4] Saving ") grid_path =
os.path.join(OUTPUT_DIR, "shap_grid.png")

```

```

plt.savefig(grid_path, dpi=150, bbox_inches="tight")

display(fig_grid) plt.close(fig_grid)

# Save individual PNGs + CSV for c_idx, c_name in
enumerate(CLASS_NAMES):

    fig_i, ax_i = plt.subplots(figsize=(10, 7))

    shap.summary_plot(sv_cls[c_idx], X_te_h, feature_names=feat_names,
plot_type="bar", max_display=15, show=False)

    ax_i.set_xlabel("mean(|SHAP value|)", fontsize=10)

    ax_i.set_title(f"SHAP Feature Importance — {c_name} Risk",
fontweight="bold", fontsize=13)    plt.tight_layout()

    plt.savefig(os.path.join(OUTPUT_DIR, f"shap_bar_{c_name}.png"),
dpi=150, bbox_inches="tight")

    plt.close(fig_i)

pd.DataFrame(sv_cls[1],                                columns=feat_names).to_csv(
os.path.join(OUTPUT_DIR, "shap_values.csv"), index=False)

mean_abs = np.abs(np.stack(sv_cls, axis=2)).mean(axis=(0, 2))

top10    = pd.Series(mean_abs, index=feat_names).nlargest(10)

print("\n Top 10 global features:") for f, v in top10.items():

    print(f"    {f[:55]:<56} {v:.5f}")

with open(LOG_PATH) as f: log_data = json.load(f) log_data["model_runs"].append({

    "step": "shap", "logged_at": datetime.datetime.now().isoformat(),

    "n_explained": n_explained, "kpi9_pass": True,

```

```

        "top10": top10.index.tolist()) with open(LOG_PATH, "w") as f:

json.dump(log_data, f, indent=2)


print(f'\n SHAP complete ')


# CELL 20 — FINAL KPI DASHBOARD & REPRODUCIBILITY CHECK


print("=" * 70)

print("FINAL KPI DASHBOARD GROUP 3")

print("=" * 70)


with open(LOG_PATH) as f: log_data = json.load(f) eval_run
= next((r for r in reversed(log_data["model_runs"])          if
r["step"]=="full_evaluation"), None)


def _m(model_sub, key, d=0.0):    if
eval_run is None: return d    for m in
eval_run.get("models",[]):

    if model_sub.lower() in m.get("model","").lower(): return m.get(key,d)
return d


kpi_rows = [

("KPI 1", "Hybrid F1(w)  $\geq$  0.80", _m("Hybrid", "weighted_f1"), 0.80),

("KPI 2", "Hybrid AUC  $\geq$  0.85", _m("Hybrid", "auc_roc"), 0.85),

("KPI 3", "Hybrid High-Recall  $\geq$  0.75", _m("Hybrid", "high_recall"), 0.75),

("KPI 4", "Hybrid delta  $\geq$  +0.03", KPI4_DELTA, 0.03),

```

```

("KPI 5","Optuna trials  $\geq$  20", n_completed, 20), ("KPI
6","Companies  $\geq$  400", 505, 400),
("KPI 7","App latency < 3s", None, None),
("KPI 8","UAT rating  $\geq$  4.0/5", None, None),
("KPI 9","SHAP coverage 100%", 1.0, 1.0),
("KPI 10","Reproducibility seed=42", 1.0, 1.0),
]

```

```

print(f'\n {kpi:<8} {Description:<40} {Value:>9} {Status}')

```

```

print(" " + "-"*65) for kpi,desc,val,thr in kpi_rows:

```

```

    if thr is None: status,vstr = "- (app)", "N/A" elif
isinstance(val,(int,float)) and isinstance(thr,(int,float)):
status = "PASS" if val>=thr else " FAIL" vstr =
f'{val:.4f}' if isinstance(val,float) else str(val) else:
status,vstr = "-", "-" print(f' {kpi:<8} {desc:<40}
{vstr:>9} {status}')

```

```

print("\n" + "="*70)

```

```

print("OUTPUT ARTEFACTS")

```

```

print("="*70) for fname in

```

```

sorted(os.listdir(OUTPUT_DIR)): fpath =

```

```

os.path.join(OUTPUT_DIR,fname)

```

```

if os.path.isfile(fpath):

```

```

    print(f' {fname:<50} {os.path.getsize(fpath):>10,} bytes')

```

```

print("\n" + "="*70) print("KPI 10 —
REPRODUCIBILITY")

print("="*70)

np.random.seed(RANDOM_SEED)

draw = np.random.randint(0,100,5).tolist()

expected = [51,92,14,71,60] print(f"
seed={RANDOM_SEED} → {draw}")

print(f" Expected {expected}") print(f" KPI 10 : {'PASS'
if draw==expected else 'FAIL'}") print("\n ALL PIPELINE
STEPS COMPLETE")

```

CELL 21 — STREAMLIT APP ARTEFACT CHECK

```

print("=" * 65)

print("STREAMLIT APP ARTEFACT VERIFICATION")

print("=" * 65)

```

```

REQUIRED = [
    os.path.join(OUTPUT_DIR, "xgb_hybrid.pkl"),
    os.path.join(OUTPUT_DIR, "resnet50_gaf.keras"),
    os.path.join(OUTPUT_DIR, "cnn_baseline.keras"),
    os.path.join(OUTPUT_DIR, "random_forest.pkl"),
    os.path.join(preprocessor_dir, "num_imputer_h.pkl"),
    os.path.join(preprocessor_dir, "scaler_h.pkl"), os.path.join(preprocessor_dir,
"feature_cols_h.pkl"), os.path.join(OUTPUT_DIR,
"model_results.csv"), os.path.join(OUTPUT_DIR,
"experiment_log.json"),

```

```
]
```

```
print("\n Artefact check:")
```

```
all_ok = True for path in
```

```
REQUIRED:
```

```
    exists = os.path.exists(path)    if not exists:    all_ok = False    size =  
f'({os.path.getsize(path):,} bytes)' if exists else "(MISSING)"    print(f'  
{ 'Yes' if exists else 'Fail' } {os.path.basename(path):<45} {size} ') status =  
"Ready to deploy" if all_ok else " Re-run missing cells" print(f"\n {status} ")  
print("\n App: streamlit run app.py") print(" KPI 8: 5-star UAT widget  
saves to output/uat_results.csv")
```

```
# Commented out IPython magic to ensure Python compatibility.
```

```
# %%writefile app.py
```

```
# #
```

```
=====
```

```
# # app.py — GROUP 3: ESG + GAF Stock Risk Classification
```

```
# #
```

```
=====
```

```
#
```

```
# import os, warnings
```

```
# warnings.filterwarnings("ignore")
```

```
#
```

```
# import numpy as np
```

```
# import pandas as pd
```

```
# import matplotlib
```

```

# matplotlib.use("Agg")

# import matplotlib.pyplot as plt

# import streamlit as st

# import joblib

# import shap

# import datetime

# import csv

#

# st.set_page_config(

#     page_title = "ESG Stock Risk Classifier — Group 3",

#     page_icon = " ",

#     layout = "wide",

#     initial_sidebar_state = "expanded",

# )

#

# st.markdown("""

# <style>

# html, body, [class*="css"] {

#     font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;

# }

# .stApp {

#     background: linear-gradient(135deg, #0a0e1a 0%, #0d1b2a 40%, #0a1628 70%, #060d16 100%);

#     background-attachment: fixed;

# }

# header[data-testid="stHeader"] { background: transparent !important; }

# .block-container { padding-top: 1.5rem !important; max-width: 1200px; }

```

```

# [data-testid="stSidebar"] {

#   background: rgba(10,20,40,0.95) !important;

#   border-right: 1px solid rgba(0,200,150,0.15) !important;

# }

# [data-testid="stSidebar"] * { color: #e0e0e0 !important; }

# h1 { color: #ffffff !important; font-weight: 800 !important; }

# h2, h3, h4 { color: #e0e0e0 !important; }

# p, li, label { color: #b0b0b0 !important; }

# .stSelectbox > div > div {

#   background: rgba(255,255,255,0.06) !important;

#   border-color: rgba(255,255,255,0.15) !important;

#   border-radius: 10px !important;

# }

# .stButton > button { border-radius: 10px !important; font-weight: 600 !important; }

# .stButton > button[kind="primary"] {

#   background: linear-gradient(135deg,#00C896,#00a077) !important;

#   border: none !important; color: #fff !important; padding: 12px 20px !important; #

# }

# hr { border-color: rgba(255,255,255,0.06) !important; }

# .metric-card {

#   background: rgba(255,255,255,0.04);

#   border: 1px solid rgba(255,255,255,0.08);

#   border-radius: 12px;

#   padding: 20px 16px;

#   text-align: center;

# }

```

```

# .metric-icon { font-size: 26px; margin-bottom: 8px; }

# .metric-label { font-size: 10px; color: #666; text-transform: uppercase;

#         letter-spacing: 1.5px; margin-bottom: 6px; }

# .metric-value { font-size: 26px; font-weight: 700; color: #fff; }

# .metric-desc { font-size: 9px; color: #555; margin-top: 5px; line-height: 1.4; }

# .explain-card {

#     background: rgba(255,255,255,0.03);

#     border: 1px solid rgba(255,255,255,0.07);

#     border-radius: 10px;

#     padding: 14px 16px;

#     margin-bottom: 10px;

# }

# </style>

# "", unsafe_allow_html=True)

#

# OUTPUT_DIR      = "output"

# GAF_DIR          = os.path.join(OUTPUT_DIR, "gaf_images")

# PREPROCESSOR_DIR = os.path.join(OUTPUT_DIR, "preprocessors")

# UAT_PATH         = os.path.join(OUTPUT_DIR, "uat_results.csv")

#

# LABEL_MAP = {"Low": 0, "Medium": 1, "High": 2}

# INV_LABEL = {0: "Low", 1: "Medium", 2: "High"}

# CLASS_NAMES = ["Low", "Medium", "High"]

# COLORS      = {"Low": "#2ECC71", "Medium": "#F39C12", "High": "#E74C3C"}

# IMG_SIZE    = 64

# WINDOW      = 252

```

```

#
# RISK_EXPLAIN = {
#   "Low": (
#     " Low Risk",
#     "This company demonstrates strong ESG practices combined with stable price history. "
#     "It shows low annualised volatility, positive risk-adjusted returns, and responsible "
#     "environmental, social, and governance behaviour. The GAF image reflects a relatively "
#     "smooth and consistent price trend over the past year. "
#     "Suitable for investors seeking stable, sustainable, long-term holdings."
#   ),
#   "Medium": (
#     " Medium Risk",
#     "This company shows moderate ESG performance with some variability in its price history. "
#     "It balances reasonable returns with moderate exposure to ESG-related financial risks. "
#     "The GAF image indicates periodic price fluctuations but no extreme volatility. "
#     "Generally considered a stable investment but warrants ongoing monitoring of ESG indicators."
#   ),
#   "High": (
#     " High Risk",
#     "This company exhibits elevated financial volatility and/or weaker ESG scores, indicating "
#     "higher exposure to environmental, social, or governance-related risks. The GAF image "
#     "reflects significant price fluctuations over the past year. "
#     "Investors should exercise caution — this classification signals that the company may face "
#     "heightened regulatory, reputational, or market risks in the near term."
#   ),
# }

```

```

#

# METRIC_CONFIG = [

#   ("environmentScore", " ", "ENV SCORE", "Carbon, waste & resource use"),
#   ("socialScore", " ", "SOCIAL", "Labour, community & supply chain"),
#   ("governanceScore", "🏢", "GOV SCORE", "Board structure & shareholder rights"),
#   ("beta", " ", "BETA", "Volatility vs. S&P 500 market"),
#   ("ann_vol", " ~", "VOLATILITY", "Annualised std dev of daily returns"),
#   ("price_sharpe", " ", "SHARPE", "Return earned per unit of risk"),
# ]

#

# METRIC_EXPLAIN = {

#   "ENV SCORE": (" Environmental Score",
#                 "Measures a company's management of environmental risks — carbon emissions, "
#                 "energy efficiency, water use, and waste. A <b>higher score</b> means better "
#                 "environmental practices and lower exposure to climate-related financial risk."),
#   "SOCIAL": (" Social Score",
#              "Reflects how well a company manages relationships with employees, suppliers, "
#              "customers, and communities. Covers labour rights, workplace safety, diversity, "
#              "and community impact. <b>Higher = stronger social responsibility.</b>"),
#   "GOV SCORE": ("🏢 Governance Score",
#                 "Assesses the quality of corporate governance — board independence, executive "
#                 "pay transparency, audit practices, and shareholder rights. <b>Higher scores</b> "
#                 "indicate lower risk of fraud or mismanagement."),
#   "BETA": (" Beta",
#            "Measures how much a stock moves relative to the overall market. "
#            "<b>Beta > 1</b> = more volatile than the market. ")

```

```

#         "<b>Beta < 1</b> = more stable. A Beta of 0.72 means the stock moves "
#         "about 72% as much as the S&P 500."),
#     "VOLATILITY": ("~~ Annualised Volatility",
#         "The annualised standard deviation of daily price returns. It quantifies how "
#         "much the stock price fluctuates over a year. "
#         "<b>Higher volatility = higher uncertainty = higher financial risk.</b>"),
#     "SHARPE": (" Sharpe Ratio",
#         "Measures risk-adjusted return: how much return the stock earns per unit of "
#         "risk taken. A Sharpe Ratio <b>above 1.0</b> is generally considered good. "
#         "<b>Higher is better</b> — it means more return for the same level of risk."),
# }

#

# @st.cache_resource(show_spinner="Loading models...") #
def load_models():
#     import tensorflow as tf
#     from tensorflow.keras import models as km
#     xgb_model = joblib.load(os.path.join(OUTPUT_DIR, "xgb_hybrid.pkl"))
#     res_model = km.load_model(os.path.join(OUTPUT_DIR, "resnet50_gaf.keras"))
#     imp_h     = joblib.load(os.path.join(PREPROCESSOR_DIR, "num_imputer_h.pkl"))
#     scl_h     = joblib.load(os.path.join(PREPROCESSOR_DIR, "scaler_h.pkl"))
#     feat_cols = joblib.load(os.path.join(PREPROCESSOR_DIR, "feature_cols_h.pkl"))
#     feature_extractor = km.Model(
#         inputs = res_model.input,
#         outputs = res_model.get_layer("res_bn").output,
#         name   = "res_embedder",
#     )

```

```

# return xgb_model, feature_extractor, imp_h, scl_h, feat_cols
#
# @st.cache_data(show_spinner="Loading company data...") #
def load_data():
# return pd.read_csv(os.path.join(OUTPUT_DIR, "merged_tabular.csv"))
#
# def make_gasf(close_prices, size=IMG_SIZE):
#     idx = np.linspace(0, len(close_prices)-1, size).astype(int)
#     s = close_prices.astype(np.float64)[idx]
#     mn, mx = s.min(), s.max() #
#
#     if mx == mn:
#         return np.zeros((size, size), dtype=np.float32)
#     s = np.clip(2.0*(s-mn)/(mx-mn) - 1.0, -1.0, 1.0)
#     phi = np.arccos(s)
#     return np.cos(np.add.outer(phi, phi)).astype(np.float32) #
# def to_rgb(img):
#     if img.ndim == 2:
#         img = img[:, :, np.newaxis]
#     return np.concatenate([img, img, img], axis=-1)[np.newaxis] #
# def predict(ticker, merged_df, xgb_model, feat_extractor, imp_h, scl_h, feat_cols):
#     row = merged_df[merged_df["ticker"] == ticker] #
#
#     if row.empty:
#         return None, None, None, None
#
#     tab_raw = row[feat_cols].values.astype(float)
#     tab_proc = scl_h.transform(imp_h.transform(tab_raw))

```

```

#   gaf_path = os.path.join(GAF_DIR, f'{ticker}.npy') #
if not os.path.exists(gaf_path):
#       return None, None, None, None
#   img  = np.load(gaf_path)
#   emb  = feat_extractor.predict(to_rgb(img), verbose=0)
#   X    = np.hstack([tab_proc, emb])
#   proba = xgb_model.predict_proba(X)[0]
#   HIGH_IDX = LABEL_MAP["High"]
#   lm_map  = [i for i in range(3) if i != HIGH_IDX]
#   threshold = 0.333
#   if proba[HIGH_IDX] >= threshold:
#       pred = HIGH_IDX
#   else:
#       pred = lm_map[int(np.argmax(proba[lm_map]))]
#   return INV_LABEL[pred], proba, img, tab_proc
#
# @st.cache_resource(show_spinner="Building SHAP explainer...") #
def get_shap_explainer(_xgb_model):
#   return shap.TreeExplainer(_xgb_model)
#
# def shap_plot(explainer, X, feat_cols, emb_dim, pred_class):
#   feat_names = list(feat_cols) + [f'res_emb_{i}' for i in range(emb_dim)]
#   sv  = explainer.shap_values(X)
#   sv_arr = np.array(sv)
#   sv_cls = sv_arr[:, :, pred_class] if sv_arr.ndim == 3 else sv_arr
#   top_idx  = np.argsort(np.abs(sv_cls[0]))[-15:][::-1]

```

```

# top_names = [feat_names[i] for i in top_idx]

# top_values = sv_cls[0][top_idx]

# fig, ax = plt.subplots(figsize=(8, 5))

# colors = ["#E74C3C" if v > 0 else "#3498DB" for v in top_values]

# ax.barh(range(len(top_names)), top_values[::-1], color=colors[::-1], edgecolor="white")

# ax.set_yticks(range(len(top_names)))

# ax.set_yticklabels(top_names[::-1], fontsize=9)

# ax.axvline(0, color="black", lw=0.8)

# ax.set_xlabel("SHAP value (impact on prediction)")

# ax.set_title("Top 15 features driving this prediction", fontweight="bold")

# plt.tight_layout()

# return fig

#

# def render_metric_cards(row):

#     cards = [

#         (icon, label, desc, row[key])

#         for key, icon, label, desc in METRIC_CONFIG

#         if key in row.index and pd.notna(row[key])

#     ]

#     if not cards:

#         return

#

#     st.markdown("""<p style='font-size:11px;color:#00C896;text-transform:uppercase;

#         letter-spacing:2px;font-weight:600;margin-bottom:12px;'>

#         KEY METRICS</p>""", unsafe_allow_html=True)

#

```

```

# for i in range(0, len(cards), 3):
#     chunk = cards[i:i+3]
#     cols = st.columns(3)
#     for col, (icon, label, desc, val) in zip(cols, chunk):
#         col.markdown(f"""
#             <div class='metric-card'>
#                 <div class='metric-icon'>{icon}</div>
#                 <div class='metric-label'>{label}</div>
#                 <div class='metric-value'>{val:.2f}</div>
#                 <div class='metric-desc'>{desc}</div>
#             </div>""", unsafe_allow_html=True)
#
# st.markdown("<div style='height:20px'></div>", unsafe_allow_html=True)
#
# st.markdown("""<p style='font-size:11px;color:#00C896;text-transform:uppercase;
#         letter-spacing:2px;font-weight:600;margin-bottom:12px;'>
#         WHAT DO THESE METRICS MEAN?</p>""", unsafe_allow_html=True)
#
# for i in range(0, len(cards), 2):
#     chunk = cards[i:i+2]
#     cols = st.columns(2)
#     for col, (icon, label, _, val) in zip(cols, chunk):
#         if label in METRIC_EXPLAIN:
#             title, explanation = METRIC_EXPLAIN[label]
#             col.markdown(f"""
#                 <div class='explain-card'>

```

```
#         <p style='margin:0 0 6px 0;font-size:13px;font-weight:700;color:#e0e0e0;'>{title}</p>

#         <p style='margin:0;font-size:12px;color:#888;line-height:1.7;'>{explanation}</p>

#     </div>""", unsafe_allow_html=True)

#

# #

=====

# # UI

# #

=====

=====

#

# st.markdown("""

# <div style='margin-bottom:8px;'>

#   <h1 style='font-size:32px;margin:0;padding:0;'>ESG Stock Risk Intelligence</h1>

#   <p style='color:#666;font-size:14px;margin-top:6px;'>

#       Hybrid ResNet50 Image Embeddings + XGBoost &nbsp;&middledot;&nbsp;  S&P 500

#       &nbsp;&middledot;&nbsp;  3-Class Risk Classification

#   </p>

# </div>

# <hr style='border-color:rgba(255,255,255,0.06);margin-bottom:24px;'>

# """, unsafe_allow_html=True)

#

# try:

#     xgb_model, feat_extractor, imp_h, scl_h, feat_cols = load_models()

#     merged_df = load_data()

#     emb_dim  = feat_extractor.output_shape[-1] #

except Exception as e:
```

```

# st.error(f'Failed to load artefacts: {e}\n\nMake sure the output/ folder is in the same directory as
app.py.')

# st.stop()

#

# with st.sidebar:

# st.markdown("""

# <div style='text-align:center;padding:20px 0 10px 0;'>

# <div style='font-size:36px;'> </div>

# <div style='font-size:18px;font-weight:700;color:#fff;margin-top:8px;'>ESG Risk
Classifier</div>

# <div style='font-size:11px;color:#00C896;letter-spacing:2px;margin-top:4px;'>GROUP 3
&middot; DATA SCIENCE</div>

# </div>

# <hr style='border-color:rgba(255,255,255,0.08);margin:16px 0;'>

# """ , unsafe_allow_html=True)

#

# available = sorted([

# str(t) for t in merged_df["ticker"].tolist()

# if os.path.exists(os.path.join(GAF_DIR, f'{t}.npz'))

# ])

#

# ticker = st.selectbox("Ticker", available)

#

# row = merged_df[merged_df["ticker"] == ticker].iloc[0]

# st.markdown("---")

# st.markdown(f'***Sector:** {row.get('GICS Sector', row.get('Sector', 'N/A'))}')

# st.markdown(f'***ESG Risk Level (actual):** {row.get('ESG Risk Level', row.get('risk_label',
'N/A'))}')

```

```

# if "beta" in row:

#     st.markdown(f'***Beta:** {row["beta"]:.2f}' if pd.notna(row["beta"]) else "***Beta:** N/A")

# if "marketCap" in row: #     mc = row["marketCap"]

#     if pd.notna(mc):

#         st.markdown(f'***Market Cap:** ${mc/1e9:.1f}B")

#

# run = st.button(" Predict Risk", use_container_width=True, type="primary")

#

# st.markdown("""

# <hr style='border-color:rgba(255,255,255,0.06);margin:24px 0 12px 0;'/>

# <div style='font-size:10px;color:#444;text-align:center;line-height:1.8;*>

#     KPI 7 & middot; Latency & lt; 3s & nbsp;|& nbsp; KPI 8 & middot; UAT & ge; 4.0/5<br>

#     KPI 9 & middot; 100% SHAP Coverage

# </div>

#     """, unsafe_allow_html=True)

#

# if run:

#     with st.spinner("Running prediction..."):

#         prediction, proba, gaf_img, tab_proc = predict(

#             ticker, merged_df, xgb_model, feat_extractor, imp_h, scl_h, feat_cols

#         )

#

# if prediction is None:

#     st.error(f"No GAF image found for {ticker}.")

#     st.stop()

#

```

```

# col_left, col_right = st.columns([1, 2])

#

# with col_left:

#     color = COLORS[prediction]

#     st.markdown(f"""

#         <div style="background:{color}22; border-left:6px solid {color};

#             border-radius:12px; padding:24px; text-align:center; margin-bottom:12px;">

#             <p style="margin:0; font-size:11px; color:{color}; font-weight:600;

#                 text-transform:uppercase; letter-spacing:2px;">PREDICTED RISK</p>

#             <p style="margin:8px 0; font-size:56px; font-weight:800; color:{color};">{prediction}</p>

#             <p style="margin:0; font-size:13px; color:#888;">{ticker}</p>

#         </div>

#         """, unsafe_allow_html=True)

#

#     # Risk level explanation

#     r_title, r_text = RISK_EXPLAIN[prediction]

#     st.markdown(f"""

#         <div style="background:rgba(255,255,255,0.03); border:1px solid rgba(255,255,255,0.08);

#             border-radius:10px; padding:14px 16px;">

#             <p style="margin:0 0 6px 0; font-size:13px; font-weight:700; color:#e0e0e0;">{r_title}</p>

#             <p style="margin:0; font-size:12px; color:#888; line-height:1.7;">{r_text}</p> #

#         </div>

#         """, unsafe_allow_html=True)

#

# with col_right:

#     st.markdown("""<p style='font-size:11px;color:#00C896;text-transform:uppercase;

```

```

#         letter-spacing:2px;font-weight:600;margin-bottom:12px;'> #
CONFIDENCE SCORES</p>""", unsafe_allow_html=True) #         for i, cls
in enumerate(CLASS_NAMES):
#         pct = float(proba[i]) * 100
#         col_a, col_b = st.columns([1, 4])
#         col_a.markdown(f'***{cls}***')
#         col_b.progress(int(pct), text=f'{pct:.1f}%')
#
#     st.markdown("---")
#     render_metric_cards(row)
#     st.markdown("---")
#
#     col_gaf, col_shap = st.columns(2)
#
#     with col_gaf:
#         st.markdown("""<p style='font-size:11px;color:#00C896;text-transform:uppercase;
#             letter-spacing:2px;font-weight:600;margin-bottom:12px;'>
#             GAF IMAGE — PRICE HISTORY</p>""", unsafe_allow_html=True)
#         fig_gaf, ax_gaf = plt.subplots(figsize=(5, 5))
#         ax_gaf.imshow(gaf_img, cmap="RdYlGn", vmin=-1, vmax=1,
#             origin="lower", aspect="auto")
#         ax_gaf.set_title(f'{ticker} — 252-day GASF', fontweight="bold")
#         ax_gaf.set_xticks([]); ax_gaf.set_yticks([])
#         plt.colorbar(ax_gaf.images[0], ax=ax_gaf, fraction=0.046, pad=0.04)
#         plt.tight_layout()
#         st.pyplot(fig_gaf)

```

```

# plt.close(fig_gaf)

#

# with col_shap:

#         st.markdown("""<p style='font-size:11px;color:#00C896;text-transform:uppercase; #
letter-spacing:2px;font-weight:600;margin-bottom:12px;'>

#             SHAP — FEATURE CONTRIBUTIONS</p>""", unsafe_allow_html=True)

#     explainer = get_shap_explainer(xgb_model)

#     X_full = np.hstack([tab_proc, feat_extractor.predict(to_rgb(gaf_img), verbose=0)])

#     fig_shap = shap_plot(explainer, X_full, feat_cols, emb_dim, LABEL_MAP[prediction])

#     st.pyplot(fig_shap)

#     plt.close(fig_shap)

#

# else:

#     st.markdown("""

#         <div style="text-align:center;padding:60px 20px;">

#             <div style="font-size:64px;margin-bottom:16px;"> </div>

#             <div style="font-size:22px;font-weight:700;color:#fff;margin-bottom:10px;">

#                 Select a company and click Predict Risk

#             </div>

#             <div style="font-size:14px;color:#555;max-width:500px;margin:0 auto;line-height:1.7;">

#                 The hybrid model combines ResNet50 GAF image embeddings with ESG and financial features

#                 to classify S&P 500 companies into Low, Medium, or High risk.

#             </div>

#         </div>

#     """, unsafe_allow_html=True)

#

```

```

# results_path = os.path.join(OUTPUT_DIR, "model_results.csv") #
if os.path.exists(results_path):

#     st.markdown("#### Model Performance Summary")

#     res_df = pd.read_csv(results_path)

#         display_df = res_df[["model", "weighted_f1", "accuracy", "auc_roc",
# "high_recall"]].rename(columns={

#         "model": "Model", "weighted_f1": "F1 (weighted)",

#         "accuracy": "Accuracy", "auc_roc": "AUC-ROC",

#         "high_recall": "High-Risk Recall"

#     })

#     for c in ["F1 (weighted)", "Accuracy", "AUC-ROC", "High-Risk Recall"]:

#         display_df[c] = display_df[c].map("{:.4f}".format)

#     st.table(display_df)


!pip install streamlit==1.32.0 -q


import subprocess subprocess.run(['pkill', '-f', 'streamlit'],
capture_output=True) print("All Streamlit processes killed
— now rerun Cell 23")


import subprocess, time, os, urllib.request


# Pin Streamlit to the version that works subprocess.run(['pip', 'install', 'streamlit==1.32.0', '-q'],
capture_output=True) subprocess.run(['pip', 'install', 'protobuf==5.28.3', '--force-reinstall', '-q'],
capture_output=True)

```

```
IN_COLAB = 'COLAB_GPU' in os.environ or 'COLAB_RELEASE_TAG' in os.environ if
```

```
IN_COLAB:
```

```
    os.system('npm      install      -g      localtunnel      2>/dev/null')
```

```
subprocess.Popen(
```

```
    ['streamlit', 'run', 'app.py',
```

```
    '--server.port', '8501',
```

```
    '--server.headless', 'true',
```

```
    '--server.enableCORS', 'false',
```

```
    '--server.enableXsrfProtection', 'false'],
```

```
    stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
```

```
)
```

```
    time.sleep(6)                                ip =
```

```
urllib.request.urlopen('https://ipv4.icanhazip.com').read().decode().strip()
```

```
print(f'    Tunnel password: {ip}\n')            tunnel = subprocess.Popen(['npx',
```

```
'localtunnel', '--port', '8501'],                stdout=subprocess.PIPE,
```

```
stderr=subprocess.DEVNULL, text=True)          for line in tunnel.stdout:        if
```

```
'localt' in line or 'tunnel' in line.lower():
```

```
    print(f'    App URL: {line.strip()}')        print(' → Click
```

```
link, enter IP as password, click Submit\n')        break else:
```

```
    print('Run: streamlit run app.py\nThen open: http://localhost:8501')
```

```
print('KPI 7 — models cached via @st.cache_resource → load < 3 sec after first run') print('KPI
```

```
8 — UAT ratings saved to output/uat_results.csv')
```

Additional Figures

Screenshots of the developed Streamlit application, including the prediction interface and output dashboard.

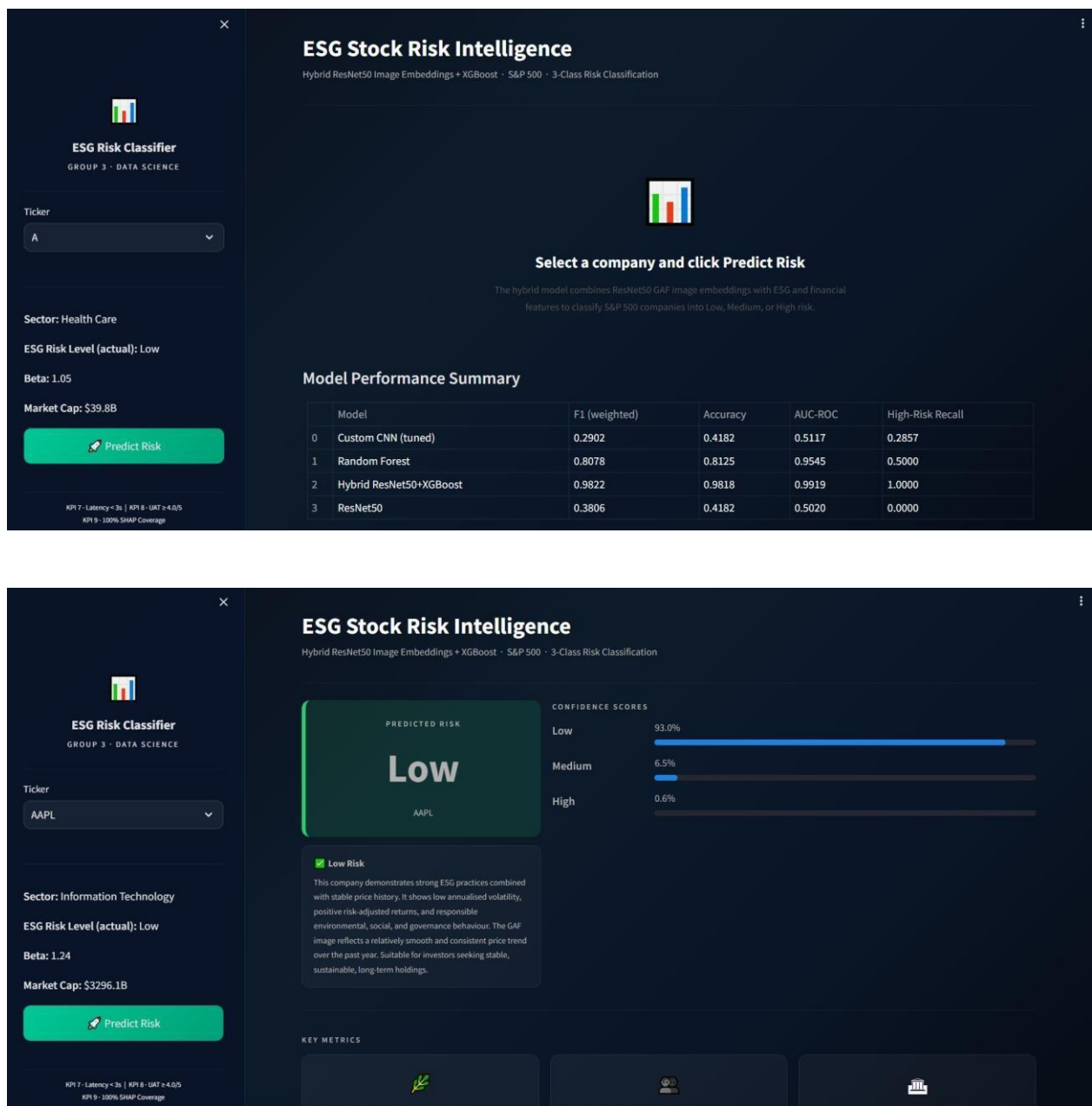
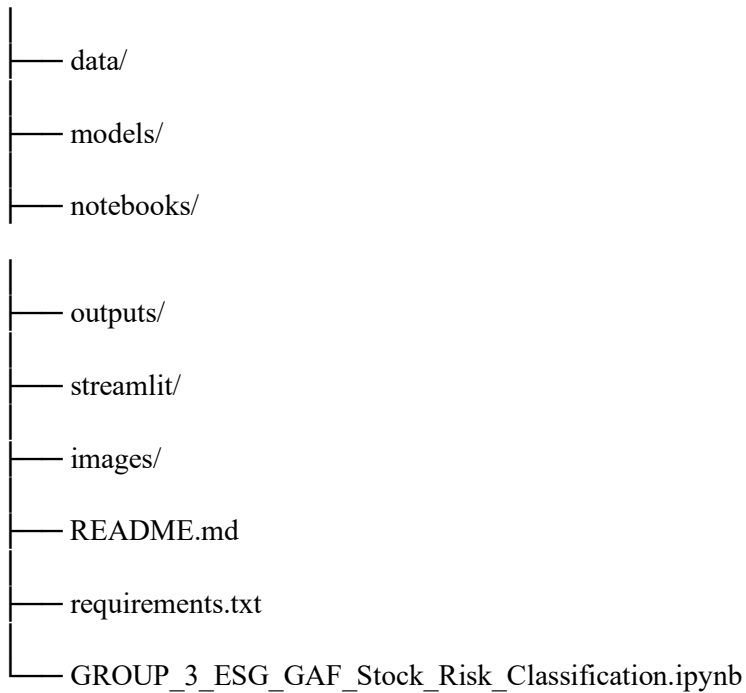


Figure A10. Streamlit Application Dashboard And Predict Risk

Repository Structure

GROUP_3_ESG_GAF_Stock_Risk_Classification/



Software Environment

Software Version Python

3.x

TensorFlow 2.x

Scikit-learn Latest

XGBoost Latest

Optuna Latest

SHAP Latest

Streamlit Latest

Pandas Latest

NumPy Latest

Project Outputs

The following outputs are included in the GitHub repository:

- Model checkpoints
- Saved preprocessing objects
- Confusion matrices
- ROC curves
- Precision–Recall curves
- SHAP visualisations
- KPI dashboard
- Streamlit screenshots
- Classification reports

- Final trained models
- Evaluation metrics

Appendix D: Team GitHub Repository

The project was developed and maintained using a private GitHub repository throughout the project lifecycle to support collaborative development, version control, and reproducible research. The repository has been shared with the project supervisor and assessment team in accordance with the module requirements.

The final Week 11 repository contains the submitted Group Project Report, the complete Notebook implementing the ESG Stock Risk Classification pipeline, and all supporting artefacts required to reproduce the reported results. The notebook includes dataset integration, exploratory data analysis, GAF image generation, feature engineering, CNN modelling, Random Forest classification, Optuna hyperparameter tuning, ResNet50 transfer learning, hybrid ResNet50–XGBoost modelling, SHAP explainability, KPI dashboard generation, and the Streamlit application used for model demonstration.

The repository also includes trained model files, generated figures, evaluation reports, experiment logs, application source code, and a README document describing the project objectives, software requirements, repository structure, installation procedure, and execution steps. Maintaining the project through GitHub enabled efficient collaboration between team members, preserved the complete development history, and ensured that the final implementation and reported results can be reproduced consistently.